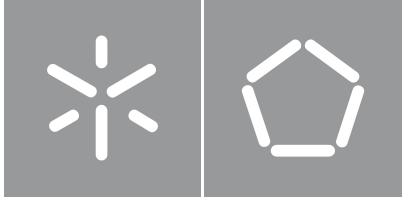
University of Minho
School of Engineering

Tiago Miguel Moreira Bacelar Pereira

Simulation of Hybrid Systems via Exact Real Arithmetic

september 2026



University of Minho
School of Engineering

Tiago Miguel Moreira Bacelar Pereira

Simulation of Hybrid Systems via Exact Real Arithmetic

Master's Dissertation in Informatics Engineering

Dissertation supervised by
Renato Neves

september 2026

Copyright and Terms of Use for Third Party Work

License granted to users of this work:

Acknowledgements

This work is financed by National Funds through FCT - Fundação para a Ciência e a Tecnologia, I.P. (Portuguese Foundation for Science and Technology) within project IBEX, with reference 10.54499/PTDC/CCI-COM/4280/2021, and within project Banksy, with reference COMPETE2030-FEDER-00892000.

I want to thank my supervisor, Renato Neves, for his rigor with my work and his patience with me. This dissertation wouldn't be possible without your guidance.

I also want to thank my friends, who I'm so lucky to always have by my side. To everyone who stayed with me throughout this journey: I couldn't have done this without your kindness.

And lastly, to my family, who gave me the blinding light of their dreams. The composure you taught me will follow me all my life. I couldn't face this challenge without your example.

Though the path is long and winding, only by walking it can you reach new horizons. So do not fear the rain, for no matter how bad it gets, you will be warm again. Do not fear your heart, for as much as it hurts it is wonderful to feel. And above all else, do not fear your scars, for, despite everything, it's still you.

Statement of Integrity

I hereby declare having conducted this academic work with integrity.

I confirm that I have not used plagiarism or any form of undue use of information or falsification of results along the process leading to its elaboration.

I further declare that I have fully acknowledged the Code of Ethical Conduct of the University of Minho.

University of Minho, Braga, september 2026

Tiago Miguel Moreira Bacelar Pereira

Abstract

Like most programs, simulations of hybrid systems mostly use floating-point arithmetic, which can introduce errors and lead to false conclusions, especially in sensitive and critical systems. This dissertation aims to fix this problem through the use of exact real arithmetic in hybrid programs. Drawing from previous research on computable real numbers backed by domain theory, as well as a monadic formalization of hybridness, a new hybrid simulator Jaguar is developed in Haskell using preexisting exact real arithmetic libraries. The greatest challenges in development are the semidecidability of comparison between computable reals and the implementation of a differential equation solver. As expected, the final product is overall slower than equivalent programs using floating-point; but there are interesting performance differences between various exact real arithmetic libraries due to their different strategies and characteristics, as explored in the benchmarking. These results are doubtlessly of interest from the perspective of designing new performant exact real arithmetic libraries, and Jaguar as a standalone tool has applications in (small) physics simulations, formal verification and more.

Keywords exact real arithmetic, hybrid systems, Jaguar, computable real numbers

Resumo

Tal como a maioria dos programas, as simulações de sistemas híbridos utilizam maioritariamente aritmética de vírgula flutuante, o que pode introduzir erros e levar a conclusões incorretas, especialmente em sistemas críticos ou sensíveis. Esta dissertação procura resolver este problema através da utilização de aritmética real exata em programas híbridos. Com base em investigação existente sobre números reais computáveis baseada em teoria de domínios, bem como numa formalização monádica da natureza híbrida destes sistemas, foi desenvolvido um novo simulador híbrido em Haskell chamado Jaguar, recorrendo a bibliotecas pré-existentes de aritmética real exata. Os maiores desafios durante o desenvolvimento foram a semidecidibilidade da comparação entre números reais computáveis e a implementação de um solver de equações diferenciais. Como seria de esperar, o produto final é, de um modo geral, mais lento do que programas equivalentes que usam aritmética de vírgula flutuante. No entanto, as bibliotecas de aritmética real exata apresentam entre si diferenças de desempenho, resultantes das suas diferentes estratégias e características, tal como demonstrado no benchmarking. Estes resultados são relevantes para o desenvolvimento de futuras bibliotecas de aritmética real exata de alto desempenho. Além disso, o Jaguar, enquanto ferramenta autónoma, tem aplicações em simulações físicas de pequena escala, verificação formal e outras áreas.

Palavras-chave aritmética real exata, sistemas híbridos, Jaguar, números reais computáveis

Contents

List of Figures

List of Tables

Acronyms

CPO Complete Partial Order.

ERA Exact Real Arithmetic.

ODE Ordinary Differential Equation.

IVP Initial Value Problem.

Glossary

real number/real An element of $\mathbb{R}$, which can be defined as the completion of $\mathbb{Q}$ under Cauchy limits.

computable real number/computable real A computational object that denotes a real number.

exact real arithmetic Arithmetic on computable reals.

computable real representation strategy/computable real representation A mathematical formalization of the concept of computable real (*e.g.* nested intervals representation, modulus of convergence representation).

computable real implementation A concrete implementation of a computable real representation strategy, usually offering functionality like arithmetic, comparison, etc (*e.g.* the packages analyzed in this dissertation).

computable real approximation A real value, or range of real values, produced by a computable real number to approximate the real number it denotes. Usually restricted to $\mathbb{Q}$ or some subset of $\mathbb{Q}$.

CompReal A Haskell typeclass defined in this dissertation that abstractly encapsulates the concept and functionality of a computable real.

CompReal instance A computable real implementation that instantiates the *CompReal* typeclass. All 5 analyzed packages had a *CompReal* instance written for them.

Chapter 1

Introduction

1.1 Context and Motivation

Hybrid systems are commonly defined as processes that possess a mix of both continuous evolution, usually modeled through differential equations, and discrete behaviors, such as conditionals, assignments and other instantaneous changes [1]. This concept proves invaluable, especially when simulating the interaction of physical, continuous processes with software-controlled systems, such as cruise controllers and pacemakers. The resulting programs are known as *hybrid programs*.

For example, consider a cruise controller. To implement it, we can write a simple program that checks the velocity of a vehicle every second and either accelerates or brakes, eventually hovering around the target velocity. The following program, associated to such a controller, was implemented in Lince [2], an imperative hybrid programming language and simulator:

```
1 v:=4;
2 while true do {
3     if v<=10 then v'=1 for 1; else v'=-1 for 1;
4 }
```

Listing 1.1: A simplistic model of a cruise controller

In detail, the program starts by assigning the value 4 to variable v (the vehicle's velocity). The statement inside the while loop is associated to either accelerate or break, depending on the current velocity, by setting the derivative of v, v', to 1 or -1, during 1 second. The while loop indicates that this behavior repeats indefinitely. Fig. 1 is a plot of the simulated velocity over time. Despite simple, this example illustrates the basic concepts of a hybrid program, namely the presence of both classical assignment, control structures, etc. as well as differential, continuous constructs and their relation to time.

Among the many difficulties of this paradigm, one is the often overlooked matter of precision and error. When developing proper methods of simulation, the standard way to represent numbers is floating point

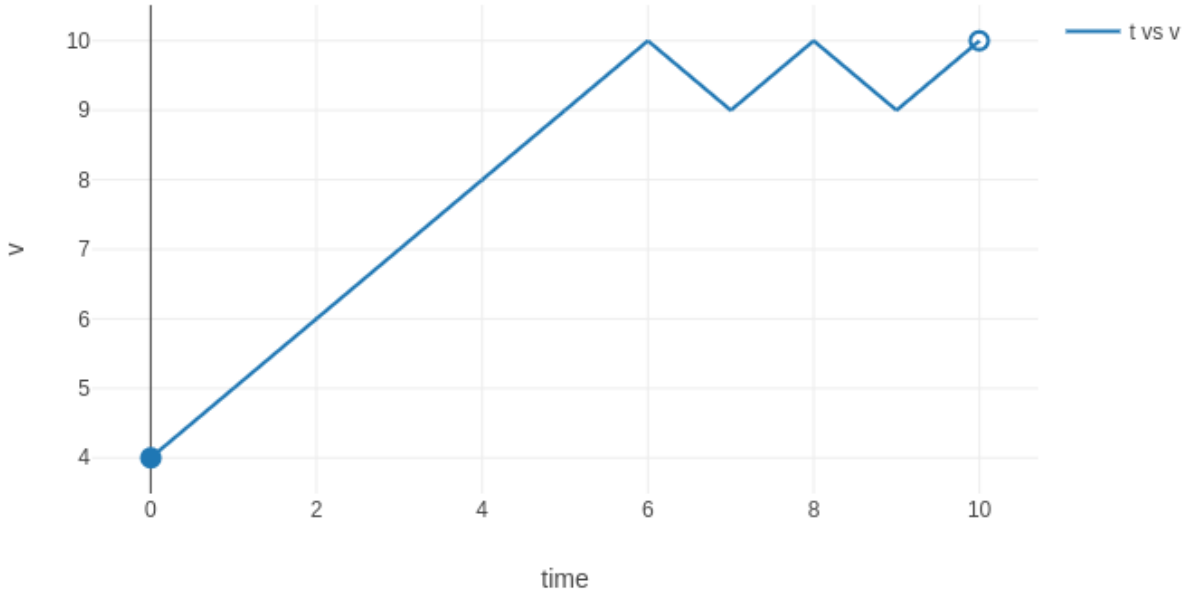


Figure 1: Plot of the velocity over time in the cruise control program

arithmetic, which treats values as a mantissa and an exponent. This concept can be, and sometimes is, used to represent numbers with an arbitrary precision by using a large size for the mantissa and exponent; but it is usually implemented according to the IEEE 754 standard [3], which defines *double-precision floating-point numbers* as having 53 bits for the mantissa and 11 for the exponent, for a total of 64 bits. This is enough for most applications, but especially in chaotic systems, where errors compound, the available precision can be quickly exhausted, and the accuracy of the obtained results is compromised. Lince itself uses double-precision floating-points, abbreviated as doubles in most programming languages, which makes it prone to errors when the values get too small. Take the following program as an example.

```
1 x:=1;
2 x'=-x for 80;
3 x'= x for 80;
```

Listing 1.2: Hybrid program showcasing insufficient precision

Fig. 2 is a plot of the simulation of variable x over time. Variable x was expected to go from 1 to a very small number (e^{-80}) at time $t = 80$ following a negative exponential (e^{-t}), and then follow a positive exponential (e^{t}) afterwards and return to value 1 at time $t = 160$. Lince, however, shows the final value of x as 5.653719, clearly an error. Since x becomes extremely small during the simulation, it must be rounded to fit in a double during the intermediate steps, resulting in the error. This error grows even larger if we replace 80 with larger values.

Using an arbitrary number of digits to represent values (arbitrary precision floating point), and/or ratio

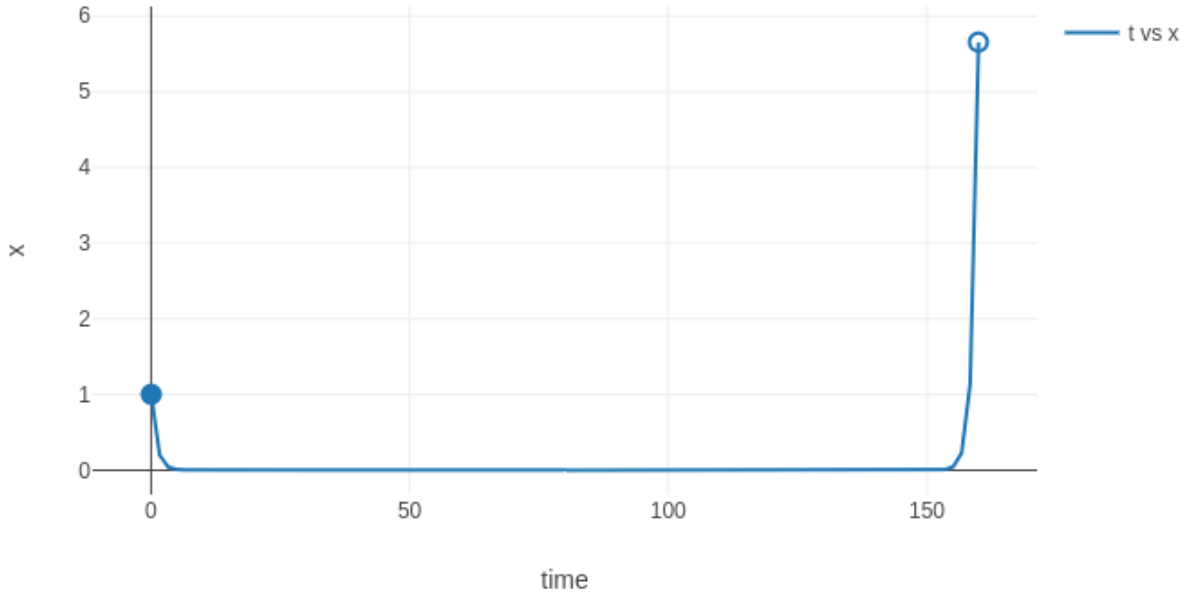


Figure 2: Plot of x over time in the exponential program

representations (numerator and denominator), allows for the exact representation of all rational numbers, but still leaves open the problem of irrational numbers, which are common when simulating physical systems (when using trigonometric functions, for instance, or with the previous exponential example). In the previous example, using arbitrary precision floating point numbers would be impossible since the value of the system is irrational along the trajectory. Unlike their rational counterparts, irrational numbers would take an infinite number of digits to represent using a regular digit representation. This problem therefore calls for alternative solutions.

1.2 Contributions

The greatest contribution of this work is the creation of a hybrid system simulator using exact real arithmetic, which was named Jaguar [4]. Jaguar uses preexisting exact real arithmetic libraries, so simulator performance is expected to be worse than simulators using floating point arithmetic, such as the aforementioned tool Lince [2], but should be perfectly faithful to the real system simulated. Jaguar's specification language is based on the previous while language with specific constructs to describe the behavior of physical processes. This choice is due to the simplicity and power of the language, which furthermore has a well-established semantics used as a basis for implementation. The simulator is implemented in the Haskell programming language [5; 6] due to its lazy paradigm and use of higher-order functions, which are particularly useful in the context of exact real arithmetic.

The theoretical component of this dissertation is mostly about compiling and systematizing already existing research from the fields of exact real arithmetic and hybrid systems. There is a wealth of research on exact real arithmetic, mostly on its theoretical aspects (a good summary of which can be found in [7]). At a more practical level, this dissertation looks at representation strategies based on sequences of approximations (*i.e.* from the point of view of implementation and performance), how they are related, and their (dis)advantages.

Original theoretical contributions are mostly limited to what was needed to implement the hybrid simulator. Due to fundamental limitations of exact real arithmetic, the simulator requires some solutions and workarounds to combine the two technologies. The main limitation to overcome is the semidecidability of inequality in exact real arithmetic, which makes comparisons hard to handle. As a workaround, the simulator uses partial comparison, up to some accuracy. This forces operations on hybrid structures to be written as parametrized operations, taking in as a parameter the accuracy of the partial comparisons.

The other big challenge to overcome is implementing a differential equation solver that produces an exact real value as its output. Numerical methods for solving differential equations, like the famous Runge-Kutta [8], only generate approximations of the solution, within an asymptotic error rate; and despite being able to reach arbitrarily good approximations (by lowering the step size, for instance), they don't provide numerical error bounds for their results, which are needed to transform a sequence of approximations into a real value. The solution presented in the current work (Section 3.5) starts by transforming the equations into a system of polynomial equations. The solution of the system is then given as a Taylor series, obtained with forward automatic differentiation [9], and a fixed step size to calculate the approximations (which can be arbitrarily improved by taking more and more elements of the Taylor series). The error bound of the results is calculated by following [10].

On the practical side, this dissertation is a comprehensive exposition of the inner workings and functionality of the developed simulator, Jaguar. Beyond the features already discussed, Jaguar is as generic as to support all libraries for exact real arithmetic discussed in this dissertation (as well as double-precision floating-point numbers) and new ones can be easily added in the future. Jaguar is also highly configurable, allowing users to choose several parameters of both the simulation and the queries. It also comes with a built-in visualizer based on a plotting library called Chart [11].

1.3 Document Structure

After this introduction, Chapter 2 gathers the technical background relevant for this project. It first delves into an exploration of existing formalizations and techniques for representing real numbers (Section 2.1). A study of several open-source libraries follows in Section 2.2, where the underlying approaches are compared and critiqued. Then Section 2.3 lays out the theory behind hybrid systems and presents the underlying challenges of their simulation.

Chapter 3 discusses the implementation of the simulator (architecture overview in Section 3.1, followed by the parser in Section 3.2, interpreter in Section 3.4, solver in Section 3.5 and visualizer in Section 3.6). It also discusses the challenges found along the way and their respective solutions, along with other notable implementation details.

In Chapter 4 the performance of each exact real arithmetic library and of the simulator is discussed.

Finally Chapter 5 details possible directions concerning future work.

Assumptions and Notation

Throughout this dissertation, the reader is assumed to be familiar with partial orders, limits, derivatives, Taylor polynomials, Taylor series, and monads [12] [13]. Functors will be represented with an upper-case sans-serif letter (*e.g.* M). If the functor is part of a monad, the latter is represented by the same letter in bold font weight (*e.g.* $\mathbf{M}$). Monad transformers are also represented in bold sans-serif font and end in a capital T (*e.g.* **StateT**). Vectors are represented as $\vec{x}$ ($\vec{x_0}$ for vectors with a subscript) and the i-th component of a vector as x_i (or $(x_0)_i$ if the vector already had a subscript). The set of natural numbers is represented by $\mathbb{N}$ and is understood to consist of all non-negative integers ($0 \in \mathbb{N}$). $\mathbb{R}_0^+$ represents the set of non-negative reals. $\overline{\mathbb{R}_0^+}$ denotes the extended non-negative reals (*i.e.* $\mathbb{R}_0^+$ extended with the infinity value ∞). $[0, \mathbb{R}_0^+)$ is the set of all intervals of the form $[0, d), d \in \mathbb{R}_0^+$ ($\emptyset \in [0, \mathbb{R}_0^+)$). Finally, $[0, \overline{\mathbb{R}_0^+})$ denotes the set of all intervals of the form $[0, d], d \in \mathbb{R}_0^+$ or $[0, d), d \in \overline{\mathbb{R}_0^+}$.

Chapter 2

State of the Art

2.1 The Theory of Exact Real Arithmetic

Since the dawn of Computer Science, scientists have studied the problem of representing real numbers. One of the first to touch on the subject was Alan Turing himself, in his now famous paper *On Computable Numbers, with an Application to the Entscheidungsproblem* [14]. In it, he uses Turing machines to formalize the concept of "computable numbers", the class of real numbers that can be represented computationally (see Section 2.1.5 for more details). Since then, several results in the area have expanded this concept, and multiple strategies were developed to represent real numbers. At the same time, the implementations of these concepts slowly developed. As available computational resources grew, more and more sophisticated strategies became viable. Cutting-edge approaches moved from floating-point arithmetic to arbitrary precision floating points (like GNU's MPFR library [15] written in C), which allow for arbitrarily small errors but still require explicit error management, and eventually to more advanced techniques that manage errors automatically (such as *iRAAM* [16], an exact real arithmetic library written in C++).

This dissertation is based on this last step. Among the many known techniques, the easiest to analyze and implement are based on sequences of approximations, like the aforementioned *iRAAM*. The mathematical underpinning of these strategies (Cauchy sequences and Cauchy completeness) is presented in Section 2.1.1. As will be shown, a naive implementation of this concept is flawed, which is why it is refined into workable strategies using interval arithmetic, motivated by domain theory (Section 2.1.2), and modulus of convergence (Section 2.1.3).

After presenting these strategies, Section 2.1.4 presents the formal definitions of precision and accuracy from numerical analysis and how they are used in the context of exact real arithmetic. These concepts will be fundamental in later sections of the dissertation, where concrete implementations of exact real representations are compared. Lastly, Section 2.1.5 summarizes everything said thus far and

lists some limitations common to all representation strategies.

2.1.1 Cauchy Sequences

There are multiple formal definitions of real numbers, but the most used definition for exact real number representation is based on Cauchy sequences. Concisely: the set of real numbers $\mathbb{R}$ is the smallest set that contains the rationals and is closed under limits of Cauchy sequences. A Cauchy sequence is defined as a sequence whose terms get progressively close to each other, converging to a single number. More formally, a sequence of real numbers $\{a_i\}_{i\in\mathbb{N}}$ is a Cauchy sequence if

$$\forall\epsilon > 0, \exists i : \forall j > i, |a_i - a_j| < \epsilon \tag{2.1}$$

So a straightforward way to represent a real number is simply as a sequence of rationals. Since the rational numbers are a dense subset of the reals, any real number can be approximated by a Cauchy sequence of rationals, or indeed by any dense subset of the rationals, such as the dyadic numbers (numbers of the form $a/2^p$, with $a, p \in \mathbb{Z}$).

Inspired by this concept, one might be tempted to build an implementation where real numbers are computationally described by an infinite list of rationals or dyadics. We might call such an implementation the plain or naive Cauchy representation of real numbers. However, there is a big problem with such an implementation. To illustrate the issue, consider the following sequence (which is clearly a Cauchy sequence since we can choose $i = 1000000$ for any ϵ to satisfy Eq. (2.1)):

$$\begin{cases} a_i = 0 & \text{if } i < 1000000 \\ a_i = 2 & \text{if } i \geq 1000000 \end{cases} \tag{2.2}$$

Despite converging to the number 2, the sequence has a value of 0 for the first 1000000 terms. And, of course, there is nothing magical about the number 1000000: we can build a similar Cauchy sequence that only "starts converging" after an arbitrarily large index, thus demonstrating the well-known fact that the convergence of Cauchy sequences is invariant under finite perturbations. In short, even if a sequence is known to be Cauchy, it is impossible to know which number it is converging to by looking only at a finite number of its terms. If one wanted to generate an approximation of the limit with an error of, for instance, $\epsilon < 0.1$, an infinite number of terms would have to be consulted. Because any algorithm that uses this sequence, by definition of algorithm, can only ever take into account a finite number of terms, Cauchy sequence representations cannot make any guarantees on the error of any result they produce.

Though insufficient *per se*, Cauchy sequences can still be used as a basis for other representation methods. The next sections offer improvements to this basic idea.

2.1.2 Domain theory and Interval Arithmetic

Domain theory [17] deals with sets equipped with a complete partial order (cpo, here represented as $\sqsubseteq$), often called domains. Intuitively, this order measures the information content of each element of the domain. Each element of a domain can be thought of as a (partial) result of a computation, and one element is greater than another if it extends its information content in a consistent way. Like any partial order, cpos must be reflexive, transitive and antisymmetric. Additionally, every chain $a_0 \sqsubseteq a_1 \sqsubseteq \ldots$ of the domain must have a least upper bound in the domain. Informally, this upper bound subsumes all information in the chain.

For example, take the kleeneans ($K_3 = \{T, F, U\}$), the 3 possible valuations of a proposition in Kleene's three-valued logic, first introduced in his paper *On notation for ordinal numbers* [18]. This domain is of special interest to us since it will form the basis for the evaluation of propositions in the simulator (Section 3.3.2). The kleeneans can be understood as a domain $(K_3, \sqsubseteq)$ where its elements are related as follows:

$$a \sqsubseteq b \iff a = U \vee a = b$$

This relation (illustrated in Fig. 3) represents the fact that we may have no information (U) about the value of a proposition, or have mutually exclusive information (*i.e.* the proposition is T or F).

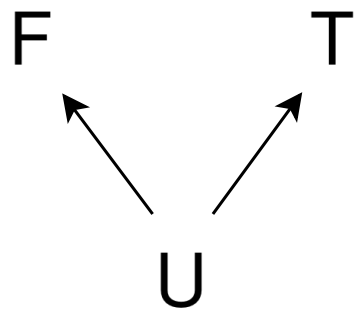


Figure 3: Hasse diagram of the domain of the kleeneans (K_3). Each arrow $a \to b$ represents the relationship $a \sqsubseteq b$. Reflexive relationships are omitted.

Another example of a domain, also of interest later in the dissertation, is the ordering domain, which extends the 3 possible outcomes of comparing two real numbers (LT: $x < y$, EQ: $x = y$ and GT: $x > y$;

equivalently, these 3 values may be thought of as the sign of $y - x$) into a domain, where each element of the domain represents a possible state of knowledge about the comparison of the two reals, as shown in Fig. 4.

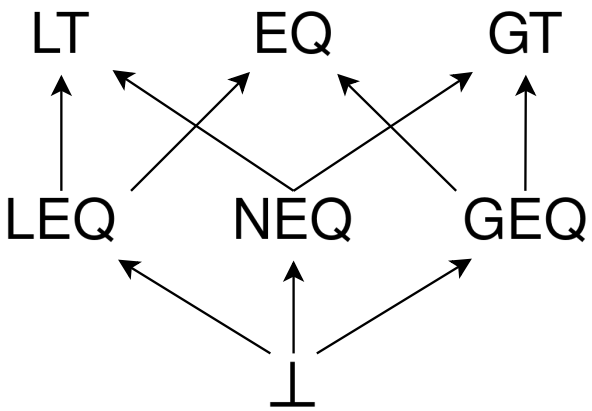


Figure 4: Hasse diagram of the ordering domain. Each arrow $a \to b$ represents the relationship $a \sqsubseteq b$. Relationships inferrable through reflexivity or transitivity are omitted.

Domain theory is the basis of denotational semantics of programming languages, since it models the idea of computational information. A computation can be thought of as a continuous[1] function, that is, a monotonic function f such that, for any chain $\{a_i\}_{i \in \mathbb{N}}$ of the domain:

$$\bigsqcup_{i \in N} f(a_i) = f(\bigsqcup_{i \in \mathbb{N}} a_i)$$

where $\bigsqcup$ denotes the least upper bound. Another recurring concept in domain theory is the bottom element $\bot$ of a domain D, defined as the least element of D. It denotes no information or a never-ending computation. In the previous examples, both domains have a bottom element (the bottom element of the kleeneans is represented as U instead of $\bot$ to avoid confusion with the usual notation for valuations of a proposition, $\top$ or $\bot$).

To represent real numbers, it is interesting to work specifically with the domain of closed real intervals (optionally extended with infinity), the reason for which will shortly become apparent. We define the domain $(I, \sqsubseteq)$ as:

$$[w, x] \sqsubseteq [y, z] \iff [w, x] \supseteq [y, z]$$

from which we can derive

[1] Not to be confused with analytical continuity. In this section, unless otherwise stated, "continuous" refers to the domain-theoretical notion of continuity relating to information and computability.

$$\bigsqcup_{i\in\mathbb{N}} a_i = \bigcap_{i\in\mathbb{N}} a_i$$

This means that an interval has greater information content than another if it is contained in it. In this domain, the bottom element $\bot$ is an interval that contains all others, so it must be $(-\inf, +\inf)$. On the other hand, a maximal element is an interval with zero width, that is, an interval of the form $[x, x]$, since the only interval contained in it is itself.

So how can this domain be used to represent real numbers? Consider a chain of these intervals:

$$[-2, 3] \sqsubseteq [-2, 1] \sqsubseteq [-1, 0.5] \sqsubseteq [0.1, 0.3] \sqsubseteq [0.1, 0.1] \tag{2.3}$$

Like the Cauchy sequences in the previous section, which converge to their limit, this chain of intervals "converges" to its least upper bound, which in this specific example is its last element, $[0.1, 0.1]$ (though note that in an infinite chain, the least upper bound may not be an element of the chain). However, unlike Cauchy sequences, each step in the chain actually constricts the upper bound, since it must be contained by all intervals. Each interval gives information about the result of the chain, so the problem with the naive Cauchy representation is avoided. Then, generating an approximation with an arbitrarily small ϵ can be achieved by iterating the chain until an interval with a width smaller than $\epsilon/2$ is found.

The real power of these chains, though, is how they formalize exact real arithmetic with continuous functions. If one wanted to know, for instance, the square of the least upper bound of this chain, they could first get the upper bound (in this case $[0.1, 0.1]$) and then square it, to obtain $[0.01, 0.01]$. But this is only possible if the chain is finite and small enough to fully iterate in a reasonable amount of time. Fortunately, there is another way: since squaring an interval is a continuous function, squaring each element in the chain must result in another chain, and the upper bound of this new chain must equal the interval we want, $[0.01, 0.01]$:

$$[0, 9] \sqsubseteq [0, 4] \sqsubseteq [0, 1] \sqsubseteq [0.01, 0.09] \sqsubseteq [0.01, 0.01] \tag{2.4}$$

Technically, in order to reach the interval $[0.01, 0.01]$ we would have to iterate just as many elements as to reach the upper bound of the first chain — after all, the chains have the same length. But the point is that each interval of the new chain still contains information about its upper bound, and iterating it yields arbitrarily good approximations of 0.1^2 even if the end of the chain is never reached.

Hopefully the core idea of the nested intervals representation is now evident: a real number x should be represented by a chain $\{a_i\}_{i\in\mathbb{N}}$ of nested rational intervals such that the least upper bound of the chain is $[x, x]$ [19]. Calculating the output $g(x)$ of some real function $g : \mathbb{R} \to \mathbb{R}$ is achieved by first finding

a continuous function $f : I \to I$ that generalizes g to intervals and then applying it to each element of the chain, where "generalize" is defined as having the property $\forall x \in \mathbb{R}, f([x, x]) = [g(x), g(x)]$. Due to the properties of continuous functions, it is guaranteed that the resulting chain $b_i = f(a_i)$ represents the result we want, $g(x)$:

$$\bigsqcup_{i\in\mathbb{N}} b_i = \bigsqcup_{i\in\mathbb{N}} f(a_i) = f\left(\bigsqcup_{i\in\mathbb{N}} a_i\right) = f([x, x]) = [g(x), g(x)] \tag{2.5}$$

As a small aside, it is important to keep in mind that operating on an interval (interval arithmetic) is, in general, harder than operating simply on its endpoints. While that is the case for monotonic operations on real numbers, such as addition and subtraction (e.g. the sum of two intervals is the interval whose endpoints are the sum of the terms' endpoints), in general it may be necessary to consider the whole interval (in the previous example with squaring intervals, notice how $[-2, 3]^2 = [0, 9]$ and not $[4, 9]$).

This concept can also be extended to multivariable functions, such as binary operations. Instead of squaring, now consider the function $g(x, y) = x+y$ and its continuous generalization $f([w, x], [y, z]) = [w+y, x+z]$. If we want to calculate $g(0.1, 0.01)$ using the two chains of the previous example (Eq. (2.3) and Eq. (2.4)), we can create a new chain by zipping them together with f (where a "zip" is defined as $c_i = f(a_i, b_i)$ and indexing past the end of a finite chain returns the last element), which results in the following:

$$[-2, 12] \sqsubseteq [-2, 5] \sqsubseteq [-1, 1.5] \sqsubseteq [0.11, 0.39] \sqsubseteq [0.11, 0.11]$$

As expected, the least upper bound of the new chain is the result we wanted, $[0.11, 0.11]$. Technically, we don't *have* to use a zip; any method of combining the two chains that increases monotonically in index and reaches arbitrarily large indices also produces a chain converging to the wanted result. For example, matching every element in a_i with every even-indexed element in b_i (*e.g.* $c_i = f(a_i, b_{2i})$) results in

$$[-2, 12] \sqsubseteq [-2, 2] \sqsubseteq [-0.99, 0.51] \sqsubseteq [0.11, 0.31] \sqsubseteq [0.11, 0.11]$$

This works because b_{2i}, the chain consisting of all even-indexed members of b_i, has the same least upper bound as b_i:

$$\bigsqcup_{i\in\mathbb{N}} b_{2i} = \bigsqcup_{i\in\mathbb{N}} b_i$$

And in general, any such reindexing will preserve the least upper bound. But do keep in mind that reindexing can heavily impact performance, both positive and negatively. In the previous example, if the

second chain, b_i, was so slow to calculate that obtaining its i-th element was twice as slow as obtaining the same index of a_i, then combining them as $c_i = f(a_i, b_{2i})$ could produce a faster-converging chain than a zip. Of course, if the reverse was true, that same method of combining them would be much slower. In general, optimizing operations this way would require prior knowledge of the performance of each chain and the width of their intervals; without prior knowledge, there is no reason to treat the two chains differently. The zip is the simplest possible combination method and also treats both chains equally, so that is what is used in practice. Considerations around reindexing and performance are fundamental to analyze operation performance in Section 4.1, so keep them in mind for later.

Finally, the domain-theoretical formalization also works between different domains: if instead of a real function we were interested in the output of a predicate $g : \mathbb{R} \to 2$ and we could find a continuous generalization $f : I \to K_3$, applying it to the chain of intervals would result in a chain of kleeneans whose least upper bound is the boolean result we want. Unlike intervals, since the kleeneans are a finite set, the least upper bound of a chain of kleeneans must be an element of the chain. So if a chain of kleeneans has a lower bound of T or F it must contain T or F, which we are guaranteed to eventually reach by iterating the chain.

While the theory is sound, it is not always possible to find continuous generalizations of g. One such situation is (in)equality. In order to generalize a predicate $g(x) := x = 0$[2], the generalization f would have to satisfy both $f([0,0]) = T$ and $\forall x \neq 0, f([x,x]) = F$, which assuming that f is continuous implies that $\forall x > 0, f([0,x]) = U$. But this allows the construction of a chain $a_i = [0, 2^{-i}]$ that contradicts the continuity of f:

$$\bigsqcup_{i\in\mathbb{N}} f\left([0,2^{-i}]\right) = \bigsqcup_{i\in\mathbb{N}} U = U \neq T = f([0,0]) = f\left(\bigsqcup_{i\in\mathbb{N}} [0,2^{-i}]\right)$$

Therefore, by contradiction, equality (and inequality, by similar reasoning) cannot be generalized to a continuous f. In general, any analytically discontinuous function g (which includes all non-constant predicates $g : \mathbb{R} \to 2$ and real functions like $g(x) = sign(x)$) will run into this problem. However, for equality[2] at least, it is possible to choose an f that generalizes g everywhere other than the discontinuity:

$$\begin{cases} f(a) = U & \text{iff } 0 \in a \\ f(a) = F & \text{otherwise} \end{cases} \tag{2.6}$$

Then, if f is applied to a chain representing some real $x \neq 0$, the result is a chain whose least upper

[2] Obviously equality is a binary operation, but the point can be made with a single-variable function that determines equality with a specific number, in this case 0. Proving that the two-variable equality also cannot be generalized is trivial after realizing that g is the partial application of equality to 0.

bound is F, as expected. If it is applied to a chain representing 0, the upper bound will be U, which is equivalent to saying the computation will never terminate. This is a necessary consequence of the fact that equality in the computable reals is co-semidecidable (or, equivalently, that inequality is semidecidable; see Section 2.1.5 for more details). That being said, if an actual implementation is willing to forsake the mathematical rigor of domain theory, it is possible to use a non-continuous f' that generalizes equality but can still return T if the interval is exactly $[0, 0]$:

$$\begin{cases} f'(a) = T & \text{iff } a = [0,0] \\ f'(a) = U & \text{iff } a \neq [0,0] \wedge 0 \in a \\ f'(a) = F & \text{otherwise} \end{cases} \tag{2.7}$$

f' is consistent with the definition of f in Eq. (2.6) ($f(a) \sqsubseteq f'(a)$), so any result obtained with f is also obtained with f'. The only difference between the two is that f' "squeezes out" a little more information in the specific situation that $[0, 0]$ is an element of the chain. This will be relevant when discussing how the hybrid simulator implements comparisons in Section 3.3.1.

2.1.3 Modulus of Convergence

Attentive readers may have realized that the chains of intervals in the previous section may not represent a real number, because their least upper bound is not required to be a zero-width interval; any width of interval is allowed. Let us call chains whose least upper bound is a zero-width interval "convergent chains". Only convergent chains represent real numbers, while non-convergent chains are an unfortunate byproduct of using the domain of intervals to represent the reals. Let us also call continuous generalizations of real functions "proper functions". As we have seen, applying a proper function to a convergent chain results in a chain which is also convergent (recall Eq. (2.5)). So even though most chains don't represent a real number (and even though checking if a chain does represent one may never terminate), a constructive approach that starts with a set of convergent chains and only combines them using proper functions will only produce convergent chains, which can then be iterated to obtain arbitrarily good approximations.

That being said, it can be inconvenient to use the domain of the intervals to reason about the real numbers, since there are many chains that don't converge. There is also a lot of freedom in what initial set of convergent chains to use (only integers *versus* dyadic numbers *versus* all rationals), how those chains are generated (*e.g.* the number 0.5 can be both represented as $[0.5, 0.5] \sqsubseteq [0.5, 0.5] \sqsubseteq \ldots$ and $[0, 1] \sqsubseteq [0.5, 1] \sqsubseteq [0.5, 0.75] \sqsubseteq \ldots$) and in how they are operated on and combined. This freedom is good for optimizing actual implementations, but can lead to unexpected or unintuitive results when using

them. For instance, when testing exact real arithmetic libraries that use the nested intervals representation, I often obtained much tighter intervals than the accuracy I requested. This is because those libraries use an initial set of very fast-shrinking chains of intervals, in an attempt to optimize operations.

These are all issues that the modulus of convergence representation avoids. The motivation for this representation strategy comes directly from the study of Cauchy sequences, more specifically from the concept of the modulus of convergence. It is defined as follows: for a given Cauchy sequence $\{a_i\}_{i\in\mathbb{N}}$, the modulus of convergence is a function $\alpha : \mathbb{N} \to \mathbb{N}$ that satisfies the following relation:

$$\forall n, i, j \in \mathbb{N}, \alpha(n) < i < j \implies |a_i - a_j| \leq 2^{-n} \tag{2.8}$$

Informally, for an arbitrary n, the modulus of convergence gives us a position on the sequence such that all future terms differ less than 2^{-n} from each other, therefore implying that all of these terms have an error of at most 2^{-n} from the limit. If such a function exists, it is possible to prove that the sequence follows Eq. (2.1) and therefore converges.

Adding a modulus of convergence to the naive Cauchy representation makes it viable because the modulus bounds the error of the sequence. For example, if we wanted to obtain an approximation with an error $\epsilon \leq 0.001$, we would first find a smaller power of two, in this case 2^{-10}, calculate the index of the sequence that guarantees this error, $\alpha(10)$, and then obtain the approximation at that index, $a_{\alpha(10)}$.

To avoid treating the sequence and its modulus separately, an exact real number representation may enforce that its sequences follow a fixed modulus of convergence $\alpha(n) = n$, to which any convergent sequence can be converted via its modulus of convergence: $b_n = a_{\alpha(n)}$. This reindexing produces a sequence of approximations where the n-th approximation has an error of at most 2^{-n}. In practice, all implementations of modulus of convergence representation use this simplification, and so when discussing the modulus of convergence representation in the rest of this dissertation it is implied that the modulus is fixed.

For all its simplicity, the modulus of convergence representation has a few problems. The biggest issue is that there is no simple way to generalize real operations to sequences with modulus. Instead, each operation has to be implemented on a case by case basis such that the resulting sequence also follows a fixed modulus of convergence. For an example see Section 2.2.3, where the implementation of addition is examined. Since these implementations may require reindexing, they have important performance implications. If the reindexing happens to require larger indexes from the faster sequence and smaller from the slower, it can be faster than the zip used by nested intervals representation, like we have seen; but on most cases it will be slower. This exact problem is explored in detail in Section 4.1 using complexity

analysis.

Another downside of this strategy is that, unlike intervals, which can exactly represent rationals with a zero-width interval, an approximation in a modulus of convergence representation can never exactly represent any value since every term of the sequence has the non-zero error bound defined in Eq. (2.8). Therefore, in a situation where an exact approximation could be produced (*e.g.* representing an integer number, like 2), nested intervals will always have an advantage, since their representation ($[2,2] \sqsubseteq [2,2] \sqsubseteq \dots$) has all information available from the very first term of the sequence. Contrast this with modulus of convergence, where generating an approximation with an error $\epsilon \leq 0.001$ will always require iterating at least ten terms of the sequence (recall we are using a fixed modulus by convention, thus $\alpha(10) = 10$).

To conclude, note that a Cauchy sequence $\{a_i \in \mathbb{R}\}_{i \in \mathbb{N}}$ equipped with any modulus of convergence $\alpha : \mathbb{N} \to \mathbb{N}$ can be converted into a valid sequence of nested intervals $\{b_n \in I\}_{n \in \mathbb{N}}$, since the modulated terms of the sequence effectively form an interval of width $2/2^n$:

$$\begin{cases} b_0 = [a_{\alpha(0)} - 1, a_{\alpha(0)} + 1] \\ b_n = [a_{\alpha(n)} - 2^{-n}, a_{\alpha(n)} + 2^{-n}] \cap b_{n-1} \quad \text{if } n > 0 \end{cases}$$

Note that the intersection with b_{n-1} is needed to enforce the nestedness of the resulting chain of intervals. For an example where omitting the intersection would result in non-nested intervals, check the following Cauchy sequence with a fixed modulus that converges to 1:

$a_0 = 0$

$a_1 = 0.5$

$a_2 = 1.25$

...

This conversion makes nested intervals a sort of intermediate step between the basic Cauchy sequences with no modulus, the least restrictive representation (so nonrestrictive, in fact, that it is insufficient), and sequences with a modulus, which are more well-behaved. For example, to represent the number 2, a plain Cauchy sequence has literally no restrictions on any finite number of its elements (see the example at Eq. (2.2)); a sequence of nested intervals forces all its intervals to contain the number 2 but does not restrict the width of the intervals in any way; and a modulus of convergence has an implicit error bound of $\pm 2^{-n}$ for its n-th term.

2.1.4 Precision vs Accuracy

Since these two terms are used interchangeably in day-to-day language, it may be helpful to revisit their formal definition in numerical analysis and how they are used in exact real arithmetic in particular. We will be using these concepts often when analyzing approximations of real values; since our representation strategies are based on sequences of approximations, the accuracy and precision of these approximations are important to keep in mind.

Accuracy refers to how close the result of a computation or approximation is to the mathematical true value being computed, measured by the absolute or relative error of the approximation. In this dissertation, accuracy is denoted by n [3] and is defined logarithmically in terms of the absolute error ϵ:

$$n = \lfloor -log_2(\epsilon) \rfloor \tag{2.9}$$

This quantity corresponds to the modulus of convergence presented in the previous section; specifically, it corresponds to the index after which approximations in a Cauchy sequence get below an error of 2^{-n}. For sequences with a fixed modulus, the term with index n is an approximation with accuracy n. But we can also extend the concept to nested intervals. If the accuracy n of a certain result or approximation $\hat{x}$ is known, we can construct an interval of width $2/2^n$ of all possible values for the true value x:

$$\hat{x} - 2^{-n} \leq x \leq \hat{x} + 2^{-n}$$

Conversely, if an interval of width δ is known to contain some true value x, then its middle point is guaranteed to be an approximation of x with an error of at most $\delta/2$, which corresponds to an accuracy of

$$n = \lfloor 1 - log_2(\delta) \rfloor \tag{2.10}$$

So having an interval/range of values for some true value x is largely equivalent to having an approximation with a known accuracy/error, which is why we can consider intervals as approximations.

Let us look at a concrete example. Take $50/16 = 3.125$ as an approximation of π. This approximation has an error of $\epsilon = |\pi - 50/16| = 0.016592\ldots$, from which we derive its accuracy of $n = \lfloor -log_2(0.016592\ldots) \rfloor = 5$. This means that in a Cauchy sequence converging to π that follows a constant modulus of convergence ($\alpha(n) = n$), this approximation could be used up to index 5 (*i.e.* the

[3] As implied by the choice of symbol, n is treated as a natural number ($n \in \mathbb{N}$). Although it wouldn't be nonsensical to consider negative accuracies as well (that is, absolute errors greater than 1), in this dissertation we will only consider natural n since we are basing the definition of accuracy on the index of a Cauchy sequence with a fixed modulus, which starts at 0.

first six terms). From this accuracy we can also infer that the true value of π must be inside the interval $[50/16 - 2^{-5}, 50/16 + 2^{-5}] = [3.09375, 3.15625]$; hence, this interval is also an approximation of π with an accuracy of 5. Of course, this example is, in a sense, "backwards"; we used the actual value of π to calculate the error (and then accuracy) of our approximation. In reality, the reason why we generate approximations is to approach an unknown true value, and so the error has to be inferred from the nature of the operations performed to obtain the approximation. This process of error inference is an important part of numerical analysis, and extremely relevant to us here as well: we want to generate error bounds as tight as possible for the amount of computational resources expended, in order to maximize performance.

Precision, on the other hand, is the resolution of the result's representation (*i.e.* how close are two consecutive representable numbers), usually given as the number of decimal or binary digits. More formally, *Accuracy and Stability of Numerical Algorithms* [20] defines precision as "the accuracy with which the basic arithmetic operations $+, -, *, /$ are performed"; in other words, precision measures the error that one basic operation introduces. So for a single basic operation, accuracy and precision coincide, but over multiple operations the errors add up and accuracy may decrease sharply. These two definitions seem contradictory, but they are in fact pointing to the same concept: the bigger the binary representation of the number, the more accurate each operation can be, since the gaps between consecutive representable numbers are smaller.

Consider dyadic numbers as an example: for a fixed p and any integer a, we can build an arithmetic within the set of dyads of the form $a/2^p$. If the result of an operation lands outside the set, we round to the closest dyad in the set. Like in floating-point arithmetic, these operations do not form a field since the rounding breaks associativity and distributivity, but we can still use the dyadics to obtain approximations of computations. Addition and subtraction can always be performed without rounding ($a/2^p + b/2^p = (a + b)/2^p$), but in the case of multiplication and division errors may be as large as $1/2^p$ depending on how the rounding is performed. Therefore, this arithmetic may be said to have a precision of p if rounding to zero (or $p + 1$ if rounding to nearest). This is similar to how precision is usually discussed in computer science in the context of floating-point numbers, but instead of using the usual definition of accuracy calculated from the relative error, we use the absolute error. Floating-point minimizes the relative error caused by the gaps between consecutive floating-point numbers, while dyadic numbers minimize the absolute error.

While precision is important when discussing the development and inner workings of an algorithm or calculation, end users are usually more interested in a result's accuracy, or, equivalently, on an error bound for the result. Consider the aforementioned example of $50/16 = 3.125$ as an approximation for π.

If it is interpreted as the dyadic number $50/2^4$, then this approximation has a precision of $p = 4$. However, notice that $51/2^4$, $-42/2^4$, or indeed *any* dyadic number with the same denominator also has a precision of 4, even though they aren't remotely close to π. Even though $50/2^4$ is the *best* approximation with precision 4, it is far from the only one. That being said, it is true that *because* it is the best approximation with precision 4, $50/2^4$ will necessarily have an accuracy of at least 5 (since being the best implies that $\epsilon \leq 0.5/2^p$ and therefore $n \geq p + 1$ by our definition in Eq. (2.9)); in this case, the accuracy of $50/2^4$ is exactly 5 like we saw above. Achieving higher accuracy may require higher precision, but accuracy is the real goal. Precision is just a means to that end.

That being said, there is a very tangible effect of precision on the final result of a computation: speed. Operating on higher precision values takes longer than lower precision ones. How much longer depends on the operation: addition and subtraction can be computed with linear complexity $\Theta(p)$; meanwhile, multiplication and division algorithms can have any complexity between $O(p^2)$ (a naive "schoolbook" multiplication/division) and $O(p \log p)$ (using Harvy-Hoeven multiplication [21] and Newton-Raphson division [22]). So an efficient algorithm should try to use the minimum precision necessary to achieve the target accuracy, otherwise it may suffer performance-wise.

2.1.5 Summary

To summarize, all presented methods of exact real representation rely on sequences of increasingly accurate approximations. The simplest representation strategy uses Cauchy sequences, which by themselves are insufficient since they don't make accuracy guarantees. Two refinements are possible: using sequences of shrinking intervals to keep track of the accuracy of the approximations at each step (nested intervals representation); and keeping a modulus of convergence, or following a fixed modulus, to make guarantees about both the accuracy of the approximations and the speed of their convergence.

There are a few theoretical quirks to any exact real representation. Perhaps most surprisingly, not all real numbers can be computationally described. This appears to contradict what has been said so far – that any real number is described by a rational Cauchy sequence. However, infinite sequences cannot be stored directly in a computation device with a finite memory. Instead, to represent an infinite sequence, a procedure or function that generates said sequence is used, which means many sequences (and by extension many real numbers) are impossible to represent. This can be proved independently of the concept of real numbers as sequences: since there are countably many programs (seeing as each program is a finite, though arbitrarily long, sequence of symbols chosen from a finite set), they can only represent countably many real numbers, no matter the representation strategy. Since there are

uncountably many real numbers, not all of them can be represented. The set of representable reals is referred to in the literature as the computable real numbers (sometimes also called recursive real numbers due to their alternative definition using recursion theory [23]).

The observation that computable reals are countable was made by Alan Turing in 1937 [14] and since then the subject has been highly explored in computable analysis. Among other interesting results: despite being countable, the computable reals were shown to not be computationally/recursively enumerable (*i.e.* there is no algorithm that is guaranteed to enumerate all computable reals such that each one is reached in a finite time) [23].

Another frustrating limitation is the fact that the equality of two computable real numbers is co-semidecidable, *i.e.* no algorithm can reliably confirm that any two given equal computable reals are indeed equal. Consequently, comparing two computable reals may never terminate if the two reals are equal. For this reason, practical implementations of the presented ideas offer workarounds or alternatives to comparison, such as performing comparison up to some specified accuracy.

2.2 Implementations: from Theory to Practice

This section is dedicated to studying existing implementations of exact real arithmetic in the Haskell programming language [5; 6]. We start with an abstract view of these implementations' features and some technical considerations specific to Haskell. Then, each package's representation strategy is analyzed, listing their capabilities and respective pros and cons. For the benchmarks of the packages' performance see Chapter 4.

2.2.1 The CompReal class

Each of the surveyed packages implements the concepts discussed in the previous sections (approximations, computable real numbers, etc.) as their own datatypes. However, for the purposes of testing, benchmarking and later developing the hybrid simulator, it is useful to group them into the same Haskell type class, which was named *CompReal* (short for "computable real"). This type class abstractly represents the concept of a computable real number, and the functionalities it defines, pictured in Fig. 5, are presented in the rest of this section. However, it shouldn't be more restrictive that it needs to be. For example, there is no requirement that reals are represented uniquely; multiple objects of an instance of *CompReal* may represent the same real number.

So what operations should our *CompReals* implement? For a start, since our theory is based on

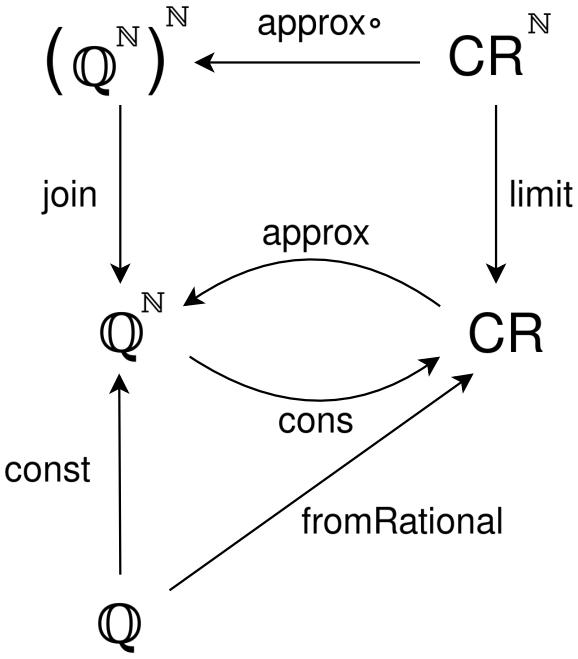


Figure 5: Core functionality of *CompReal* (abbreviated as CR). All arrows preserve the real number being represented, so the diagram commutes after mapping values into $\mathbb{R}$

sequences of approximations, our computable reals should support being approximated up to a certain accuracy. These approximations should be *Rationals* to generalize any particular approximation datatype the libraries might be using. In Fig. 5 this functionality is provided by function *approx*. Formally, some *CompReal* r represents a real number x iff for all $n \in \mathbb{N}$

$$|approx\ r\ n - x| \leq 2^{-n}$$

For example, take a computable real r representing π whose internal representation is the following list of nested intervals $\{a_i\}_{i\in\mathbb{N}}$:

$a_0 = [3, 4]$

$a_1 = [3.1, 3.2]$

$a_2 = [3.14, 3.15]$

$a_3 = [3.141, 3.142]$

$a_4 = [3.1415, 3.1416]$

...

Recall how approximations are calculated in nested intervals representations: by iterating the list until a good enough interval is found. Evaluating $approx\ r\ 6$ should result in an approximation with an accuracy of at least 6 (which, recall from Eq. (2.9), means we want an approximation with an error $\epsilon < 2^{-6} =$

0.015625 or, equivalently, an interval of width $2^{-5} = 0.03125$), so a_0 and a_1 must be skipped, since they are not tight enough for the required accuracy. Since a_2 has a width of 0.01, its centerpoint 3.145 is guaranteed to have an error of at most 0.005. Therefore, 3.145 is a valid approximation. Taking the centerpoint of any of the tighter intervals would also give a valid approximation, but in general generating tighter intervals requires higher computational costs, so it is usually desirable to use the earliest possible interval to generate the approximation.

Computable reals should also support the reverse: a constructor that takes an approximation function or a list of approximations (*Rationals*), where the first term has an accuracy $n \geq 0$, the second term has $n \geq 1$, etc, and produces a representation (*CompReal*) of the real value they converge to. This is pictured in Fig. 5 as function *cons*. For instance, if we feed *cons* with the following geometric series, where $a = 1$ and $r = \frac{1}{2}$[4]:

$S_0 = 1$

$S_1 = 1 + \frac{1}{2} = \frac{3}{2}$

$S_2 = 1 + \frac{1}{2} + \frac{1}{4} = \frac{7}{4}$

...

The resulting computable real $cons\ S$ must represent the number 2. Notice that *cons* assumes a constant module of convergence, which in this particular case is satisfied, since the error of each approximation is $R_n = |S_\infty - S_n| = \frac{a}{1-r} - a\frac{1-r^{n+1}}{1-r} = |a\frac{r^{n+1}}{1-r}| = 2^{-n}$. For other geometric series (or other sequences in general), this condition might not hold, so some terms might have to be filtered out before applying *cons* to ensure the filtered sequence follows the modulus. In any case, using *cons* always requires some known bound to the error of each approximation, either to ensure they follow the modulus or to generate a sequence that does. Also notice that *cons* is only meaningfully defined if the sequence of approximations is consistent, *i.e.* each approximation doesn't contradict the ones that came before.

$approx$ and $cons$ are not required to be inverse functions; $approx \circ cons$ may change the sequence of rationals, and $cons \circ approx$ may result in a *CompReal* with different internal values (*e.g.* different intervals in a nested intervals representation). However, both compositions should preserve the real number being represented.

Obviously, computable reals must also support numerical operations. In addition to the basic arithmetic operations, real numbers are also closed under trigonometric functions, exponentiation (with a positive base), and logarithms (on positive values). It must also be possible to promote any rational value to a computable real – this is function $fromRational$ in Fig. 5, which can be obtained solely from the

[4] The usual definition of a geometric series ($a_n = ar^{n-1}$) starts at index 1. This example uses the alternative definition $a_n = ar^n$ that starts at index 0, leading to the slightly different summation formula $S_n = \sum_{k=0}^{n} ar^k = a\frac{1-r^{n+1}}{1-r}$.

constructor as $cons \circ const$. Likewise, other operations such as addition, multiplication, etc can also be obtained from $approx$ and $cons$ – Fig. 6 is a diagram illustrating such a construction for addition. These restrictions are specified by Haskell's *Floating* type class, which *CompReal* extends.

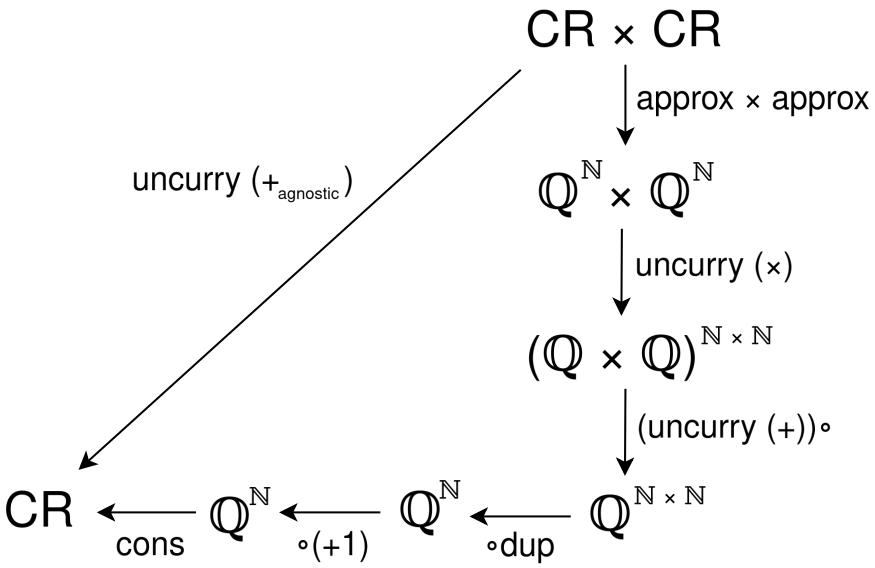


Figure 6: A representation-agnostic implementation of the sum of two *CompReals* (abbreviated as CR). The $\circ(+1)$ ensures the correct error bounds for the sequence before using $cons$. The implementation of $limit$ presented below works similarly; $join$ skips the first element because its error is too large.

Another natural operation to support is limits. Unlike the constructor, which simply constructs a computable real from a sequence of approximations (which is, as explained before, our theoretical basis for computable reals), a limit (function *limit* in Fig. 5) takes in a sequence of converging computable reals and produces a single computable real. Like with *cons*, using *limit* requires a known error bound for each term, to ensure the computable reals in the sequence follow a constant modulus of convergence. For example, imagine we wanted to calculate the *limit* of the following sequence of reals, each represented by a sequence of nested intervals:

$\pi \to [3, 4]\ \ [3.1, 3.2]\ \ [3.14, 3.15]\ \ \ldots$

$\pi + 1/2 \to [3, 4]\ \ [3.6, 3.7]\ \ [3.64, 3.65]\ \ \ldots$

$\pi + 3/4 \to [3, 4]\ \ [3.8, 3.9]\ \ [3.89, 3.9]\ \ \ldots$

...

If we know that this progression of reals follows a fixed modulus of convergence for their limit (*i.e.* the first real, π, has an error of 1, the second real has an error of $1/2$, etc.), then calculating the limit will involve creating a new computable real (represented by a new sequence of nested intervals) that converges to said limit. We can do so by, for example, taking the first interval from the first term and expanding it

with the error 1, taking the second interval from the second term and adding the error $1/2$, etc.

$[2, 5]\ \ [3.1, 4.2]\ \ [3.64, 4.15]\ \ ...$

The resulting sequence of nested intervals is a representation of the real we were trying to calculate, $\pi + 1$. Note that at no point in this process did we reach the value of the limit; we only rearranged the intervals in order to create a new computable real that corresponds to that value, and can then *approx*imate this new computable real to get as close to the limit as we want.

Like the arithmetic operations, the function *limit* is also obtainable using *approx* and *cons* with the help of an auxiliary function $join : (\mathbb{Q}^{\mathbb{N}})^{\mathbb{N}} \to \mathbb{Q}^{\mathbb{N}}$. To better illustrate this construction, here is a sequence of converging real numbers, each with its own sequence of approximations:

$r_0 \xrightarrow{approx} a_{0,0}\ \ a_{0,1}\ \ a_{0,2}\ \ a_{0,3}\ \ ...$

$r_1 \xrightarrow{approx} a_{1,0}\ \ a_{1,1}\ \ a_{1,2}\ \ a_{1,3}\ \ ...$

$r_2 \xrightarrow{approx} a_{2,0}\ \ a_{2,1}\ \ a_{2,2}\ \ a_{2,3}\ \ ...$

...

The limit of this sequence of reals, $r_\infty = limit\ r$, can be constructed with *cons* by taking a path through the approximations with a constant modulus of convergence. To make it clear how to obtain such a path, look at the following table representing the maximum errors of each real and approximation. The top row shows the error of each approximation $a_{n,m}$ to its real r_n (guaranteed by the postcondition of *approx*) and the left column shows the error of each real r_n to the limit of the sequence r_∞ (guaranteed by the precondition of *limit*). The body of the table is the error of each approximation $a_{n,m}$ relative to the limit r_∞, and can be obtained by adding the error at the top of its column and the error at the left of its row:

	$\lvert r_n - a_{n,0}\rvert \leq 1$	$\lvert r_n - a_{n,1}\rvert \leq 1/2$	$\lvert r_n - a_{n,0}\rvert \leq 1/4$
$\lvert r_\infty - r_0\rvert \leq 1$	$\lvert r_\infty - a_{0,0}\rvert \leq 2$	$\lvert r_\infty - a_{0,1}\rvert \leq 3/2$	$\lvert r_\infty - a_{0,2}\rvert \leq 5/4$
$\lvert r_\infty - r_1\rvert \leq 1/2$	$\lvert r_\infty - a_{1,0}\rvert \leq 3/2$	$\lvert r_\infty - a_{1,1}\rvert \leq 1$	$\lvert r_\infty - a_{1,2}\rvert \leq 3/4$
$\lvert r_\infty - r_2\rvert \leq 1/4$	$\lvert r_\infty - a_{2,0}\rvert \leq 5/4$	$\lvert r_\infty - a_{2,1}\rvert \leq 3/4$	$\lvert r_\infty - a_{2,2}\rvert \leq 1/2$

It should now be clear how to obtain an appropriate sequence of approximations for r_∞: simply take the diagonal and skip the first element. Hence, $join\ f = \backslash n \to f\ (n+1)\ (n+1)$. This is the best choice when optimizing the convergence rate, though it may not be the most performant if one of the directions of the matrix is significantly faster to iterate than the other (e.g. if calculating an approximation $a_{n,m}$ had a time complexity of $O(nm^3)$). In such a case, optimizing for performance might look like taking a sloped path through the matrix instead of the diagonal; but, of course, such considerations would require *a priori* knowledge about the performance of iterating the input sequences, which means that in

the general case they are unviable for optimizing pure Haskell code.

Finally, real numbers should support comparison. As stated before, comparison is semidecidable on the computable reals, so the respective instantiation of Haskell's *Ord* class might never halt when comparing two reals. An in-depth exploration of why this is problematic can be found in Section 3.3.1, which showcases how comparisons work in the hybrid simulator. In order to guarantee halting, a new class *CompOrd* was created to capture the idea of comparing increasingly accurate approximations, in the domain theoretic sense. To compare two values, function *domCompare* takes an additional parameter n and first *approx*imates the two operands to the given accuracy n. If the difference between the approximations is greater than 2^{1-n}, the approximations can be compared in the usual way, returning LT or GT. Otherwise, a non-maximal element of the ordering domain (pictured in Fig. 4) is returned.

Putting everything together we can write some Haskell code for these functionalities. The actual code in Jaguar is divided in more classes, and some functions are omitted here, but the core idea remains the same:

```
1  --data Ordering = LT | EQ | GT --defined in Prelude
2  data MidOrdering = LEQ | NEQ | GEQ
3  data OrderingDomain = Bottom | Middle MidOrdering | Top Ordering
4  class CompOrd a where
5    domCompare :: a -> a -> Int -> OrderingDomain
6
7  class (Floating r, CompOrd r) => CompReal r where
8    approx :: r -> Int -> Rational
9    cons :: (Int -> Rational) -> r
10   limit :: (Int -> r) -> r
11   limit f = cons (\n -> approx (f (n+1)) (n+1))
```

Listing 2.1: Haskell code for *CompReal* class

An important point to stress is that *CompReal* and $\mathbb{Q}^{\mathbb{N}}$ need **not** be isomorphic. Requiring so from instances of *CompReal* would force them to work internally with *Rational* numbers and remove a lot of freedom from implementations. On a related note, an intensive use of the operations *approx* and *cons* can have serious impacts on performance, which is why these operations should be used judiciously. Thus, even though *limit*, *fromRational* and all numeric operations could be representation-agnostic (*i.e.* by manipulating sequences of approximations obtained with *approx* and translating back to *CompReal* using *cons*, as shown in the previous examples), doing so would be terrible for performance. Instead, each instance of *CompReal* implements these functionalities independently, optimized to the specific internal structure of its representation, and using its library's code and functionalities whenever possible.

2.2.2 Technical Considerations

Before analyzing the exact real arithmetic packages, a couple technical points are worth considering: integer/rational number representation and memoization.

Haskell, as a high-level functional programming language, provides an arbitrary precision *Integer* data type that greatly simplifies number representation. Although the theory discussed thus far assumes rationals (which can be represented as two *Integers* – numerator and denominator) as approximations for real numbers, all analyzed Haskell packages use only the dyadic numbers, which are numbers of the form $a/2^p$, with $a, p \in \mathbb{Z}$. What is useful about them is they can also be represented as two *Integers* (the libraries actually use *Int*, the fixed-size[5] counterpart to *Integer*, for the exponent p) and have simple sum and product operations, which benefit from bitshift optimizations of powers of 2. The main disadvantage of dyadic numbers is that the representation of all other rational numbers also becomes an infinite sequence. So, for example, representing the number $1/3$ with nested intervals and rational approximations could be done simply as:

$$[1/3, 1/3] \sqsubseteq [1/3, 1/3] \sqsubseteq \dots$$

Since we can represent $1/3$ exactly, the very first interval already has exact accuracy. However, if we limit ourselves to dyadic numbers, non-dyadic numbers such as $1/3$ have to be represented with increasingly small intervals, just like the irrational numbers:

$$[0/2^0, 1/2^0] \sqsubseteq [0/2^1, 1/2^1] \sqsubseteq [1/2^2, 2/2^2] \sqsubseteq \dots$$

In the second sequence of intervals the accuracy of each interval is limited by its precision. Consequently, to achieve a certain accuracy, calculations on this sequence might need to use many more terms than the sequence of rational intervals and operate at high precisions, leading to worse performance.

Another concept to have in mind is memoization, which can be defined as an optimization technique where the result of intermediate calculations is cached to avoid repeating them, therefore increasing performance. [6] has a good chapter explaining memoization in the context of optimizing recursive functions using dynamic programming. In Haskell, some degree of memoization can be done automatically by taking advantage of its lazy evaluation model. Essentially, when assigning the result of a computation to a variable, the computation isn't performed until the value of the variable is used. If the value is used multiple times, the computation is only run the first time. This allows carefully structured programs to memoize results by storing them in variables or data structures. Depending on the program, memoization can sometimes turn exponential time complexity into polynomial time complexity – a massive improve-

[5] Although the exact size depends on the Haskell implementation, with most implementations using 64 bits, an *Int* is guaranteed to have at least 30 bits. This can make them dangerous when used as the exponent of extremely small dyadic numbers, but is usually more than sufficient.

ment. That said, all memoization is fundamentally a trade-off of time costs for memory costs, and both memoization techniques presented below increase memory use.

In the aforementioned framework of exact real arithmetic where computable reals are represented as sequences of approximations, there are two situations that can be optimized by memoization: when calculating the same terms of the sequence twice, which we shall call type 1 memoization; or when calculating the terms of a sequence out of order – type 2 memoization. The first type is simple to implement: by storing each term of the sequence in a data structure (a list, for instance), they are cached for future use. The downside is that producing an approximation will require accessing the data structure and incurring the related costs, up to linear in the case of linked lists.

The second type of memoization is more nuanced. It takes advantage of the observation that the n-th approximation of a real number is at least as accurate as any of the previous ones, and therefore, it is a valid result for all previous terms of the sequence. This is relevant when the sequence in question is the result of a long series of operations, and therefore each term has a high computational cost. Our observation allows us to return the already calculated n-th term when calculating a previous term would require long computations, thus increasing performance. This technique reduces memory costs compared to type 1 memoization, since each computable real only stores the most accurate approximation calculated so far. It also works for repeated uses of the same term, though notice the difference from type 1: if term 100 is used once and then term 10 is used multiple times, the approximation calculated for term 100 (which might have a lot more precision and, therefore, be slower to work with) will be returned for all of them. Note, however, that this makes each approximation dependent on whether subsequent approximations have already been calculated, which is an impure effect. This could in principle be modeled as a monad (or incorporated into the IO monad), but using this hypothetical monadic implementation would be pretty cumbersome. The presented packages that perform this optimization instead opt for the use of unsafe Haskell functions to produce this impure effect.

2.2.3 Package Survey

ERA

ERA [24] is the oldest and simplest of the surveyed packages. It represents real numbers as a Cauchy sequence of dyadic numbers with a fixed modulus of convergence, stored as a function *Int* $\rightarrow$ *Integer*, where each *Integer* is the numerator of a dyadic number. Formally, a real number x is represented by a function $f : \mathbb{N} \rightarrow \mathbb{N}$ iff:

$$\forall n \in \mathbb{N}, |x - f(n)/2^n| < 1/2^n$$

So the n-th term of the Cauchy sequence is the dyadic number $f(n)/2^n$, has precision/exponent n and is guaranteed to have an error smaller than $1/2^n$, which is the same as saying it has accuracy of at least n, the defining feature of a fixed modulus of convergence representation. Using integer functions to represent sequences of dyadics is a really simple representation method, which is why all packages that use modulus of convergence representations take this approach. Since the minimum accuracy of each term is equal to its precision ($n \geq p$), precision and accuracy are sometimes used interchangeably in these packages.

An operation in *ERA* produces a new function that for every input accuracy evaluates the functions of the operands at possibly higher accuracies and produces an output that also follows the fixed modulus. For instance, *ERA* (and all other modulus of convergence packages) implement the sum of two reals (represented by functions f and g and resulting in a real represented by function h) as

$$h(n) = \left\lfloor \frac{f(n+2) + g(n+2)}{4} \right\rceil$$

Informally, the n-th approximation for the sum of two reals is given by the sum of the $n+2$ approximations of the operands, divided by 4 and rounded to the closest dyadic with exponent n. This escalation of accuracy reflects the fact that a sum inherits the error of its operands; since the error of the operands is added together, if both operands introduce an error of ϵ their sum might have an error of 2ϵ. So to obtain an approximation with an accuracy of n we need to use approximations of the operands with at least $n+1$ accuracy, sum them together and then convert the result (which will be a dyadic number with an exponent of $n+1$) to a dyadic with an exponent of n by dividing the numerator by 2 and rounding to an integer. However, this rounding may also introduce an error of ϵ; so an accuracy of $n+2$ is needed from each operand (summed together, divided by 4 and rounded) so that the result has the target accuracy of n.

Other operations work in similar ways. Each one has to be implemented *ad hoc*, taking into account the specific way it propagates errors. Adding two approximations adds the error of the two, but multiplying can scale errors up or down; so multiplication must be implemented differently to take that behavior into account. That being said, addition is still a useful case study to understand the general shape of operations in a modulus of convergence representation, namely how they might escalate accuracy as more and more operations are performed.

Sadly, *ERA* doesn't have many utilities to work with its real numbers other than the basic arithmetic

operations. It also does not memoize any of its results, making it the slowest of the surveyed packages.

exact-real

exact-real [25] uses the same approach as *ERA* (fixed modulus of convergence stored as integer functions representing dyadic numbers) with additional functionality, most notably, type 2 memoization. It is implemented as a mutable variable for each real containing the most accurate dyadic approximation of the number calculated so far. In Haskell, mutable variables live in the IO monad, so to avoid dealing with monads *exact-real* uses $unsafePerformIO$ to turn the calls to IO into pure, though unsafe, functions.

exact-real also includes the wanted accuracy of a real number in its type signature using Haskell's DataKinds extension. This slightly complicates the syntax (and is a bit awkward to work with in an arbitrary-accuracy context, requiring existential types and the like) but allows a straight implementation of Haskell's comparable and number classes since the information about the accuracy is present at the type level, which is a pretty interesting approach.

ireal

To analyze these packages I tried to categorize them as nested intervals representations or modulus of convergence representations. *ireal* [26], however, defies this categorization. It too uses dyadic numbers represented by integer functions, but instead of being based on the modulus of convergence it explicitly takes inspiration from nested intervals. Formally, an *ireal* value represented by a function $\langle f, g \rangle$ contains a real number x if

$$\forall n \in \mathbb{N}, f(n)/2^n < x < g(n)/2^n$$

Notice how this definition can also represent open intervals; just like in the domain theoretic formalization presented in Section 2.1.2, an *ireal* object is an interval, with single real numbers being a special case. This is important for *ireal* since the creator wanted to ensure a solid theoretical ground for the implementation. However, *ireal* adds a second restriction to its representation strategy: when representing a single number, the n-th interval's endpoints must be exactly $2/2^n$ apart. Putting the two restrictions together, we end up with a formulation functionally equivalent to *ERA* and *exact-real* but that takes up twice the space since it uses two *Integers* for each term instead of one. If *ireal* was only used to represent single numbers, this would be redundant; but this approach allows explicitly working with open real intervals and performing interval arithmetic, which is a really interesting feature. As for my categorization, since operations are still implemented to preserve the modulus whether operating on an interval or a single

real, it still falls in the modulus of convergence representation strategy, and has the same behavior and performance as the other packages that use modulus of convergence.

Like *exact-real*, *ireal* also implements type 2 memoization by making use of mutable variables and $unsafePerformIO$, though better encapsulated. By its own admission, this is the least elegant part of the package, but essential for the performance of large computations.

CDAR

CDAR [27; 28] is a nested intervals implementation based on centered dyadic approximations and inspired by *iRAAM* [16], an exact real arithmetic C++ library. Each approximation is stored as a center point and an error bound, both dyadic numbers with the same exponent. This is equivalent to storing the two endpoints of an interval, as described in the paper, but is slightly more compact in terms of storage and has performance benefits when working with the error bound of the approximation. The downside is that numeric operations are harder to implement, since the image of an interval is not in general centered on the image of its center point (when performing multiplications, for example).

A real number is represented as an infinite list of these approximations, taking advantage of Haskell's laziness. This way, it possesses type 1 memoization. Operations using two real numbers are implemented as a zip of the two lists. Since the sequence of approximations doesn't follow a modulus of convergence, requesting an approximation with some given accuracy requires explicitly iterating the list until a suitable approximation is found.

Even though the accuracy of *CDAR*'s approximations don't follow a predictable pattern, their precisions (the exponent of the dyadic numbers) do: every sequence of approximations has the same progression of precisions, which starts at 80 and grows by 50% with each index (80, 120, 180, 270, etc.). This pattern is enforced to prevent unbounded growth in the precision of the sequences. If an operation would produce approximations with larger precision than the pattern, it is rounded to the target precision, even if doing so loses a bit of accuracy.

aern2-real

aern2-real [29] is very similar to *CDAR*, also making use of centered dyadic approximations and lists and following a similar precision pattern. However, it implements a plethora of other utilities and operations, of note: a comparison function between two reals, returning an infinite list of kleeneans (the three-valued logic equivalent of booleans: true, false or indeterminate – Fig. 3); handling of partial functions and errors (using another package *collect-errors* [30] of the same author); and the calculation of limits as a built-in

operation. Though most of these aren't of use to our particular use case, an efficient implementation of limits is of the utmost importance to the hybrid simulator, and the other features are interesting additions.

aern2-real has two other important differences from *CDAR*: it has a different, more optimized implementation of some mathematical functions (trigonometric functions, exponentiation, etc.); and instead of iterating the whole list of nested intervals to find an approximation with a given accuracy, *aern2-real* skips the first few intervals in the list in an attempt to avoid redundant computations. The greater the wanted accuracy, the more intervals it skips. This can sometimes save a lot of time, but can also be slower if too many intervals are skipped.

2.2.4 Comparison and Summary

Package	Strategy	Data Structure	Approximation	Memoization
ERA	Modulus of Convergence	Function	Dyad	None
exact-real	Modulus of Convergence	Function	Dyad	Type 2
ireal	Modulus of Convergence	Function	Dyad Interval	Type 2
CDAR	Nested Intervals	List	Centered dyad	Type 1
aern2-real	Nested Intervals	List	Centered dyad	Type 1

Table 1: Package comparison

In a sense, the details of the individual implementations are irrelevant, insofar as all of them support (or can be extended to support) all of the same base features, specified in Section 2.2.1. However, these details are still significant for performance.

To summarize how these different details interact and impact performance:

- Strategy defines how quickly the approximations converge throughout the sequence. Modulus of convergence prescribes a fixed convergence rate, and thus must implement each operation *ad hoc* in order to preserve the modulus. On the other hand, nested intervals have no restrictions on convergence – consecutive intervals' widths may halve, be cut in ten, or even stay the same – which makes operations simpler to implement.

- Using lazy infinite lists as a data structure has the advantage of automatically granting type 1 memoization (which comes with performance benefits but also memory costs), but the disadvantage of forcing an explicit iteration to access later elements in the list. Functions can support type 2 memoization, but by default have none. Since nested intervals representations already require

explicitly iterating their intervals, they translate perfectly into lists, while modulus of convergence representations use functions to avoid the linear access time of lists.

- The choice of approximation affects the way integer and rational values are represented, which may reduce the number of approximations needed. However, more complicated approximations have a greater overhead. All of the analyzed representations use dyadic numbers as a base, making use of their simplicity.

- Memoization is absolutely crucial for performance, especially when numbers are used multiple times for calculations. Type 1 stores the value of every approximation of a number, causing greater memory use. Type 2 only stores the most accurate approximation calculated so far, which is only advantageous when the same approximation is used repeatedly or the sequence is used out of order – but less memory is used.

2.3 The Fundamentals of Hybrid Systems

2.3.1 Formalizing Hybridness

The current standard formalism for hybrid systems is the hybrid automaton, originally introduced in [31]. In essence, these are automata where states are labeled with differential equations (continuous behavior), and edges, marked with guards and variable assignments, determine the transitions between states (discrete behavior). While a useful model, we don't actually need to consider automata to reason abstractly about hybridness. Instead, we will consider the basic features of hybrid systems with a functional lens, and in particular follow a monadic description of hybridness which will then be used to formalize the denotational semantics of our language (Section 3.4.3) and implement the language in practice.

The main characteristic of hybrid programs is their relation to time, namely, how the evolution of the system may be both continuous and discrete. It is important to distinguish the simulated time from the real time, though: some hybrid programs simulate hundreds of years instantly, while others take hours to simulate a single second. When talking about time from now on, unless otherwise stated, it refers to simulated time, as opposed to execution time, performance, etc.

A hybrid program can be modeled as a function that, given a state, returns the evolution of the state through time, also known as a trajectory. For instance, given a state of n real numbers, a hybrid program would be a function of type $\mathbb{R}^n \to (\mathbb{R}^n)^{\mathbb{R}_0^+}$ [6]. Programs usually don't define the trajectory of the state

[6] In this example the program always outputs an infinite trajectory regardless of the input. In the next section this type is refined to allow finite trajectories.

indefinitely though, only up to some positive duration d. This allows programs to be composed: the composition of two programs is a new program whose trajectory follows the first program's trajectory to its end and then feeds the last state into the second program and follows its trajectory. Of course, since hybrid systems allow both continuous and discrete changes, the resulting trajectories may be discontinuous.

2.3.2 The Hybrid Monad

Modeling hybridness as a monadic effect greatly facilitates both reasoning about and implementing simulations of hybrid systems in practice. Any instantaneous change in state, in the form of some traditional computation, can be converted into a hybrid program through the hybrid monad's unit function; and continuous evolutions are represented in general as a trajectory. This way, the hybrid monad represents the mix we were looking for of both discrete and continuous changes.

A general description of hybridness allows for any kind of state. However, as pointed out by the creators of Lince – an imperative hybrid programming language – in [2], there are advantages in fixing the state type such that every operation both takes and produces the same type of state. Due to its imperative nature, Lince only works with states of n real variables ($\mathbb{R}^n$), using differential equations as trajectory generators. As an advantage, Lince can formalize its syntax with a parametrized monad $\mathbf{H}_S$, where S is the fixed type of the state of the program (which in Lince's case happens to be $\mathbb{R}^n$ for some $n \in \mathbb{N}$):

$$\mathsf{H}_S\, X = \sum_{I \in [0, \mathbb{R}_0^+)} S^I \times X + \sum_{I \in [0, \overline{\mathbb{R}_0^+}\rangle} S^I$$

Recall the notation: $[0, \mathbb{R}_0^+)$ is the set of intervals of the form $[0, d), d \in \mathbb{R}_0^+$; and $[0, \overline{\mathbb{R}_0^+}\rangle$ is the set of all intervals of form $[0, d], d \in \mathbb{R}_0^+$ or $[0, d), d \in \overline{\mathbb{R}_0^+}$.

Let's break down the definition. $\mathsf{H}_S X$ is defined as a coproduct. The left-hand side represents what the paper calls *convergent* trajectories, which are trajectories that have a certain duration $d \in \mathbb{R}_0^+$ and a defined endpoint at time $t = d$. The definition separates these into a product: $\sum_{I \in [0, \mathbb{R}^+)} S^I$ is the open trajectory up until the endpoint and X is the endpoint (the duration d is implicitely given as the upper bound of I). The right-hand side of the coproduct represents *divergent* trajectories, which are trajectories that don't have an endpoint. These may come about in two situations: either the trajectory has "infinite duration", as in form $S^{\mathbb{R}_0^+}$ (these are *open divergent*); or it has a finite duration but no endpoint (*closed divergent*). This second situation may seem puzzling, but consider the following example of a closed divergent trajectory:

```
1  x := 1;
```

```
2  while true do { wait x; x := x/2 }
```

Listing 2.2: Hybrid program resulting in a closed divergent trajectory

Calculating the state of the system at $t = 1$ would require an infinite number of iterations of the while loop. The moment $t = 1$ is what we call a zeno point (since this situation is reminiscent of Zeno's paradoxes); the state of the system at and after this point is not defined (see Section 3.4.1 for more details on zeno points and how they are handled by the hybrid simulator).

The creators of Lince also show that $\mathbf{H}_S$ is an instance of the generalized writer monad, which allows for properties about $\mathbf{H}_S$ (such as the fact that it is indeed a monad) to be demonstrated abstractly though the generalized writer monad. This is how its monad operators specialize to $\mathbf{H}_S$: the unit function $u : X \to \mathsf{H}_S X$ returns an empty convergent trajectory with the inputted endpoint; and the Kleisli lifting $(^\star)$ (also known as monadic bind) transforms a function $f : X \to \mathsf{H}_S Y$ into a function $f^\star : \mathsf{H}_S X \to \mathsf{H}_S Y$. If the hybrid trajectory inputted to $f^\star$ is divergent, it is returned as is; otherwise, f is applied to its endpoint and the resulting hybrid trajectory is concatenated with the first. As expected, the output is convergent only if both trajectories are themselves convergent.

In the paper, $\mathbf{H}_S$ is also shown to be an Elgot monad [32] by using the bind operator to define an Elgot iteration operator as a least fixed point. Elgot iteration is later used in the paper to formalize the denotational semantics of Lince's while loops. This dissertation doesn't cover this detail, though, as it requires a fairly specialized concept for a relatively small payoff.

Chapter 3

Implementation of a Hybrid Simulator

3.1 System Architecture

This chapter presents the implementation of this project's hybrid simulator, named Jaguar [4].

Jaguar's architecture (Fig. 7) has four main components: a parser, which takes in the hybrid program and produces a syntax tree; an interpreter that runs the semantics of the language; a solver, responsible for solving systems of differential equations and providing the respective solutions to the interpreter; and a visualizer, that displays the whole evolution of the system as a plot constructed from multiple queries to the interpreter. It is also possible to query the interpreter directly to obtain the state of the system at any given time.

These components were designed to be as modular as possible and are mostly interchangeable, with future changes and improvements in each component not affecting the others. The biggest dependency between them is Jaguar's syntax tree, which is produced by the parser and consumed by the interpreter and solver. All components are also type-agnostic, supporting any type that instantiates a few typeclasses. Besides the computable real types presented in Section 2.2.3, Jaguar can also run using regular floating-point numbers (though, of course, the correction of the results is contingent on the program at hand not requiring more numerical precision than what the floating-point numbers can provide), and can be made to support many more in the future simply by instantiating the necessary classes.

The rest of this chapter takes a closer look at each of these components: Section 3.2 is about the

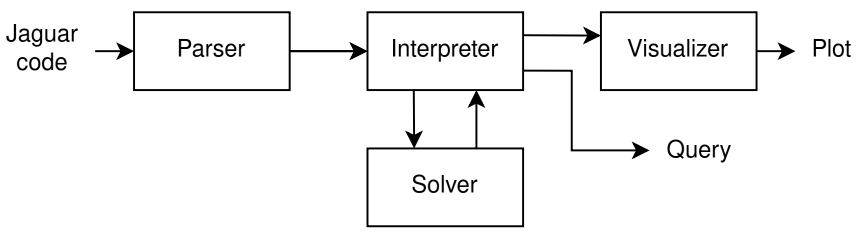


Figure 7: Diagram of Jaguar's architecture

parser, Section 3.4 explains some general features of the interpreter and presents the denotational semantics of the language (with Section 3.3 being about the specific challenge of supporting types with semidecidable comparison), Section 3.5 explains the workings of the solver and their mathematical motivation, and Section 3.6 describes the visualizer.

3.2 Parsing a Hybrid Language

To parse our hybrid language, the Happy [33] parser generator was chosen. Happy takes an annotated BNF grammar specification and produces a Haskell module containing an LR parser for the grammar. Happy is also the parsing library that the Glasgow Haskell Compiler (GHC), the *de facto* standard compiler for Haskell, uses to parse its own syntax. Happy's pretty comprehensive functionalities proved to be more than enough for the project's needs.

A Jaguar program is composed of one or more statements. Statements must be separated by semicolons, and the program may optionally end with one. Each statement may contain boolean expressions and numerical expressions, which are evaluated using the expected operator precedence rules (multiplication has higher precedence than addition, which has higher precedence than comparison, which has higher precedence than disjunction). Listing 3.1 showcases all atomic statements (assignments and differential statements) and some examples of operator precedence.

```
1  //Assignments receive a numerical expression
2  x := 0;
3  v := 4 + 2 * x; //Interpreted as 4 + (2 * x)
4
5  /* Differential statements receive a numerical expression
6     for each variable and one for the duration
7     (the duration is evaluated before any changes take place) */
8  x' = v, v' = 1 for 2*v;
9
10 //Wait is just a differential statement with no variables
11 wait 1;
12
13 //Operator precedence works as expected
14 if v > 5 + 2 * x || x > 10
15 then v := 0
16 else v := 1;
17
```

```
18  //There are also infinite differential statements
19  x' = v forever //Semicolon at the end of the program is optional
```

Listing 3.1: Jaguar program with no particular meaning showcasing atomic statements and operator precedence

Control statements (if-then-else and while statements) take programs as arguments. THese programs may be either a single statement or multiple statements encased in brackets. Inside the brackets, statements must be separated by semicolons, with an optional semicolon at the end. Line breaks and indentation are always ignored.

Jaguar also features C-style comments, as both examples demonstrate. Line comments begin with a double slash // and block comments begin with /* and end with */.

```
1   x := 0;
2   v := 4 + 2 * x;
3
4   if x > 10
5   then v := 0 //A semicolon here is not allowed
6   else { x := 10; v := 0; }; //The last semicolon inside the brackets is
      optional
7
8   if x > 10 then x := 0; //Else clauses are optional
9
10  while x < 10 do { x' = v for 1 }
```

Listing 3.2: Jaguar program with no particular meaning showcasing control structures and subprograms

Under the hood, the semicolon is actually the sequencing operator, which takes two programs and combines them into a single program that executes its two parts in sequence. Sequencing two hybrid programs is an associative operation, which means the associativity of the semicolon is ultimately arbitrary from a semantic perspective. However, to make use of Haskell's lazy evaluation, it is advantageous to make the semicolon right-associative. This way, querying the program for the state at time t only has to interpret the program up to that point.

The full specification of the language's syntax, in BNF notation and its precedence rules, can be found in Appendix A.

3.3 The Challenges of Semidecidability

Up until now, the features discussed were common among other hybrid simulators. However, dealing with computable real numbers results in some unique challenges, mainly when it comes to comparison. This section presents these challenges and the solutions developed to address them.

3.3.1 Strict Comparison

Take as an example the following program, which is meant to calculate the base 2 logarithm of a small number $x < 1$, rounded down ($\lfloor log_2(x) \rfloor$):

```
x := 0.0001;  //can be any number less than 1
y := 1;
ans := 0;
while x < y do { ans := ans - 1; y := y / 2 }
```

Listing 3.3: Program that calculates the base 2 logarithm of x

This algorithm is mathematically correct: it always takes a finite number of steps for any input x, and always reaches the correct answer. However, notice what happens if we try to run the program with the input $x = 0.5$: the second iteration of the while loop will try to make the comparison $0.5 < 0.5$. Mathematically, it should evaluate to false, but as explained in Section 2.1, comparing two equal computable real numbers will never terminate. As a result, our program will never terminate, even though it is mathematically sound!

The fundamental limitation here is the semidecidability of comparison in the computable reals: whenever our hybrid simulator tries to fully evaluate a comparison, it may never terminate, giving no further feedback to the user. From a design standpoint, this places on the user the responsibility of ensuring that no comparison ever compares two equal numbers. Which, while in some cases is simply impractical, in others it is downright impossible.

Ideally, such comparisons would produce an error instead of running indefinitely. We can do that by evaluating the comparison only up to a certain accuracy. Formally, when evaluating $r < s$ to accuracy n, we first approximate r and s to intervals of rational numbers $a \leq r \leq b$ and $c \leq s \leq d$ with an accuracy of n (*i.e.* $b - a, d - c \leq 2^{1-n}$ as per Eq. (2.10)). If $b < c$, then we can be sure that $r < s$ and the original comparison is true. If $d \leq a$, then we are sure $r \geq s$ and the comparison is false. In all other situations, we can't resolve the comparison; the given accuracy isn't enough. All that can be inferred is that $|r - s| \leq 2^{2-n}$. In these cases an error is raised, giving valuable feedback to the user. Analogous

reasoning can be used for $>$, $\leq$ and $\geq$.

Of course, this means that comparing two different but close real numbers ($|r - s| \leq 2^{2-n}$ at most, depending on their approximations) can also result in an error; but these false positives are inevitable if we wish to guarantee our program eventually terminates. It is up to the user to interpret the error, figure out the cause and decide how to solve it; either by raising the comparison accuracy, or by rewriting the program to avoid the faulty comparison, or even re-enabling exact comparisons (by "raising the comparison accuracy to infinity") if they are confident the program will terminate.

The comparison accuracy n could be specified for each comparison individually, but to avoid cluttering the syntax it was decided instead that all comparisons in a program would have the same accuracy, which can be supplied as an external configuration when running the program.

3.3.2 Boolean Expressions

Since comparisons may not produce a result, boolean expressions using those comparisons have to deal with three possible outputs for a comparison: true, false and inconclusive. In the previous sections, it was implied that any inconclusive comparison would immediately raise an error, but this doesn't have to be the case. Take, as an example, the following program:

```
1 x := 1;
2 y := 1;
3 while x > 0.125 || y < 30 do { x := x / 2; y := y * 2; }
```

Listing 3.4: Hybrid program showcasing the usefulness of three-valued logic

Although the comparison $x > 0.125$ is indeterminate at the beginning of the fourth iteration, the whole expression can be inferred to be true: since $y < 30$ is true, regardless of the value of the first term, the disjunction is true. In a sense, we can treat indeterminate values as simply another possible output from a comparison, and then work out the possible results of the disjunction. If both true and false results are possible, then the whole disjunction is indeterminate.

Essentially, what is being described is Kleene's logic, a three-valued logic, used as a way to capture epistemological knowledge [34]. This logic, as the name implies, works with 3 logic values: T, F and a middle value U. By interpreting the middle value to mean "a current lack of information about a proposition", we can use Kleene's logic to evaluate compound propositions from the valuations of their atoms. Interestingly, this notion can also be conceptualized from a domain theory lens as the flat domain of the booleans, containing three values: true, false and bottom (Fig. 3). Such a formalization is useful because it allows us to extend what was said before about computable real numbers in Section 2.1.2

to the semantics of Jaguar: since operators on the domain of the booleans are continuous, evaluating boolean expressions will approach the actual result of the denoted condition, and expressions evaluated with greater comparison accuracy are always consistent with those evaluated with lower accuracy.

Introducing Kleene's logic allows Jaguar to decide many expressions that were undecidable before, but is far from a full symbolic evaluation of the expression. For example, in an expression like $p \vee \neg p$, where p is some comparison, should p be inconclusive, the expression will be evaluated as inconclusive, even though classically it must always be true (the well-known tautology/contradiction problem of Kleene's logic). In the future, more sophisticated models of reasoning can be implemented to solve such problems.

Finally, notice that Kleene's logic is useless if the user re-enables exact comparisons (up to "infinite accuracy"), since the comparison never terminates and the whole program hangs.

3.3.3 Lax Comparison

The solution described above for the semidecidable comparison problem is as good as it gets: it detects faulty behavior and also allows the user to manage false positives. However, there are still problems which it isn't suitable to solve. Take as an example the following program, a simple while loop with an integer counter:

```
1 i := 0;
2 while i < 10 do { i := i + 1; }
```

Listing 3.5: Simple "for" loop with an integer counter variable

The program is meant to exit the loop at 10 iterations, but the comparison $i < 10$ will raise an error when $i = 10$. A simple workaround is to replace the value 10 with a number we know i will never be too close to and that still produces the same results when evaluating the comparison. In this case, 9.5 works. But this puts the chore of finding such a number on the user, and the resulting code is much less readable (and much, much uglier).

But in this particular case, this replacement shouldn't be necessary. Since we know i is restricted to the integers, we can guarantee that an indeterminate comparison means that $i = 10$ for any accuracy $n > 2$, since the comparison is only indeterminate when $|i - 10| \leq 2^{2-n}$. Our restriction of i gives us enough information to make these deductions, effectively bypassing the error.

For these situations, two new operators were created: determinably less than ($<!$) and determinably greater than ($>!$). Unlike the strict comparison operators (the ones introduced in Section 3.3.1 that can return true, false or indeterminate), these new lax comparators only return true or false: true if the comparison is determinably true, and false in all other cases. They are lax in the sense that they always

produce an answer, even when there isn't enough information.

This opens the door to all sorts of misbehavior in the program, of course: $9.99 <! 10$ will evaluate to false if the comparison accuracy is not high enough. Indeed, the output of this operator may be incorrect for all comparisons $r <! s$ where $r < s \leq r + 2^{2-n}$. However, so long as this critical range is avoided, the lax comparators are guaranteed to return a correct result. If both operands are integers, this works out to $n > 2$, as stated above. For other cases it may vary. These operators are not the focus of the dissertation and were included mostly for practicality; they sacrifice the rigor of the strict operators in exchange for simpler and more readable programs.

3.3.4 Hybrid Semidecidability

Another unique problem to the crossing of hybrid systems and arbitrary-precision real numbers is the issue of semidecidability on the concatenation point of two hybrid programs. Take the following program:

```
1 x:=0;
2 wait 1;
3 x:=1;
4 x' = 1 for 1;
```

The output of this program should be a function $x(t)$ such that:

$$\begin{cases} x(t) = 0 & \text{if } 0 \leq t < 1 \\ x(t) = t & \text{if } 1 \leq t \leq 2 \end{cases} \tag{3.1}$$

Although this function is mathematically well-defined, in order to evaluate it to any particular value of t we must perform the comparison $t < 1$ to decide between the two branches of the function. However, because comparison is semidecidable on the computable reals, evaluating the function may be a never halting computation for $t = 1$, and may take arbitrarily long when evaluated for values close to 1.

To solve this problem, as before, we must use a parametrized comparison, taking in the desired accuracy of our query. Note that this accuracy doesn't necessarily need to match the comparison accuracy used for in-program comparisons. The query accuracy is only used to choose between two (or more) consecutive branches of the program, leaving the actual execution of the program (variable assignments, choosing between if-else branches and while-loop iterations) completely unaffected. Indeed, the query accuracy can even be chosen independently for every query, without affecting each other into the past nor into the future.

Taking that into account, to give the user the most flexibility and control, it was decided that a query would be a curried function $\mathbb{R} \to \mathbb{N} \to [\mathbb{R}^n]$ that takes the time t and the desired query accuracy

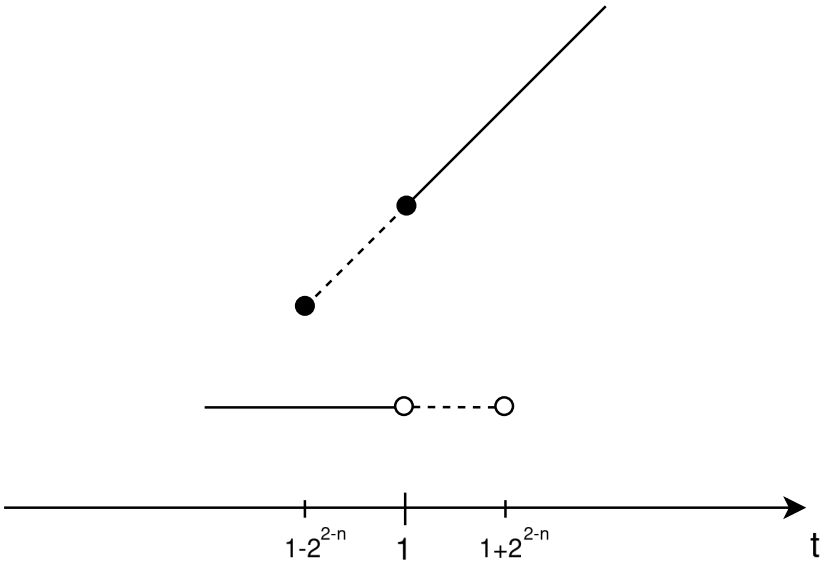


Figure 8: Diagram representing hybrid semidecidability for the program in Eq. (3.1)

n and outputs all possible hybrid states within range of that accuracy. The biggest drawback of this solution is that it may lead to some function branches being evaluated outside their original domain. In this particular example (Eq. (3.1)), evaluating the query $t = 0.999$ with a query accuracy $n < 12$ may result in two possible outputs: 0, the correct output; and 0.999, which is clearly impossible to obtain from the mathematical definition, but must be considered nonetheless.

The minimum query accuracy required to avoid hitting the wrong branch (12 in the previous example) is calculated from the comparison threshold 2^{2-n} discussed in Section 3.3.1. This threshold is closed when querying before the concatenation point and open when querying after it. This can be visualized as extending the hybrid branches by 2^{2-n} time units in either direction, as pictured in Fig. 8. Queries inside the range $[1 - 2^{2-n}, 1 + 2^{2-n})$ may hit both branches, depending on the specific implementation of computable reals.

3.4 A Hybrid Interpreter

With the semidecidability issues out of the way, let us turn our attention to the more straightforward problems of implementing a hybrid simulator. These will serve as motivation for Jaguar's denotational semantics, presented in Section 3.4.3.

3.4.1 Iteration Limit

Unlike traditional imperative programs, since hybrid programs are run with an extra parameter t (the time at which to evaluate the state of the system), it is possible to write programs with an infinite duration:

```
x' = 1 forever
```

Listing 3.6: Hybrid program with infinite duration

Although it is impossible to calculate the final state of the program (and so is concatenating another program after this one, since statements after an infinite program are unreachable), the state of the system at each time t can still be calculated. In this case, $x(t) = t$. It is even possible to write a program that performs an infinite number of operations:

```
while true do { wait 1; x := x + 1 }
```

Listing 3.7: Hybrid program with infinite operations

The while loop in this example is infinite, leading to infinite iterations of the statements in brackets, but this program can still run fine. This is because, although it is impossible to run the whole while loop, running the program with any specific t will result only in a finite number of iterations of the loop. The result is the well-defined trajectory $x(t) = \lfloor t \rfloor$.

What is problematic, however, is an infinite number of operations in a finite amount of time. This can be achieved with, for example, an infinite while loop with smaller and smaller durations for each iteration, such that the series obtained from adding their durations is convergent:

```
x := 1;
while true do { wait x; x := x/2 }
```

Listing 3.8: Hybrid program with infinite operations in a finite amount of time

This time, all iterations of the while loop are run before $t = 2$. Although it can be inferred that the value of x at $t = 2$ should be zero, operationally calculating the state of the system for any $t \geq 2$ would require running an infinite number of operations. We say that $t = 2$ is a zeno point of the system.

The same can be done with loops of zero duration, of course. For the previous two examples, removing the wait expression from the loop results in a zeno point at $t = 0$.

Zeno behavior is usually disregarded in the literature, since it is difficult to analyze and its effects aren't computationally realisable anyway. Despite the existence of some formalizations [35; 36], treating zeno points operationally (to either calculate the value of the system at the zeno point or even just detect that there is zeno behavior) is unfeasible. However, there is a simple alternative to zeno detection: by setting a

limit to the number of iterations the program is allowed to perform, we effectively prevent zeno behavior. If this limit is reached, the simulator will raise an error. The motivation for implementing this iteration limit is similar to the motivation for comparison accuracy presented in Section 3.3.1: it allows users of the hybrid simulator to avoid unintended zeno behavior in their programs, with no analysis overhead, just by adding a parameter to the simulation (though note that reaching the iteration limit doesn't necessarily mean zeno behavior is at fault; just like errors in comparison, false positives may happen and should be handled by the user). To allow programs like Listing 3.6 to run unbothered, the user may also specify they want no iteration limit at all.

3.4.2 Runtime Errors

As seen in the previous sections, our language can throw errors. To handle errors without raising a native exception, the endpoint of a hybrid program may be either a valid state or an error state, containing an error message. If a program's endpoint is an error, the endpoint of its composition with any other program results in that same error. Conversely, the composition of two programs outputs an error if and only if any of the two programs outputs an error.

The following is a list of all situations that cause errors:

- An inconclusive boolean expression, caused by a strict comparison (Insufficient comparison accuracy)
- Too many iterations of all while loops, summed together (Iteration limit exceeded)
- A differential statement with a non-positive or inconclusive duration

3.4.3 Language Semantics: Formalization

To sum up: Jaguar must run with some external configurations (the comparison accuracy and the iteration limit), some of which may change over time (iteration limit decreases by one every iteration). Jaguar must also handle errors, which are treated as a special endstate that inhibits program composition. And of course, the result of a program should be a hybrid object ($\mathbf{H}_S$) consisting of an evolution function and an endstate.

Knowing all this, here are the definitions of the types used in the interpreter.

$$S = \mathcal{X} \to \mathbb{R}$$

The program state S is known in as an *environment*: it corresponds each variable in $\mathcal{X}$ (which contains all variables defined in the program plus the current simulated time t) to their current values. Boolean and arithmetic expressions in the program are functions of the program state ($S \to \mathbf{J}\ \mathbb{B}$ and $S \to \mathbb{R}$, respectively);

$$ES = (1 + \mathbb{N}) \times (1 + \mathbb{N})$$

The external state of the program ES consists of the iteration limit and the comparison accuracy, both of which are optional. It is external in the sense that it doesn't evolve over time like the hybrid state, though it may still change with each program statement;

$$Err = String \times S$$

The definition of the error type Err is ultimately arbitrary, since its contents don't affect the functioning of the interpreter. I opted for an error message and the state of the program at the moment of the error, to make error messages as descriptive as possible.

All of this behavior can be put together into a single type and monad, **J**, which is a composite monad made up of monad $\mathbf{H}_S$ as well as monad transformers **StateT** (for managing the external state) and **ExceptT** (for handling exceptions):

$$\mathsf{J}\ X = ES \to \mathsf{H}_S(Err + (X \times ES))$$
$$\mathbf{J} = \mathbf{StateT}\ ES\ (\mathbf{ExceptT}\ Err\ \mathbf{H}_S)$$

Using **J**, we can interpret a Jaguar program p as a function of type $[\![p]\!] : S \to \mathbf{J}S$ and define it inductively for each statement. Assignments change the value of one variable in the state for another; the notation $\sigma\nabla[\vec{a}/\vec{x}]$ represents the state where the value of each variable $x_i \in \mathcal{X}$ in $\vec{x}$ is $a_i \in \mathbb{R}$ and all other variables have the same values as in σ. The monadic unit η then maps the state into $\mathbf{J}\ S$:

$$[\![x := a]\!](\sigma) = \eta(\sigma\nabla[a\sigma/x])$$

Differential statements are the most complicated of the bunch. To write their definition a new auxiliary operator $\phi_\sigma^{\vec{x}=\vec{a}} : \mathbb{R}_0^+ \to S$ is needed to calculate the solution of the differential equation. Differential statements always create convergent trajectories, so i_L is used to map a trajectory and an endpoint into an $\mathsf{H}_S\ S$. Finally, the lifting $\iota : \mathbf{H}_S \to \mathbf{J}$ lifts the hybrid into a Jaguar program output:

$$[\![\vec{x} = \vec{a}\ \text{for}\ d]\!](\sigma) = \iota(i_L(i_{I=[0,d\sigma)}(\lambda t.\sigma\nabla[\phi_\sigma^{\vec{x}=\vec{a}}(t\sigma)/\vec{x}]), \sigma\nabla[\phi_\sigma^{\vec{x}=\vec{a}}(d\sigma)/\vec{x}]))$$

Sequentiation is defined using the monadic bind by evaluating the first program and passing its result to the second:

$$[\![p; q]\!](\sigma) = [\![q]\!]^\star([\![p]\!](\sigma))$$

Conditional statements first evaluate the condition and then decide which branch to follow. Evaluating the condition may have monadic effects (*i.e.* throw an error), so the branching operator $\triangleleft\, \triangleright : (\mathbf{M}\ X, \mathbf{M}\ \mathbb{B}, \mathbf{M}\ X) \to \mathbf{M}\ X$ must be monadic:

$$[\![\text{if } b \text{ then } p \text{ else } q]\!](\sigma) = [\![p]\!](\sigma) \triangleleft b\sigma \triangleright [\![q]\!](\sigma)$$

A denotational definition for while loops is slightly more complex since it requires the use of very specialized constructs, namely Elgot iteration. However, it can be straightforwardly specified using recursion as

$$[\![\text{while } b \text{ do } p]\!](\sigma) = [\![\text{while } b \text{ do } p]\!]^\star([\![p]\!]^\star(\kappa\sigma)) \triangleleft b\sigma \triangleright \eta(\sigma)$$

where $\kappa : S \to \mathbf{J}S$ returns an error if the iteration limit is reached and otherwise decreases it by one and returns the state unchanged. This is essentially how the loops are implemented in Haskell.

The advantage of defining the behavior of the language in such a monadic, composable way is that the implementation in Haskell is extremely close to the denotational semantics shown here. The actual implementation is just slightly more complicated, mostly because it also has to keep track of discontinuities (which weren't covered here to keep the focus on the behavior of the language; see Section 3.6.1 for details).

3.5 Differential Equation Solver

As explained above, the continuous behavior of our hybrid programs is modeled using systems of ordinary differential equations (ODEs). This section explains how Jaguar uses differential equations to evolve a set of initial conditions – the well-known initial value problem – when working specifically with computable real numbers.

The initial value problem (IVP) can be written in a simplified form as follows: given the initial value $\vec{x}_0$ of some unknown function $\vec{x} : \mathbb{R} \to R^n$ and a function $\vec{f} : \mathbb{R}^n \to \mathbb{R}^n$ such that

$$\begin{cases} \vec{x}(0) = \vec{x}_0 \\ \vec{x}\,'(t) = \vec{f}(\vec{x}(t)) \end{cases} \tag{3.2}$$

find the value of $\vec{x}(t)$ for any $t \in \mathbb{R}$. For some functions $\vec{f}$ it is possible to obtain exact solutions analytically; for instance, given $f(x) = x$ the solution is an exponential function $x(t) = x_0 e^t$. However, in the general case, without making assumptions about $\vec{f}$ it is impossible to obtain the solution using analytical methods. Hence, we turn to numerical methods.

ODEs are a classic problem in numerical analysis, and several algorithms have been proposed over the centuries to find their solutions, from the Euler method to the more complex Runge-Kutta family of algorithms [8]. In general, all these involve calculating or estimating the first (and/or higher) order derivative(s) of $\vec{x}$ at a and then estimating $\vec{x}(a + h)$, for some step size h, with a formula that uses the obtained derivative(s). Different methods are then a balancing act between using higher order derivatives and many small step sizes, which are more accurate but more expensive to compute, versus lower order derivatives and few big step sizes, which are less accurate but much faster to compute, all while preventing compounding errors from all of the estimating. Because of their strategy, these methods don't usually have an error bound for their results, only an asymptotic error dependent on the step size (*e.g.* the 4th-order Runge-Kutta method (RK4) has an asymptotic error of $O(h^4)$, meaning that running the method with half the step size results in an error approximately 16 times smaller).

This puts us in an awkward position. The arbitrary accuracy of computable real numbers has two requirements: that the accuracy of the solution's approximation can be increased on demand; and that any obtained approximation for the solution has a strict error bound, not just an asymptotic error estimate. And, ideally, it should be done with as few numerical operations as possible, since operations on computable reals are slower than their floating-point counterparts. Although they satisfy the first requirement, methods like Runge-Kutta fail the second and, depending on the specifics, may not fulfill the third; so they are not viable candidates for our solver.

Instead, we will be using the Taylor method, which consists in using arbitrarily high order Taylor polynomials to approximate the solution. By using the exact values of the derivatives and not approximations, the only error introduced is from the truncation of the potentially infinite Taylor series. Acquiring a more accurate approximation is as simple as using more terms of the series, thus reducing the truncation error. And as a bonus, if the series is finite and the end is reached we get an exact formula for the solution with zero error. Most importantly, this method has an exact error bound for its result, which is discussed later in this section.

To implement the Taylor method, we therefore need to calculate the derivatives of $\vec{x}$ at 0. Although obtaining the first-order derivative can be done with a simple application of $\vec{f}$, the math gets more and more involved for higher derivatives, with successive applications of the chain rule quickly increasing the

complexity of the expression:

$$\vec{x}\,'(0) = \vec{f}(\vec{x}(0)) = \vec{f}(\vec{x}_0)$$

$$\vec{x}\,''(0) = (\vec{f}(\vec{x}(0)))' = \vec{f}\,'(\vec{x}(0))\vec{x}\,'(0) = \vec{f}\,'(\vec{x}_0)\vec{f}(\vec{x}_0)$$

$$\vec{x}\,'''(0) = (\vec{f}\,'(\vec{x}(0))\vec{x}\,'(0))' = \vec{f}\,''(\vec{x}(0))\vec{x}\,'(0)^2 + \vec{f}\,'(\vec{x}(0))\vec{x}\,''(0) = \vec{f}\,''(\vec{x}_0)\vec{f}(\vec{x}_0)^2 + \vec{f}\,'(\vec{x}_0)^2\vec{f}(\vec{x}_0)$$

There is also the issue of finding the derivatives of $\vec{f}$. Remember that in the context of the hybrid simulator, $\vec{f}$ is not an arbitrary function, but an explicit formula that contains the variables of the system and combines them using a fixed set of elementary functions and operators. Therefore, finding its derivatives could be done symbolically from that formula; but this approach has the same problem as the derivation of the derivatives of $\vec{x}$ above: as the order of the derivative gets higher, the complexity of the expression grows quickly.

To solve both problems at once, we can use automatic differentiation [9]. Automatic differentiation is a group of techniques used to calculate the derivative of functions expressed as programs by systematically applying the chain rule to each elementary operation in the program. Forward automatic differentiation (FAD), in particular, computes the value and derivative of a function simultaneously, propagating them forward through the composition of elementary operations.

One method for implementing FAD is representing the value of expressions and all of their derivatives as (possibly infinite) lists. A constant expression is represented as a singleton list ($2 \sim [2]$) and an independent variable t with value t is represented as a two element list $\mathsf{t} \sim [t, 1]$. Expressions may then be built by operating on these base representations. For example, to obtain the list representing the expression $\mathsf{a(t)} = \mathsf{t} + 2$ we sum the list representations of t and 2 (since $(u+v)' = v' + v'$) to obtain $\mathsf{a(t)} \sim [t+2, 1]$; this represents the fact that $a(t) = t + 2$, $a'(t) = 1$ and higher derivatives are zero. Though do note that, in general, operations on these representations are more complex than a simple zip of the operands, since the derivative of an expression may depend on both the value and the derivative of the subexpressions that make it up (e.g. $(uv)' = u'v + uv'$).

Using these representations, obtaining all derivatives of the solution to an ODE is extremely elegant. Given the formulation in Eq. (3.2) (simplified to a single equation rather than a whole system for this example), first translate the function f into a function f that operates on representations rather than values; and then define x_0 recursively:

$$\mathsf{x}_0 \sim x_0 : \mathsf{f}(\mathsf{x}_0)$$

The resulting list contains all derivatives of x at $t = 0$. From there it is possible to build a Taylor

series that equals x. The process is essentially the same for systems of ODEs.

To turn the Taylor series into a computable real number we take the partial sums of the series (which are Taylor polynomial approximations of x) and calculate their limit. However, recall from Section 2.2.1 that limits require known error bounds for each approximation. There is actually a formula for the error E_k of a Taylor polynomial [12]: if A, B are known such that $\forall t \in [a, a+h], A \leq x^{(k+1)}(t) \leq B$, then

$$A\frac{h^{k+1}}{(k+1)!} \leq E_k(a+h) \leq B\frac{h^{k+1}}{(k+1)!}$$

Using the error bound, it is possible to calculate the appropriate step size h and necessary order k to achieve the desired error. However, although A and B can be found using interval arithmetic by calculating $x^{(k+1)}([a, a+h])$, implementing interval arithmetic for arbitrary functions is not trivial. This error bound is also pretty loose, which would need to be balanced by using small step sizes.

Jaguar instead uses another approach, taking advantage of polynomial ODEs. A polynomial ODE is an ODE where the derivative of the unknown function is given by a polynomial. Of interest, these ODEs have a stricter, *a priori* error bound for Taylor polynomial approximations of their solution based only on their coefficients and initial values [10]:

$$(E_k(h))_i \leq \frac{|c_i||Mh|^{k+1}}{1-|Mh|} \tag{3.3}$$

for $|h| < 1/M$, where M depends on the coefficients and $\vec{c}$ depends on $\vec{x}_0$:

$$c_i = \begin{cases} (x_0)_i \text{ if } |(x_0)_i| > 1 \\ 1 \text{ if } |(x_0)_i| \leq 1 \end{cases}$$

Therefore, if we can turn a system of ODEs into a system of polynomial ODEs, we can then use this error bound. As a bonus, since the error bound can be calculated *a priori*, we can calculate in advance the order k of the Taylor polynomials to use in the limit, instead of having to check the error bound of each k incrementally.

The process of turning a system of ODEs into a system of polynomial ODEs is known as polynomial projection, and is demonstrated in [10]. It essentially consists of adding additional variables and rewriting expressions in terms of one another's derivatives. For example, the ODE $x'(t) = sin(x(t))$ can be transformed into the following polynomial system

$$x'(t) = a(t)$$
$$a'(t) = a(t)b(t)$$
$$b'(t) = -a(t)^2$$

by introducing the variables $a(t) = sin(x(t))$ and $b(t) = cos(x(t))$. These auxiliary variables can be found algorithmically by recursively decomposing the expressions for the derivative of each variable and creating a new variable every time an expression isn't a sum, product or natural exponent (since these operations are closed on polynomials). This process is guaranteed to terminate because each expression is a finite composition of elementary functions, whose derivatives always form a closed dependency loop (e.g. sin $\rightarrow$cos $\rightarrow$-sin).

Finally, to calculate the solution $\vec{x}(t)$ at some t, since the step size of this method must be less than $1/M$ it may be necessary to perform multiple steps to reach the desired t. These steps should have a fixed size except for the last one, so that the intermediary steps can be reused when calculating $\vec{x}(t)$ for other values of t. So what should this fixed size be? A small size means a tighter bound but requires more steps; a big size means less steps but requires a larger k to make up for the looser bound. An optimal choice for the size would depend on the specific performances of the operations involved. Jaguar uses $h = 1/2M$ since it is a good balance between maximizing step size and minimizing k (exactly half of the allowed range) but also because it simplifies the error formula to the following:

$$|(E_k(h))_i| \leq \frac{|c_i|}{2^k}$$

From this formula it is evident that the accuracy of the approximation increases by 1 for each increase of 1 in k, which is perfect for our use case of calculating limits as defined in Section 2.2.1. Recall that the $limit$ function takes in a list of approximations with increasing accuracies; in order to provide said list, we need to calculate the modulus of convergence for the sequence of Taylor polynomial approximations. This modulus, $\alpha_{h,i} : \mathbb{N} \rightarrow \mathbb{N}$, takes in an accuracy n and calculates the smallest k for which the k-th order Taylor polynomial approximation has an accuracy of at least n (i.e. $E_{\alpha_{h,i}(n)} \leq 2^{-n}$). Then, the list $(l_n)_{n\in\mathbb{N}}$ such that $limit\ l_n = x_i(h)$ is given by:

$$l_n = \sum_{j=0}^{\alpha_{h,i}(n)} \frac{x_i^{(j)}(0)}{j!} h^j$$

So what should $\alpha_{h,i}(n)$ be? For the fixed steps $h = 1/2M$, substituting the formula of the error results in

$$
\begin{aligned}
& |(E_{\alpha_h(n)})_i| \leq 2^{-n} \\
\Leftarrow & \frac{|c_i|}{2^{\alpha_{h,i}(n)}} \leq 2^{-n} \\
\Leftrightarrow & |c_i| \leq 2^{\alpha_{h,i}(n)-n} \\
\Leftrightarrow & max(|(x_0)_i|, 1) \leq 2^{\alpha_{h,i}(n)-n} \\
\Leftrightarrow & log_2(max(|(x_0)_i|, 1)) \leq \alpha_{h,i}(n) - n \\
\Leftrightarrow & \alpha_{h,i}(n) \geq log_2(max(|(x_0)_i|, 1)) + n \\
\Leftarrow & \alpha_{h,i}(n) = \lceil log_2(max(|(x_0)_i|, 1)) \rceil + n
\end{aligned}
\tag{3.4}
$$

And for the last step, with an arbitrary h ($r = |Mh|$ for convenience), solving for $\alpha_{h,i}(n)$ yields

$$
\begin{aligned}
& |(E_{\alpha_{h,i}(n)})_i| \leq 2^{-n} \\
\Leftarrow & \frac{|c_i| r^{\alpha_{h,i}(n)+1}}{1-r} \leq 2^{-n} \\
\Leftrightarrow & 2^{log_2(r)\alpha_{h,i}(n)+1} \leq \frac{(1-r)2^{-n}}{|c_i|} \\
\Leftrightarrow & log_2(r)\alpha_{h,i}(n) + 1 \leq log_2\left(\frac{(1-r)}{|c_i|}\right) - n \\
\Leftrightarrow & \alpha_{h,i}(n) + 1 \geq \frac{log_2\left(\frac{1-r}{|c_i|}\right) - n}{log_2(r)} \\
\Leftrightarrow & \alpha_{h,i}(n) + 1 \geq \frac{log_2\left(\frac{|c_i|}{1-r}\right) + n}{log_2(1/r)} \\
\Leftrightarrow & \alpha_{h,i}(n) \geq \frac{log_2\left(\frac{max(|(x_0)_i|,1)}{1-r}\right) + n}{log_2(1/r)} - 1 \\
\Leftarrow & \alpha_{h,i}(n) = \left\lceil \frac{log_2\left(\frac{max(|(x_0)_i|,1)}{1-r}\right) + n}{log_2(1/r)} \right\rceil - 1
\end{aligned}
\tag{3.5}
$$

If the Taylor series is finite (when the solution to the differential equation is a polynomial), the whole stepping process can be skipped since the series is an exact formula for the function and can be calculated for arbitrarily big steps without the need for limits, which can be heavy on performance, especially when nested. However, since it is impossible to verify if the series is finite by iterating it (because iterating an infinite list will never terminate), Jaguar only checks if its length is at most five. This will include most simple physical systems, which are defined with non-recursive low-order derivatives (e.g. velocity and acceleration independent of position). If so, the exact solution is taken; otherwise, the stepping strategy

is employed. This and other aspects of the solver have plenty of room for improvement in the future, and any optimizations in this component will translate directly to a faster performance of the language overall.

3.6 Visualizer

The visualizer takes in the hybrid evolution produced by the interpreter and plots the value of each variable versus time by evaluating the evolution at multiple sample points. To generate the plot, the Chart [11] library was used.

To place a computable real on the plot, it is first approximated with some accuracy (which can be provided as a parameter by the user), generating a range of values around the approximation that bounds the actual value of the real. The approximations are then joined together with a line and the ranges are joined as the shaded areas. Fig. 9 and Fig. 10 both show the plot of the same program, shown below. Fig. 10 was generated with a higher accuracy, making the shaded area so small it is no longer visible.

```
1  y := 0; v := 1; g := 1; c := 0.8;
2  while true do {
3    y'=v,v'=-g for 2 * v / g;
4    v := -c * v;
5  };
```

Listing 3.9: Jaguar code for an (inelastic) bouncing ball

Since the exact duration of each bounce is known ($2 * v_i/g$), the program is written such that the ball's position is never negative[1]; so we would expect the plot to never cross the x-axis. However, Fig. 9 shows the line clearly going below the x-axis and even looping backwards at the bouncing points ($t = 2$, $t = 3.8$, etc.). This happens because of the semidecidability of hybrid branches talked about in Section 3.3.4. Querying the program immediately after $t = 2$, for example, results in two possible branches: the continuation of the downward branch, going into the negatives, and the new upwards branch. Since the visualizer knows that the downwards branch is before the upwards branch (see next section for details), the line goes down and then connects back to the beginning of the upwards branch before $t = 2$, resulting in a loop. These artifacts disappear when accuracy is increased (Fig. 10).

3.6.1 Discontinuity Tracking

It was already explained in Section 3.3 how discontinuities (which may arise at the point of concatenation of two hybrid programs) are handled locally, when making individual queries. However, to facilitate a big

[1] Some formulations of the bouncing ball in the literature don't use the $2 * v_i/g$ formula. Instead, they repeatedly check if the ball has reached the ground at constant intervals. These formulations really *do* result in negative positions for the ball, since the ball moves unimpeded until the position is checked. The formulation presented in Listing 3.9 deviates from these others to illustrate how the visualizer can create visual artifacts (*e.g.* showing the line crossing the x-axis even when the real position is never negative).

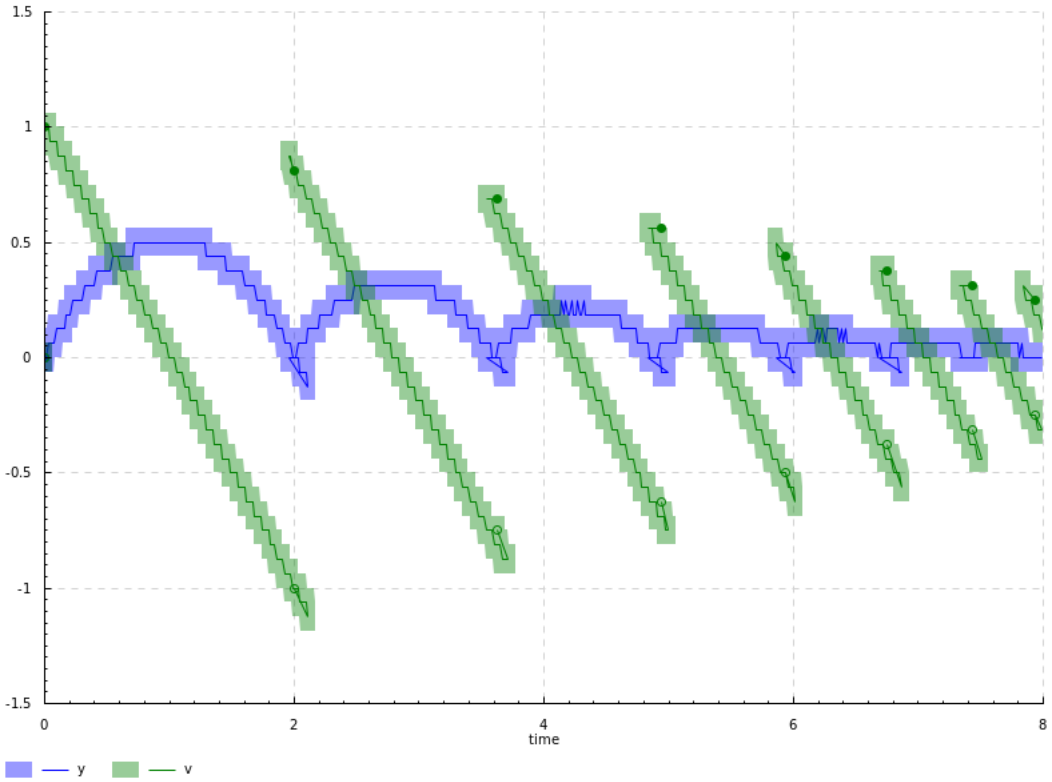


Figure 9: Plot of a Jaguar program with an accuracy of $n = 4$

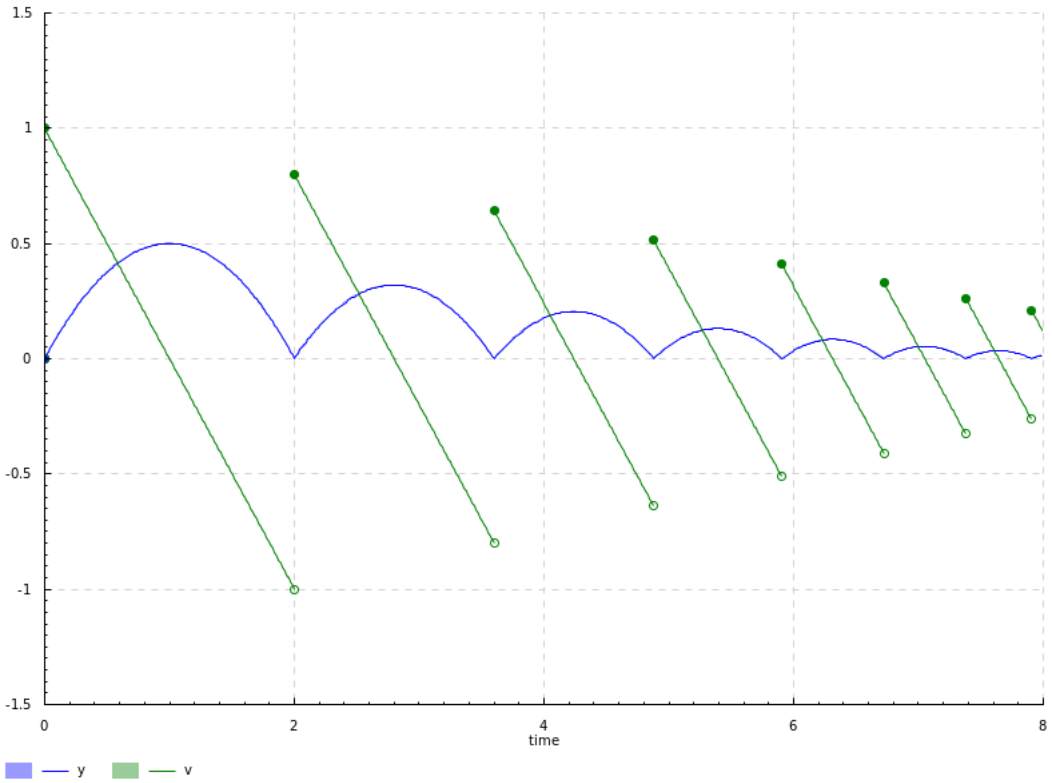


Figure 10: Plot of a Jaguar program with an accuracy of $n = 10$

picture analysis of a program, Jaguar also includes tools for tracking discontinuity points throughout the whole trajectory (shown in the plots as the empty and full circles). This is as simple as keeping track of every point at which one or more assignments are made[2]. Not every assignment effectively generates a discontinuity, since it may not change the state, but checking if the state changed requires comparisons (which, as seen before, leads to semidecidability troubles), so all assignments are tracked. The resulting list of discontinuities is returned alongside the hybrid evolution as the output of a Jaguar program.

Each discontinuity is located between two differential trajectories. To match each discontinuity to its neighboring hybrid branches without directly comparing the time of each, each discontinuity and differential trajectory also include order information in the form of a step counter. Basically, alongside the state of the system, Jaguar keeps a counter that starts at zero and increases with both differential statements and assignments. This way, each differential trajectory is associated with a unique step number. This step number is also used to connect the lines of different hybrid branches in the visualizer.

A discontinuity is described by all that information put together: a real number corresponding to the time of the discontinuity; the hybrid state and step number "before" the discontinuity (as in, the state the system would have at the moment of discontinuity if the program ended without the assignment(s) that caused the discontinuity); and the hybrid state and step number at the moment of the discontinuity. Discontinuities are returned as a lazy list, taking advantage of the right associativity of Jaguar's parsing.

[2] The solutions to the differential equations are assumed to be continuous. Since the solver relies on Taylor series, the generated solutions will always be continuous evolutions anyway. No effort was made to detect or simulate mathematical discontinuities.

Chapter 4

Results

This chapter is dedicated to the benchmarking results of both the exact real arithmetic packages and the hybrid simulator. To understand the obtained results, it is important to build up from the basics, first analyzing the performance of the basic operations and how they interact, before looking at the benchmarks of the simulator. The chapter is divided into Section 4.1, focusing on the basic operations of the five packages; Section 4.2, which analyzes benchmarks for the differential equation solver and the hybrid simulator as a whole; and Section 4.3, where a practical case study is formulated and analyzed.

Since there are five libraries being compared with one another, a big part of this analysis will be an explanation on where their differences in performance come from, be it from fundamental differences in their respective strategies or from some specific implementation detail. These comparisons will then allow us to infer which libraries are the most appropriate to use with the hybrid simulator.

All code was compiled with the Glasgow Haskell Compiler (GHC) 9.10.3 using Stack 3.3.1 and executed on Ubuntu 22.04 running on an AMD Ryzen 5 UM425UA laptop with 8GB RAM. The results were obtained by performing multiple runs of the code samples (at least 10 runs for the slowest tests) with the help of benchmarking library criterion [37]. The presented results are the arithmetic averages of the multiple runs. The full code can be found on github [4] and run with stack, but the source code for each test was also included in the appendix (Appendix C) for ease of reference.

4.1 Exact Real Arithmetic Benchmarks

To measure the performance of an operation on computable reals is effectively to measure the performance of approximating the result of that operation to some accuracy n. This means that, right from the start, there is an extra axis to our analysis: the performance of an operation usually depends on the requested accuracy, which is why the presented benchmarks are graphs of execution time (in milliseconds) over accuracy. Logarithmic axis are used to make asymptotic complexity more visible. Although constant

factors (which in logarithmic axis appear as translations) are somewhat important, for large computations the asymptotic behavior (the shape and slope of the curves) as the accuracy increases is the most helpful predictor of performance, so that is where this analysis focuses the most.

The simplest operation we can start by analyzing is $fromRational$ (Fig. 11), which, recall from Section 2.2.1, promotes a rational to a computable real. Since the analyzed packages work with dyadic approximations, approximating the result of $fromRational$ entails generating a dyadic number that approximates the wanted rational within a certain accuracy. As the accuracy grows, the size of the numerator and denominator grow by the same amount, which explains the (approximately) linear cost of the operation after $n \approx 10^3$, where constant costs (the initial horizontal section) are outweighed. Looking a bit closer at the results, there also seems to be a logarithmic factor at play ($O(n \log n)$ or $O(n \log^2 n)$), probably related to Integer operations, in all packages except *exact-real*. This difference is likely due to implementation specific optimizations, such as the use of bitshifting operations instead of exponentiations and multiplications, which introduce a logarithmic factor.

There is also $fromInteger$ (Fig. 12), which works the same way but limited to integer inputs. Since most rationals can't be represented exactly by a dyadic, while integers can, nested intervals representations (*CDAR* and *aern2-real*) can optimize $fromInteger$ by storing a single interval with zero width, which is a valid approximation for any accuracy. This can also be done in $fromRational$ with rationals whose denominator is a power of 2 (Fig. 13), which *aern2-real* also seems to do but *CDAR* doesn't, presumably to avoid extra logic and checks that could slow $fromRational$ down (notice how *CDAR*'s non-dyadic $fromRational$ is faster than *aern2-real*'s by a constant factor).

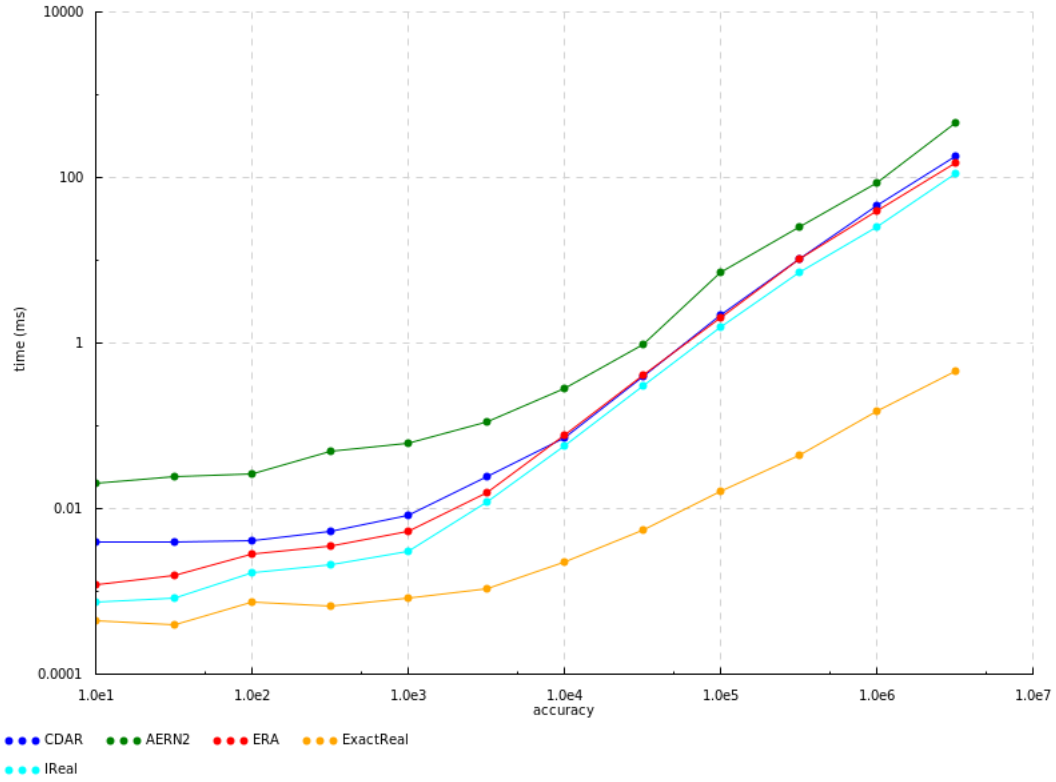


Figure 11: Benchmarks for $fromRational\ \frac{22}{7}$, scaled logarithmically

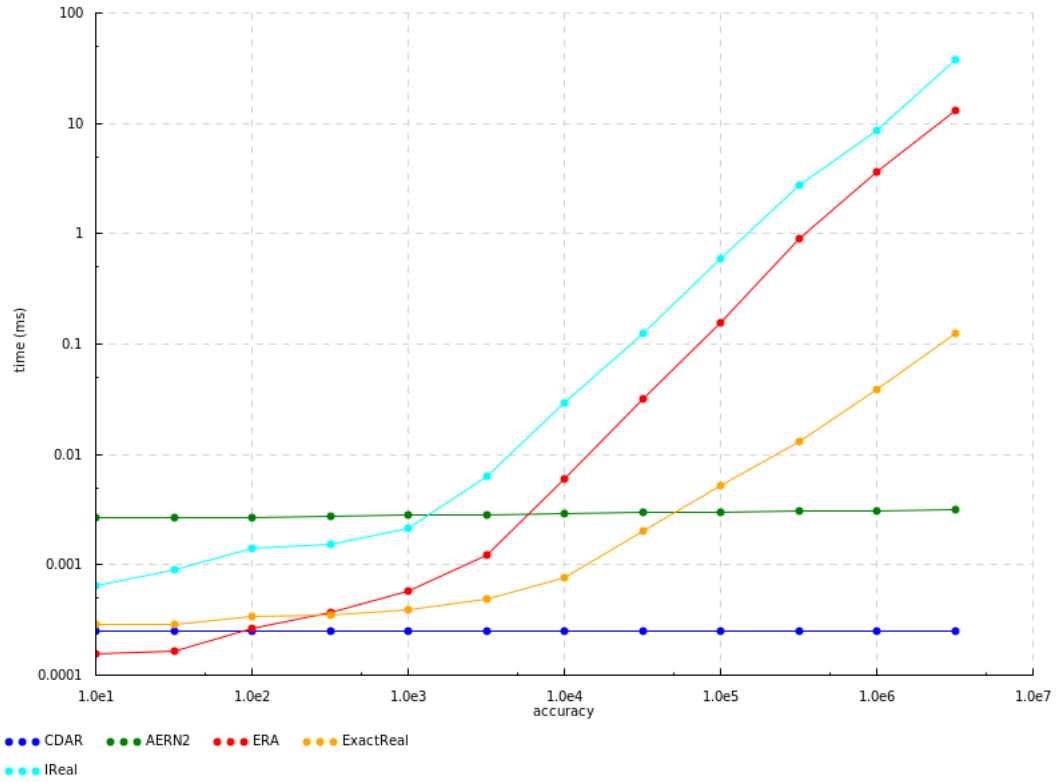


Figure 12: Benchmarks for $fromInteger\ 1$, scaled logarithmically

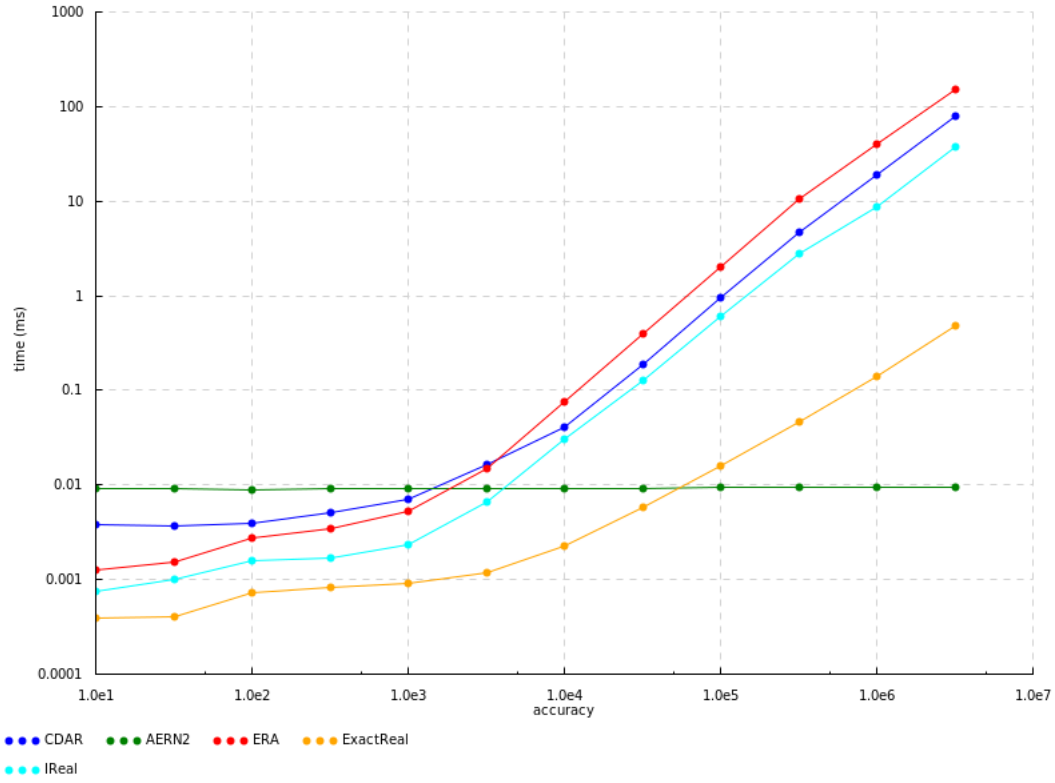


Figure 13: Benchmarks for $fromRational\ \frac{9}{64}$, scaled logarithmically

As expected, the constructor $cons$ (Fig. 14) has a very similar performance to non-dyadic $fromRational$, since they are essentially the same operation. However, something to keep in mind with $cons$ is that, since it receives a function or list, the performance of the resulting computable real is tied to the performance of the function it was constructed from. If instead of a constant function we pass $cons$ the partial sums of a series (Fig. 15) for example, the times rise significantly and are pretty much the same between the packages, since the bottleneck is now calculating and iterating the list.

The same is true for $limit$, which works very similarly to $cons$. Fig. 16 shows the limit of the same series as Fig. 15 where each element is first calculated as a rational and them converted to a computable real with $fromRational$, and they correspond perfectly. However, the performance of $limit$ has yet another factor: the performance of each individual real in the list. Even if the list itself is fast to calculate, if each element of the list is a slow computable real, so will be the resulting limit.

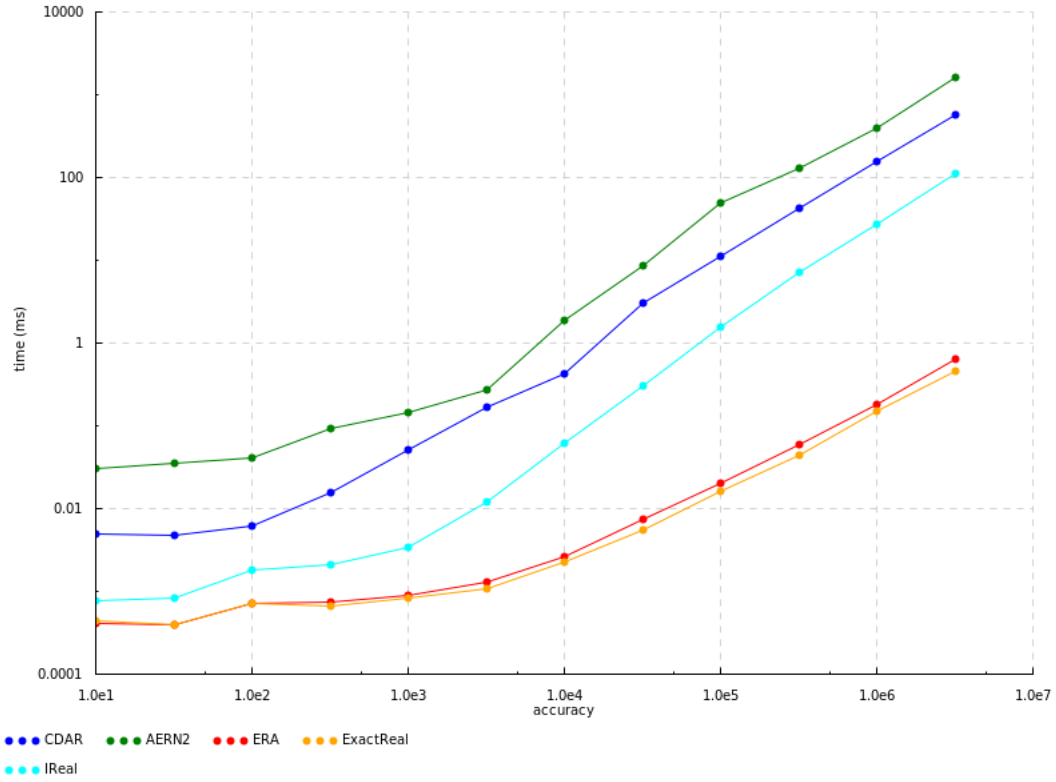


Figure 14: Benchmarks for $cons\ (const\ \frac{22}{7})$, scaled logarithmically

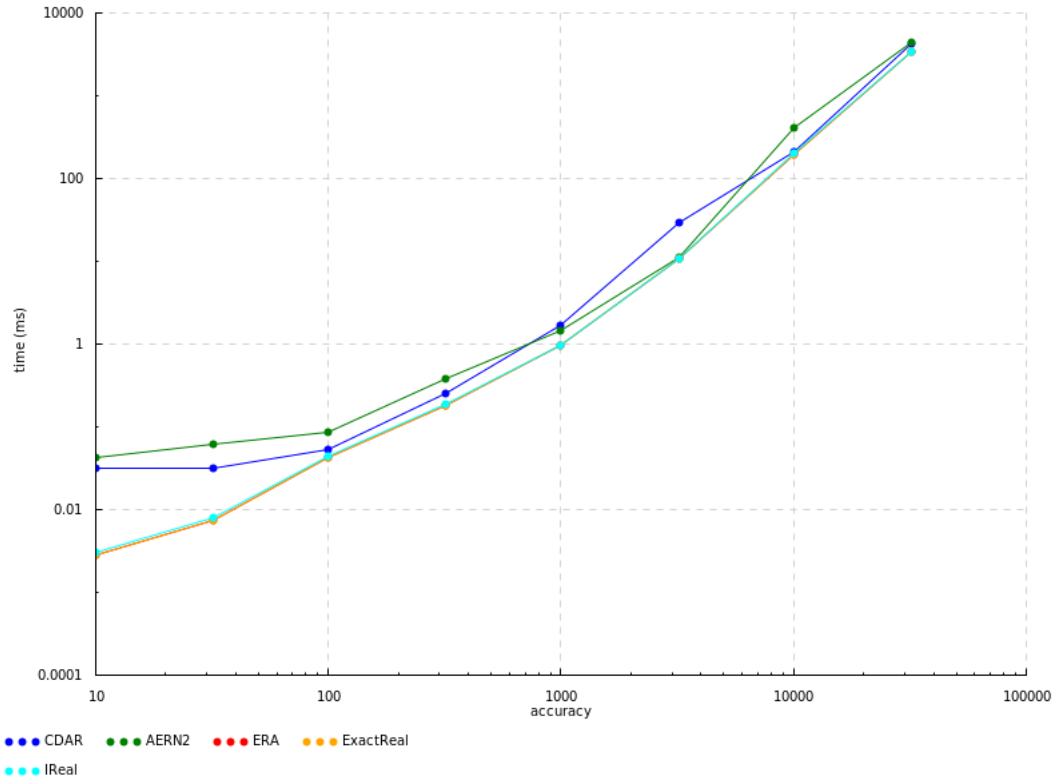


Figure 15: Benchmarks for $cons$ of a series, scaled logarithmically

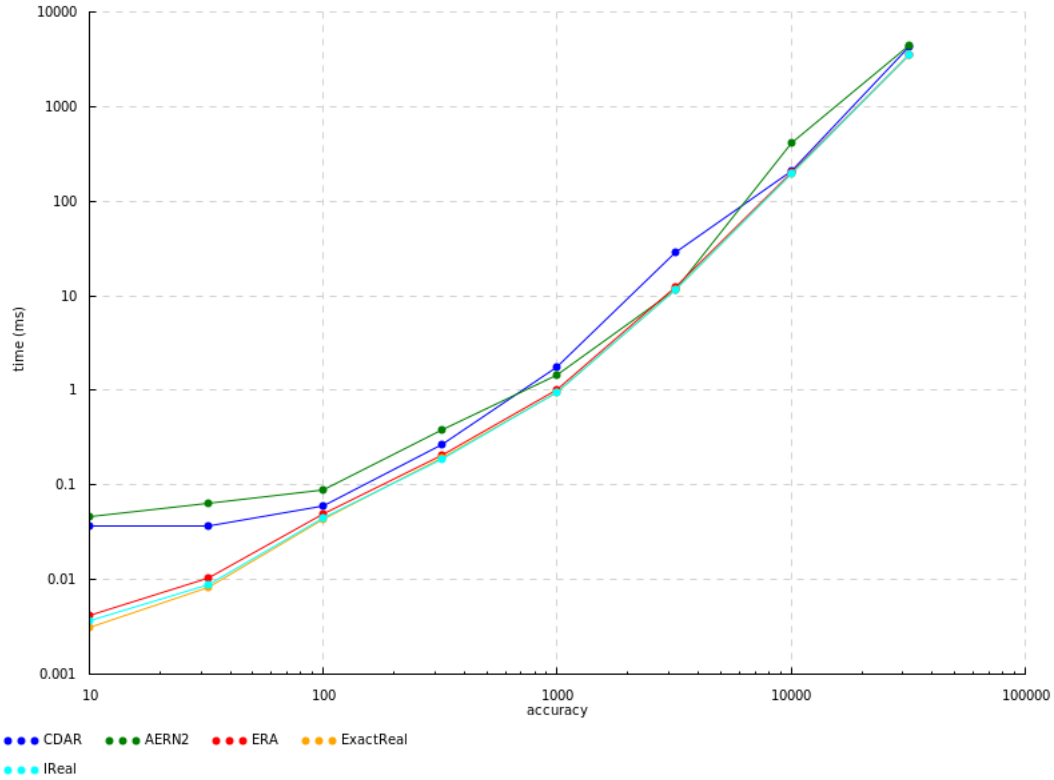


Figure 16: Benchmarks for $limit$ of a series, scaled logarithmically

This point is worth emphasizing: since computable reals are basically approximation functions, the performance of all operations on computable reals will necessarily depend on the performance of the inputs. This is why it is essential to have performant constructors ($fromInteger$, $fromRational$ and $cons$), since these produce the building blocks that other operations then use for computations.

That is not to say that the operations themselves have no effect on performance. Quite the opposite, in fact: the biggest difference in performance among the packages is seen on calculations with a high operation depth, and can be explained by their representation strategy. To demonstrate this concept we will analyze the performance of adding together a large list of size k. Other operations will behave similarly, but addition is easier to analyze because errors in the approximations are distributed evenly between terms and are independent of the values of the terms themselves, unlike, for instance, multiplication, which can scale errors up or down depending on the value of the factors. The terms in the list are all non-dyadic numbers generated with $fromRational$, so all packages start on roughly equal footing.

In Section 2.2.3 it was explained how addition works on modulus of convergence representations (*ERA*, *exact-real* and *ireal*): the n-th approximation of the sum is obtained by adding together the $n + 2$ approximation of each term, dividing by four and rounding to the nearest dyadic of denominator 2^n. However, if multiple sums are performed in sequence, like so:

$((a + b) + c) + d$

then this approach has the unfortunate consequence of escalating the accuracy requested from each term. In this example, the term d will be evaluated at $n + 2$ accuracy, the term c at $n + 4$ and terms a and b at $n + 6$. The deeper a number is in the nesting of operations, the more accuracy it requires. And because more accurate approximations have bigger precision in modulus of convergence representations ($p \approx n$, therefore $O(p) = O(n)$), adding and dividing them is also slower. The time complexity of division depends on the complexity of the used multiplication algorithm, which will be denoted as a function $M(p)$. So the total time complexity of the sum of the list equals $\sum_{i=1}^{k} O(M(n+2i)) = O(kM(n+k))$, which for the fastest multiplication algorithm $M(p) = p \log p$ results in $O((k^2+nk) \log (n + k))$. This seems to match the quadratic growth of the three modulus of convergence packages in Fig. 17 and Fig. 18. *ireal* does have a small spike at $k = 10000$ for reasons that elude me, but otherwise follows the pattern.

Nested intervals representations don't have this problem because their operations are a zip. It is trickier to calculate an asymptotic complexity for them, since they have much more freedom in the precision and accuracy of their intervals and how intervals are searched, but using same assumptions as before ($O(p) = O(n)$), we can intuitively say that complexity should be around $O(kM(n))$. Looking at Fig. 17, both appear to be linear in k as predicted, despite *CDAR* gaining a greater constant factor at $k = 320$.

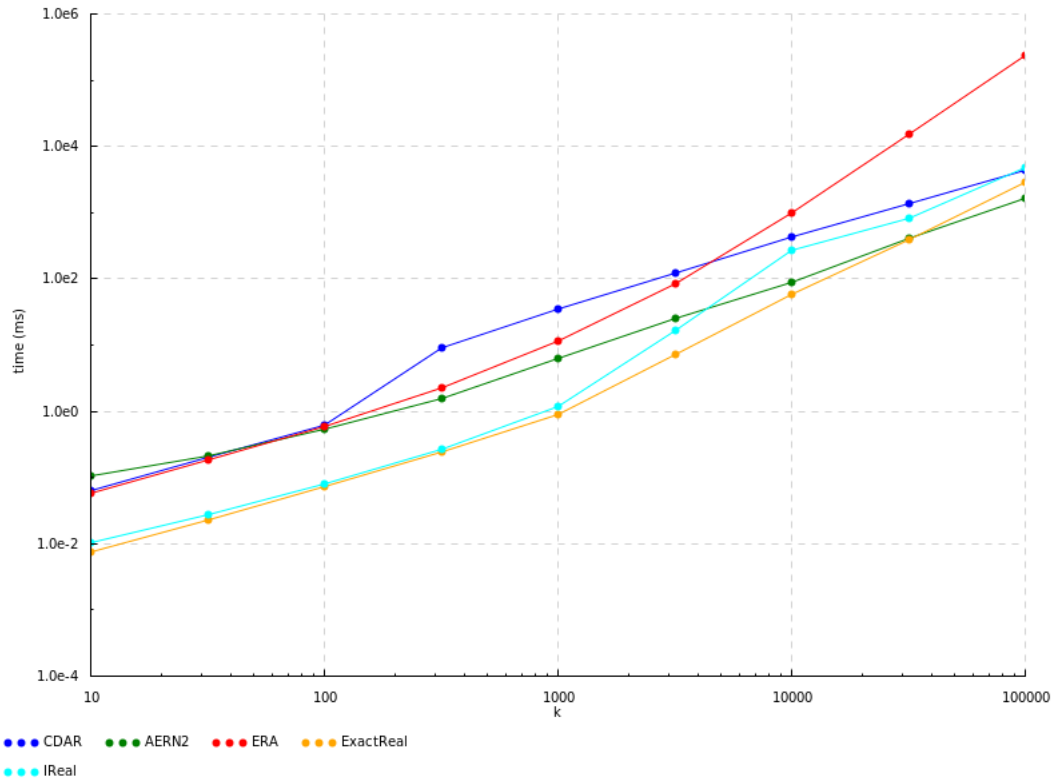


Figure 17: Benchmarks for sum of a list of k elements at an accuracy of $n = 1000$, scaled logarithmically

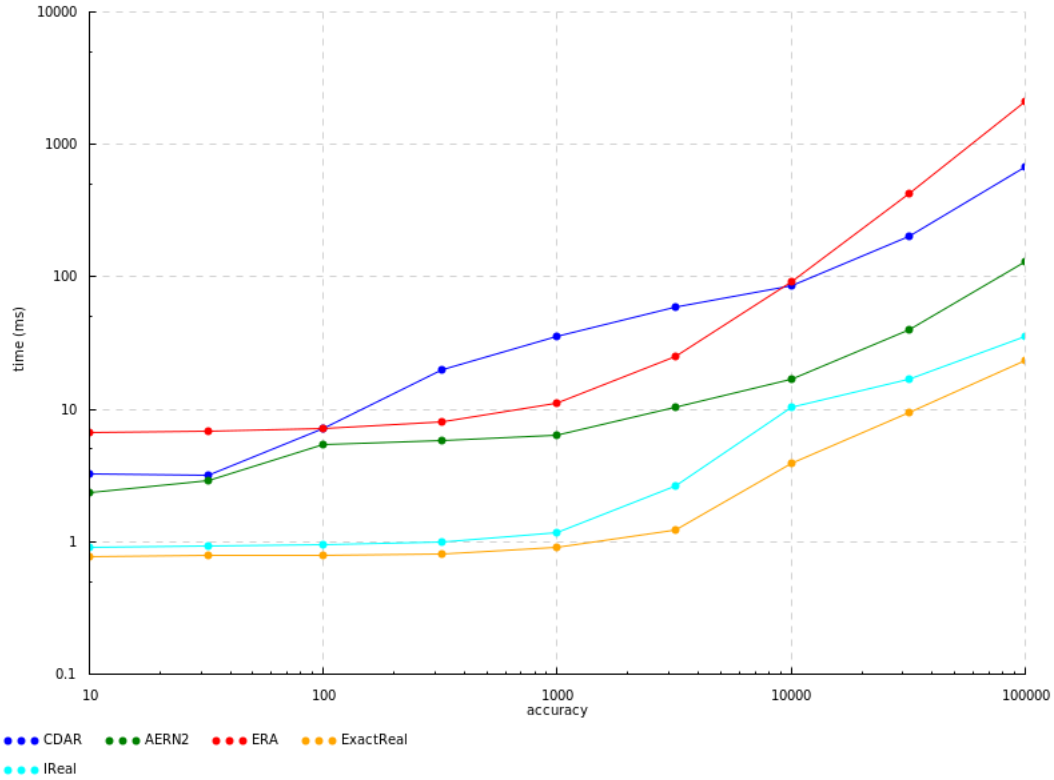


Figure 18: Benchmarks for sum of a list of $k = 1000$ elements, scaled logarithmically

Smart readers may have already realized that, since addition is associative, there is a simple optimization available in this specific case. By rearranging the nesting of the parenthesis into a tree, it is possible to minimize the operation depth to at most $\log_2 k$:

$$(a + b) + (c + d)$$

This way, the accuracy required from each number will be at most $n + \log_2 k$, and so the total time complexity is merely $\sum_{i=1}^{k} O(M(n + \log k))$ for modulus of convergence representations, which is a massive boost, as shown in Fig. 19. Nested intervals representations, as expected, aren't affected. Sadly, this can only be done in specific cases like this one and doesn't generalize to non-associative operations.

To reiterate, although this simple example uses addition, the insight it provides is applicable in any calculation with a high degree of nesting, and explains why nested intervals representations have the advantage in long, nested computations such as a hybrid simulation. This advantage compounds with the $fromInteger$ optimization, making modulus of convergence representations excruciatingly slow by comparison in all but the simplest hybrid programs, as we will see in the next section.

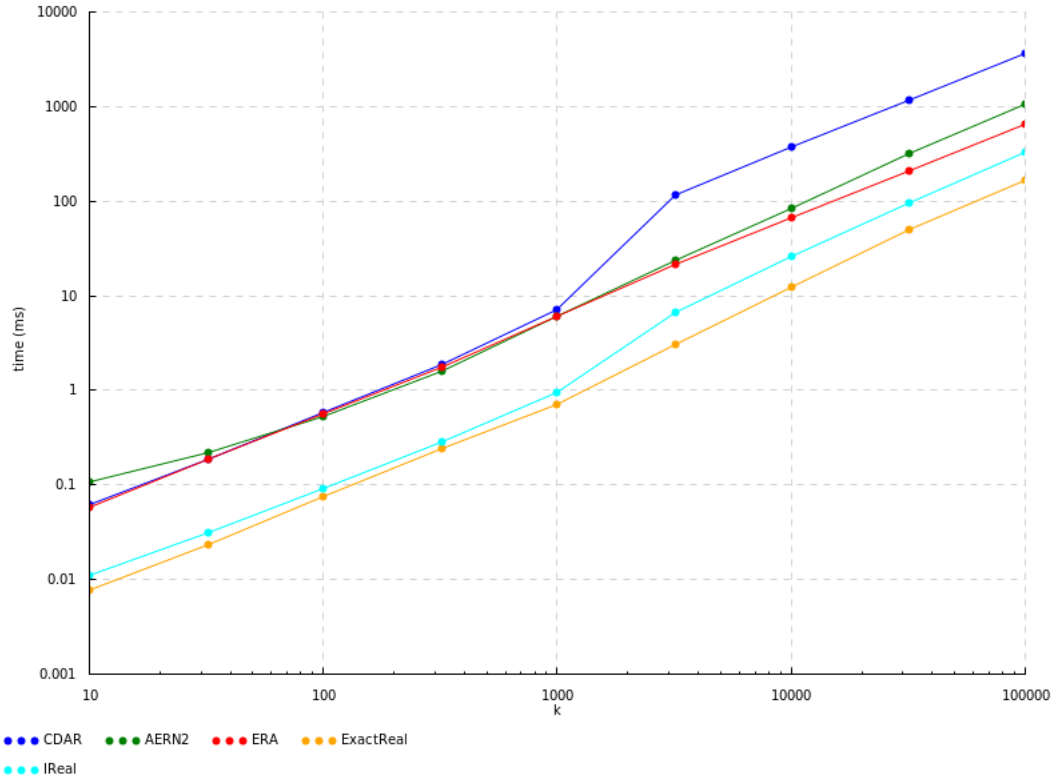


Figure 19: Benchmarks for sum of a tree of k elements at an accuracy of $n = 1000$, scaled logarithmically

While the basic arithmetic operations are implemented similarly in all packages, more complicated operations such as trigonometric functions can be calculated using various methods, and different packages choose different methods, some more efficient than others. Fig. 20 shows the performance of each package when calculating the sine of numbers between $-\pi$ and π, after which the pattern repeats since sine is periodic.

While these differences in performance should definitely be taken into account when choosing the most appropriate package for some specific application, they are not fundamental to each package's representation strategy. These differences come from different methods of calculating sine, not from the underlying strategy, and it would be entirely possible to optimize the slowest packages by switching to a faster method (though it may be true that the *optimal* method is somehow dependent on the package's representation strategy, such that one strategy leads to a faster optimal implementation of sine over another).

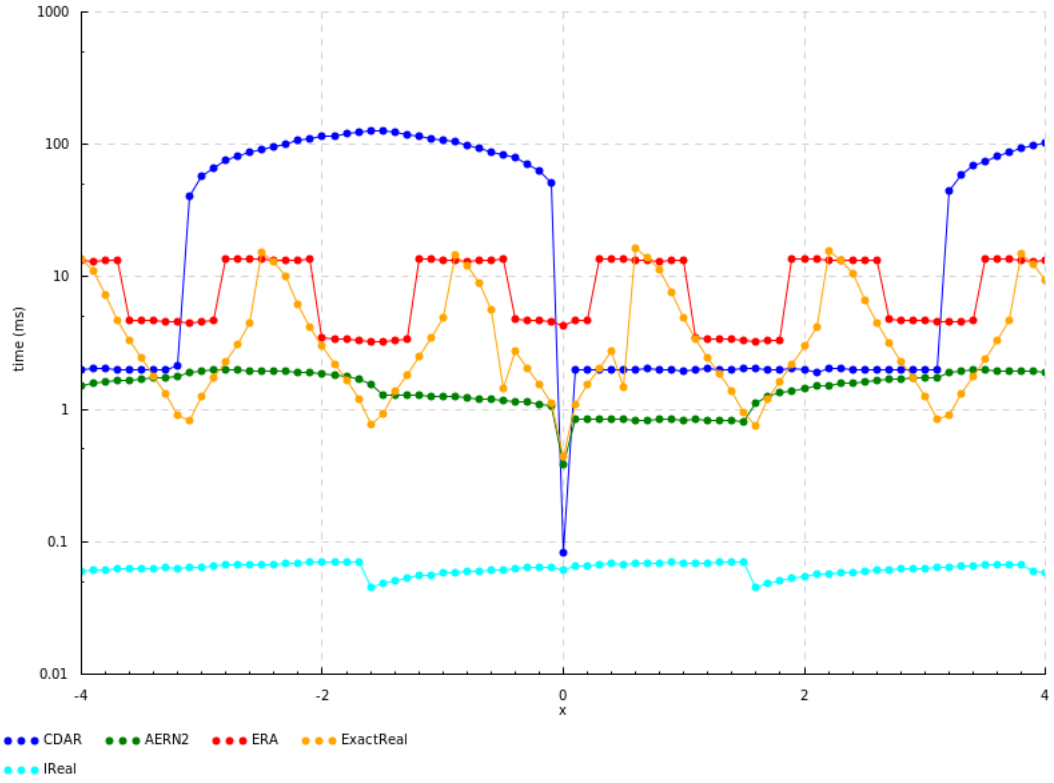


Figure 20: Benchmarks for $sin(x)$ at an accuracy of $n = 1000$, scaled logarithmically in the y-axis

The last major difference in performance between packages is explained by memoization. As mentioned in Section 2.2.2, memoization is an optimization technique that avoids performing repeated calculations by storing their value in a variable or data structure. In the context of exact real arithmetic this can be done in two different ways: type 1, in-order memoization, and type 2, out-of-order memoization. Of the analyzed packages, *CDAR* and *aern2-real* implement type 1 while *exact-real* and *ireal* implement type 2.

In all benchmarks shown so far, memoization hasn't had the opportunity to influence results because all calculations are only used once. We can optimize one of the previous tests, the sum of a tree, to make use of memoization by using the observation that all elements of the list are the same, and therefore sums of the same number of terms should have the same result. The resulting code is very similar to the usual implementation of exponentiation, which uses the same observation to optimize repeated multiplications:

```
1 dp_sum x 1 = x
2 dp_sum x k = case k `mod` 2 of
3                0 -> y
4                _ -> x + y
5     where y = dp_sum (x + x) (k `div` 2)
```

Listing 4.1: Haskell code for the sum of a list of k instances of the same value

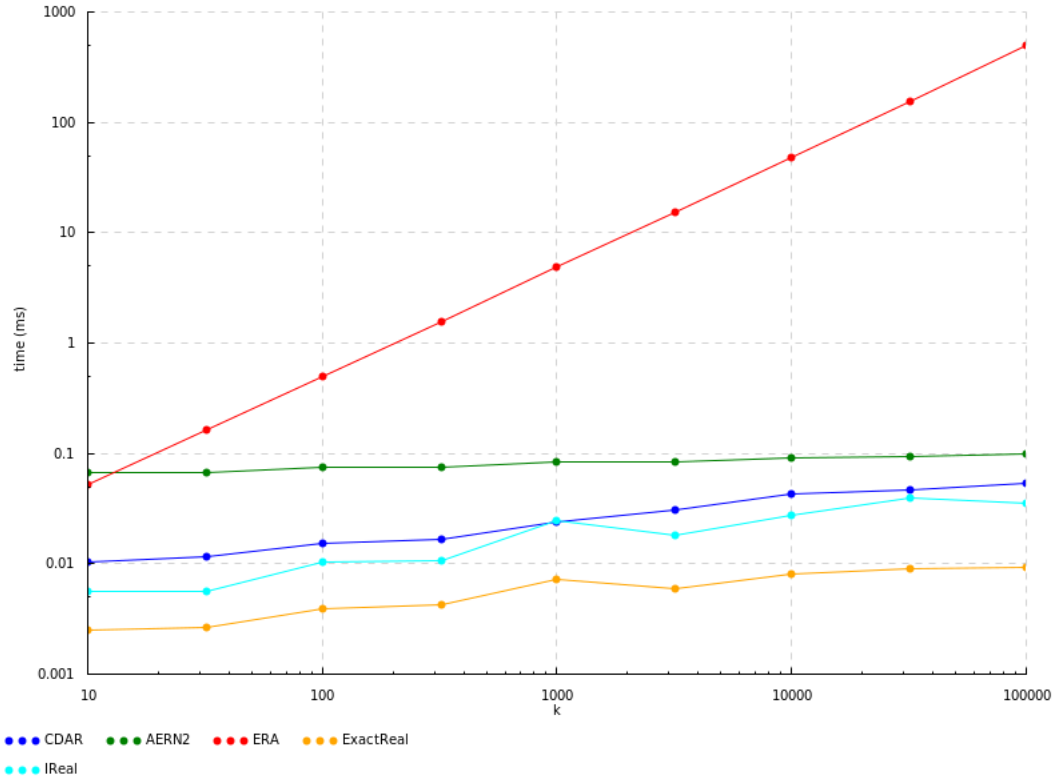


Figure 21: Benchmarks for $dp\_sum\ (fromRational\ \frac{22}{7})\ k$ at an accuracy of $n = 1000$, scaled logarithmically

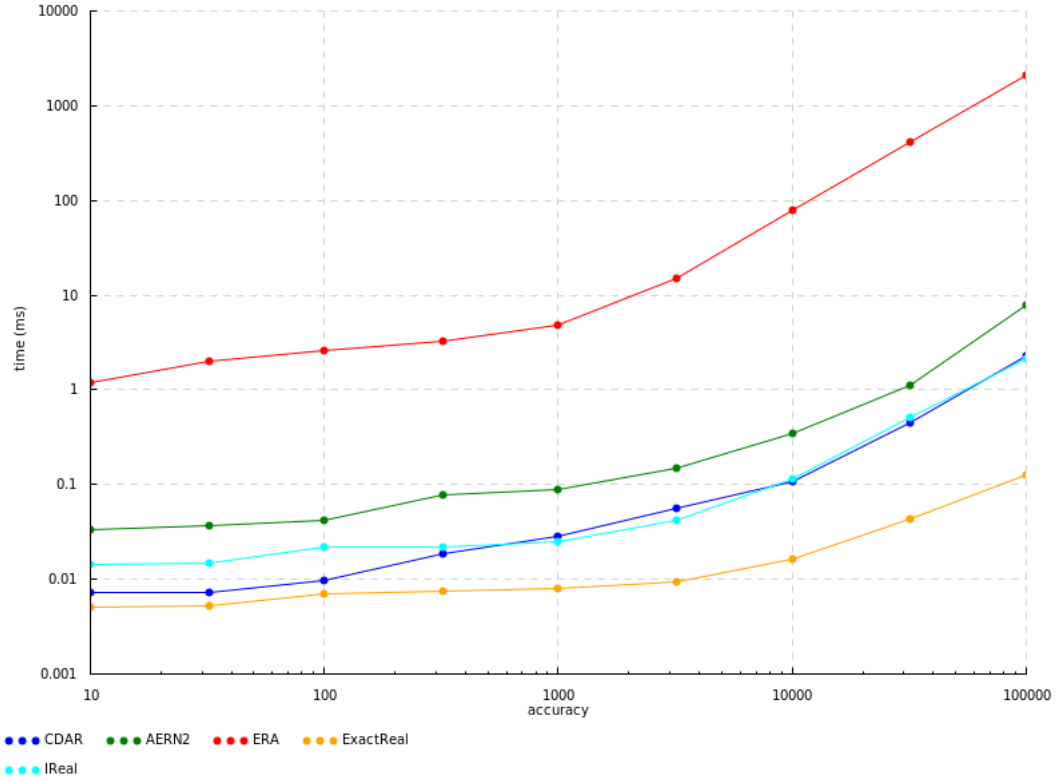


Figure 22: Benchmarks for $dp\_sum\ (fromRational\ \frac{22}{7})\ 1000$, scaled logarithmically

Unlike previous tests, where k operations are needed to add the k elements together, this test only takes between $\lfloor \log_2 k \rfloor$ and $2\lfloor \log_2 k \rfloor$ exact real arithmetic operations[1] by reusing the value x multiple times per step. If the package supports memoization, the complexity of the test is logarithmic on k. If it doesn't, however, each x is unrolled into its own calculation, which then recursively unrolls further, effectively recreating the structure of the tree this code attempted to optimize.

The results are shown in Fig. 21 and Fig. 22. As we would expect, only the packages that implement memoization show improvements, while *ERA*'s performance stayed the same from Fig. 19. It is also interesting how *exact-real* and *ireal* follow the same pattern in Fig. 21, since they have the same memoization type and similar implementation. If the reader is confused as to how their runtime isn't monotonic in k (*e.g.* $k = 1000$ has a greater runtime than $k = 3200$), this is because the number of operations performed is not proportional to $\log_2 k$, but oscillates between $\lfloor \log_2 k \rfloor$ and $2\lfloor \log_2 k \rfloor$. It just so happens that $k = 1000$ needs more operations than $k = 3200$, and, as we have seen, modulus of convergence representations are highly sensitive to the number of operations when these aren't nested properly.

[1] dp_sum performs either 1 or 2 operations if k is even or odd, respectively, and then recursively calls itself with k divided by 2. This is equivalent to counting the number of ones and zeroes in the binary representation of k (ignoring the first bit, which is always a one), each adding 2 and 1 operations, respectively. Therefore, the total number of operations must be between $\lfloor \log_2 k \rfloor$ (all bits except the first are zeroes) and $2\lfloor \log_2 k \rfloor$ (all bits are ones).

4.2 Hybrid Simulator Benchmarks

After analyzing the fundamentals, we can now look at the hybrid simulator. Obviously, the performance of the simulator will vary greatly depending on the hybrid program – more complex programs perform more complex operations and a greater number of them, which leads to slower runtimes.

There are two sources of complexity in hybrid programs: instantaneous changes (variable assignments and conditional structures) and continuous changes (differential statements). The complexity introduced by instantaneous statements is immediately visible: the more complicated the expression (and the more nesting it introduces, in the case of modulus of convergence representations), the more it contributes to the total runtime. To illustrate this with a simple example, the following program recreates the sum of a list analyzed previously as a hybrid program. The runtimes are presented in Fig. 24.

```
1  x := 0;
2  while true do {
3    wait 1;
4    x := x + 22/7;
5  }
```

Listing 4.2: Hybrid program that at time t calculates the sum of $\lfloor t \rfloor + 1$ elements

For this specific hybrid program, the parameters of the simulation (comparison accuracy and iteration limit) don't really matter, since there are no comparisons and no risk of zeno behavior. However, the query accuracy does matter. Since each wait has a duration of 1, the query accuracy can be safely set to 3, which guarantees that queries will only return branches at most $2^{2-n} = 2^{2-3} = 0.5$ time units away, as shown in Fig. 23. Therefore, queries may hit at most two possible branches. Since our queries will be made at integer values of t, the correct branch must be the second one.

Even from this simple example it is obvious how slow modulus of convergence representations really are. This hybrid program is noticeably slower than the equivalent benchmark in Fig. 17. This can be explained by simulator overhead: besides calculating the variables, Jaguar also needs to keep track of the simulated time. Indeed, even if no other operations are performed, simply adding together the times of the wait statements is already enough to produce a quadratic time complexity for modulus of convergence representations. The following program does just that, and the resulting runtimes are shown in Fig. 25.

```
1  x := 0;
2  while true do wait 1;
```

Listing 4.3: Hybrid program that does nothing, one time unit at a time

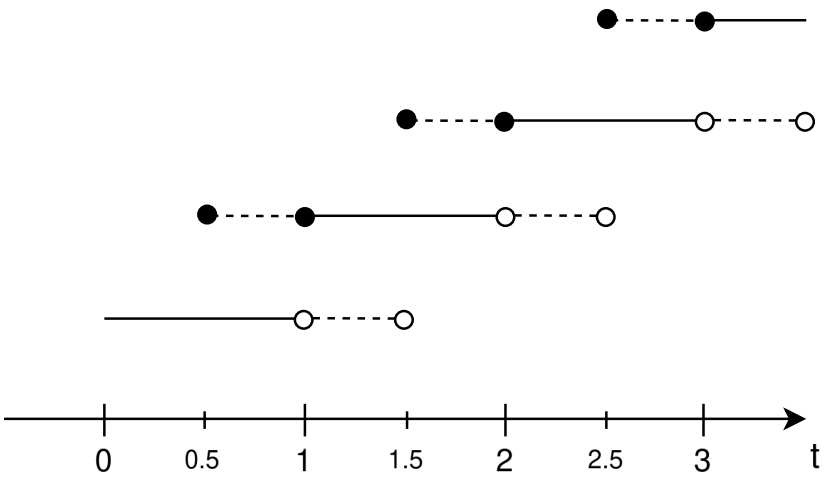


Figure 23: Diagram representing hybrid semidecidability for sum of a list program with query accuracy $n = 3$

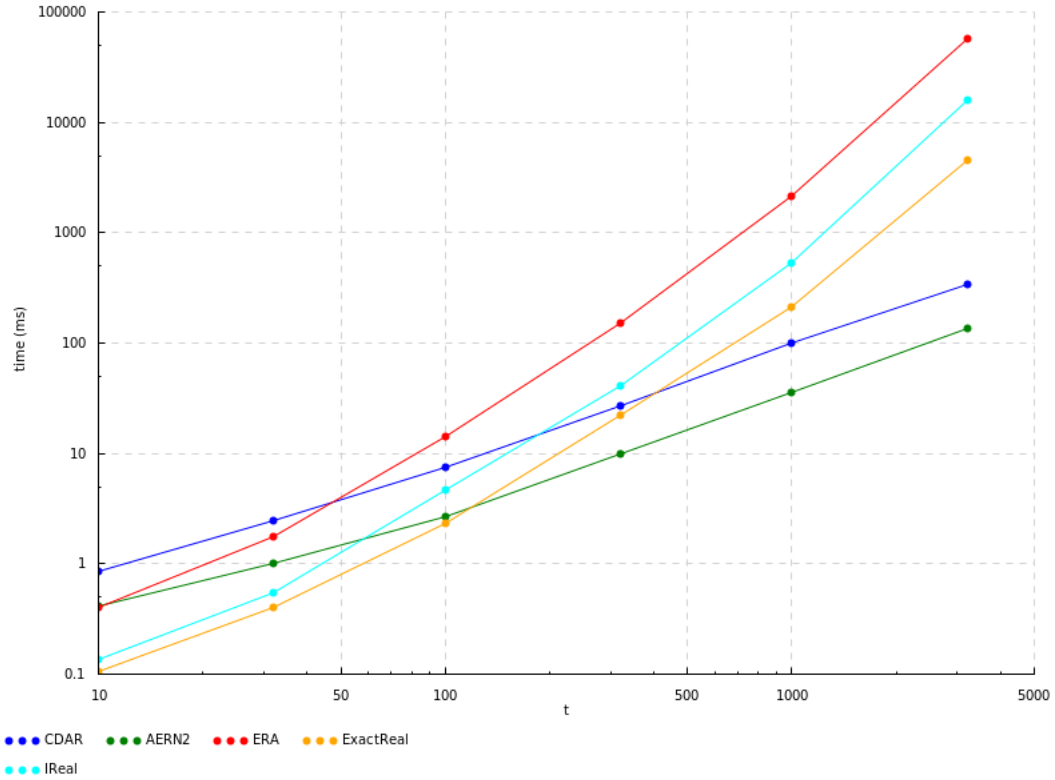


Figure 24: Benchmarks for a hybrid program that calculates sum of $\lfloor t \rfloor$ elements, evaluated at $n = 1000$, scaled logarithmically

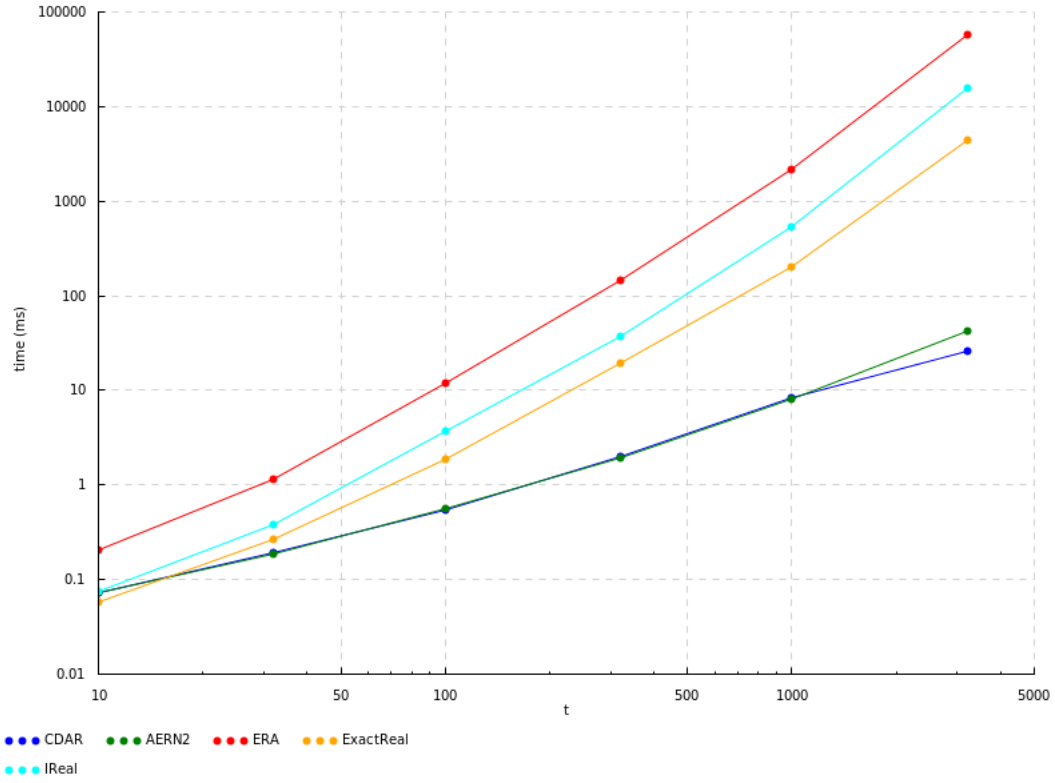


Figure 25: Benchmarks for loop of wait statements, evaluated at $n = 1000$, scaled logarithmically

Differential statements are a bit harder to analyze. Recall from Section 3.5 how Jaguar calculates differential statements: if the Taylor series/polynomial of the solution to the differential equation is finite and has 5 or fewer terms, the solution is given exactly by that polynomial, and so querying the state of the hybrid system only requires evaluating the polynomial. Therefore, if there are multiple such differential statements in a program, each one essentially acts like an assignment, setting each variable to the corresponding polynomial evaluated with the duration of the differential equation.

If, on the other hand, the Taylor series is infinite or has more than 5 terms, Jaguar uses the *limit* of the partial sums of the series to calculate the solution. For a single variable and a fixed step size of h, this operation is roughly equivalent to the following Haskell code:

```
1  solve :: (CompReal r) => [r] -> Int -> r -> r
2  solve as k0 h = limit ((drop k0 $ scanl1 (+) $ zipWith (*) as $ map (h^)
     [0..]) !!)
```

Listing 4.4: Simplification of the differential equation solver for solutions with an infinite Taylor series

In the code above, as are the Taylor coefficients (which depend only on the differential equation) and k_0 is the initial degree of the Taylor series used in the limit (derived in Eq. (3.4); depends on the initial value). To understand the cost of this operation, let us first consider the time complexity of calculating a

single monomial of the Taylor series. Calculating a monomial (an expression of the form $a_k * h^k$) takes one multiplication and an exponentiation by k, which can be performed as roughly $log_2 k$ multiplications; however, as we saw with Listing 4.1, that optimization relies on memoization, so we should expect *ERA* to be much slower than the other libraries.

The monomials are then summed together to obtain the list of reals for the limit. At this point, an attentive reader might naturally assume that modulus of convergence representations are also at a disadvantage, since a large number of additions is performed sequentially. But actually, instead of using $scanl1$, the solver performs these additions as a tree. As a result, this summation (and the solver as a whole) has similar asymptotic complexity for nested intervals and modulus of convergence representations.

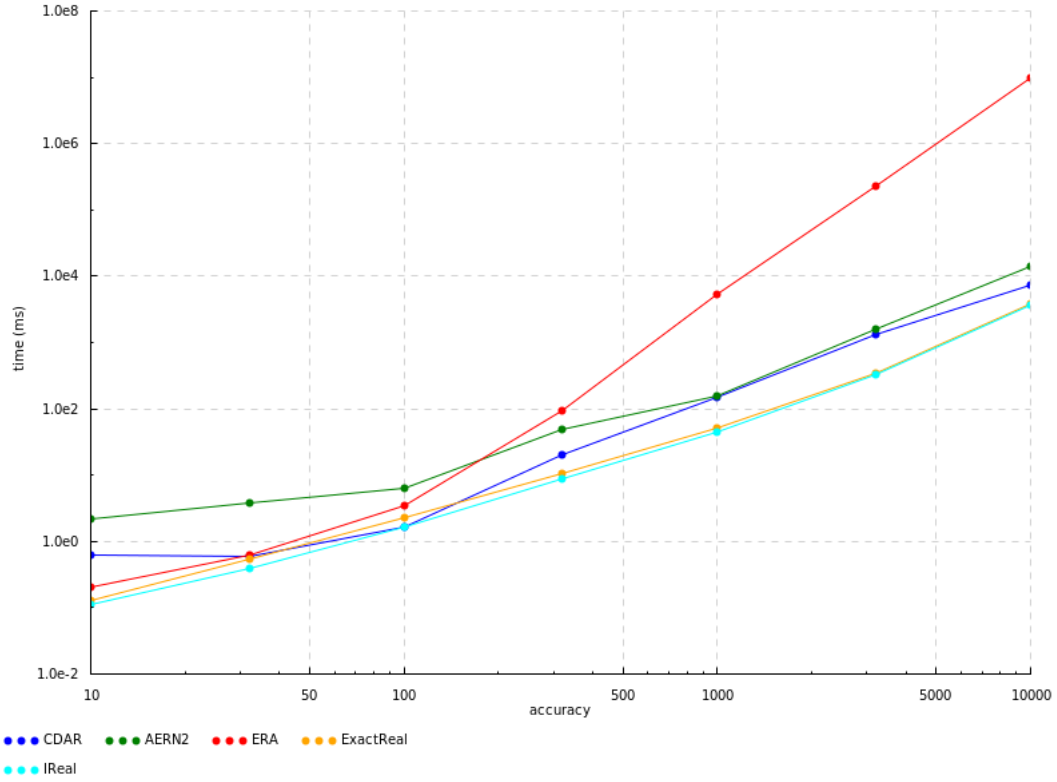


Figure 26: Benchmarks for ODE solver ($x'(t) = x(t)$) at $t = 0.4$, scaled logarithmically

Fig. 26 shows the times of solving the differential equation $x'(t) = x(t)$ with initial value $x(0) = 1$ at $t = 0.4$. This requires a single variable step, which is slightly more complex than the fixed step simplification in Listing 4.4, but the core idea is the same. As expected, all libraries except *ERA* have similar performances, around $O(n^2)$, though an exact complexity is hard to calculate from the code.

To evaluate the differential equation at a further time t (or perform multiple differential operations), the result of the $limit$ is taken as the new initial value and passed into another $limit$. For the previous example ($x'(t) = x(t)$), fixed steps have a length of 0.5, so calculating the solution for $t = 1.2$, for

instance, requires two fixed steps and one variable step of length 0.2. Since longer variable steps are slower than shorter ones, the overall runtime as t increases is a somewhat periodic series of bumps, slowly trending upwards as the number of fixed steps grows. Fig. 27 shows the results of such a test; benchmarks for *ERA* were not included because they were taking too long to generate, and the ones that didn't (for lower t's) were warping the y-axis too much. It is not entirely clear to me why *exact-real* has such a different performance from *ireal* in this test, though after some profiling it seems to be related to the specifics of how they each memoize results. For the runs I analyzed, *exact-real* spends most of its runtime packing/unpacking the memoized values, while the runtime of *ireal* and the other libraries is spent performing multiplications and additions.

In conclusion, from all the tests and benchmarks presented thus far, the nested intervals libraries *CDAR* and *aern2-real* are the most appropriate to use with the hybrid simulator for arbitrary programs, though *ireal* and *exact-real* might also be able to run some simpler programs. *aern2-real* also shows some advantage over *CDAR* in specific situations (dyadic $fromRational$ and some inputs of trigonometric functions)

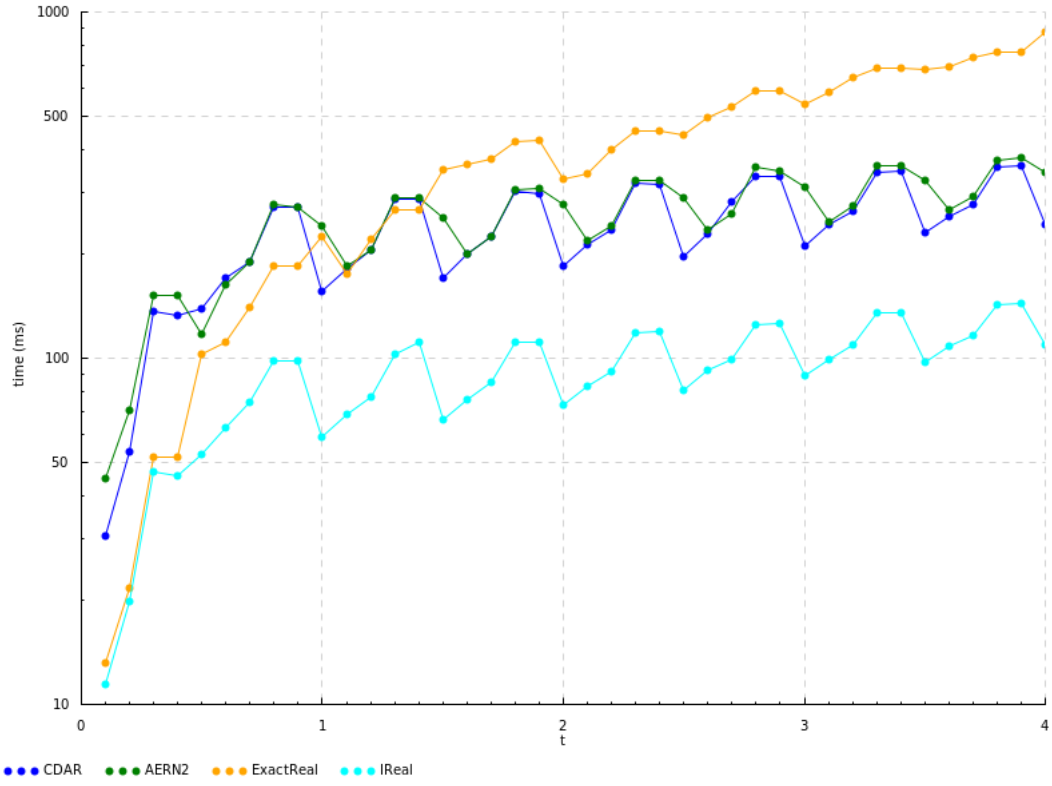


Figure 27: Benchmarks for ODE solver ($x'(t) = x(t)$) for $n = 1000$, scaled logarithmically in the y-axis

4.3 Case Study: Inverted Pendulum Balancing

This section serves as a showcase of the hybrid simulator for a concrete problem: balancing an inverted pendulum. Inverted pendulums are an extremely relevant example for exact real arithmetic since they naturally produce irrational values, and the inherently chaotic nature of pendulums causes even small errors to quickly add up. It is also an interesting problem from the perspective of control theory and a perfect demonstration of the hybrid simulator.

For simplicity, let us consider an inverted pendulum attached to the ground. Unless perfectly positioned, left on its own the pendulum will eventually fall to either side. To prevent this, every so often (20 times per second for instance) we will check if the pendulum's angle from the vertical is greater than some value, like $45°$. If so, we apply a force to the pendulum towards the center. If not, the pendulum's motion is only affected by gravity. This repeats for as long as the pendulum is above the ground.

To write a hybrid program that describes the inverted pendulum we first need to formalize the system. For the variables of our system, we can use the signed angle of the pendulum from the vertical θ (in radians; positive is counterclockwise, to the left of the vertical) and its angular velocity ω ($\theta' = \omega$). The force of gravity can only act tangent to the pendulum, so its effect on the angular velocity should be proportional to the sine of θ. More specifically, the angular acceleration, ω', is the linear acceleration a divided by the length of the pendulum l (Fig. 28). So the formula for the angular acceleration is $\omega' = \frac{g}{l} sin(\theta)$, where g is the gravitational constant; but we can treat $\frac{g}{l} = k$ as a constant to simplify the math. Finally, the restitution force, which we will model as instantaneous, will have the effect of instantaneously decreasing (or increasing, if $\theta < 0$) the angular velocity by some constant amount $\Delta\omega$.

Putting it all together, we only need to choose values for the constants and initial values for the variables. For simplicity, I chose $k = 1$ and then experimented with different $\Delta\omega$ values, eventually settling on $\Delta\omega = 0.25$. As for the initial values, the pendulum starts stationary ($\omega = 0$) at a slight deviation from the equilibrium, $\theta = 0.1$.

```
1  theta:=0.1; w:=0;
2  while theta < pi/2 && theta > -pi/2 do {
3    if theta < -pi/4 then w := w + 0.25;
4    if theta > pi/4 then w := w - 0.25;
5    theta'=w, w'=sin(theta) for 0.05;
6  }
```

Listing 4.5: Jaguar program that models inverted pendulum balancing with instantaneous restitution

All that is left is to choose the appropriate simulation parameters. Since all for statements in the

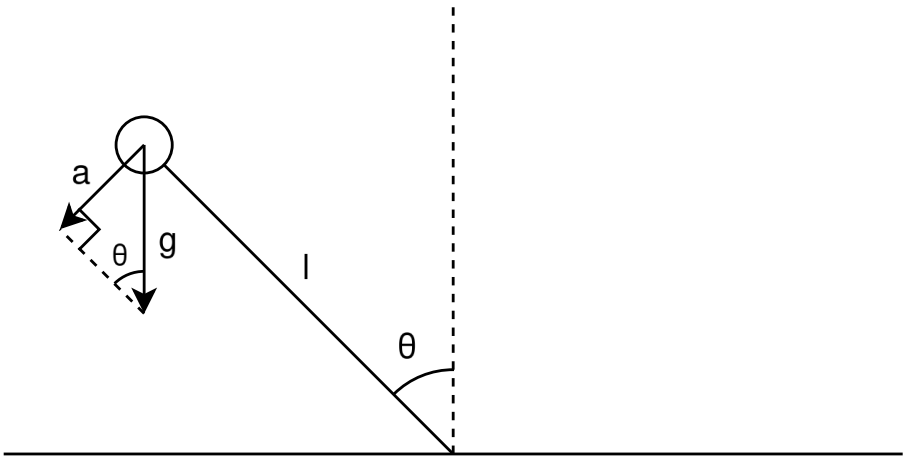


Figure 28: Diagram of an inverted pendulum, labeled with the relevant physical quantities

program have a duration of 0.05 there is no risk of zeno behaviour, so we can safely run the program with no iteration limit. Moreover, a query accuracy of 8 is sufficient to limit the queries to at most two hybrid branches regardless of the query time ($2^{2-8} = 0.015625 < 0.05/2$). Like before, when querying a multiple of 0.05 the second branch is the correct one. As for the comparison accuracy, there isn't a definitive best value here; as long as the value of θ isn't too close to $\pi/2$ and $\pi/4$ at the time of comparison, the program runs normally. I arbitrarily chose $n = 8$, which seems to be enough for the graphs and tests presented below.

So how does this program behave? Fig. 29 is a plot of the first 40 time units of the evolution of the system obtained with Jaguar using *aern2-real*. The exact command that generated it can be found in the appendix in Appendix B. The system behaves as expected: the pendulum starts by falling freely to the left until around $t = 3$, when θ becomes greater than $\pi/4$ and the restitution force kicks in, creating the discontinuities in the angular velocity. After seven of these, θ is once again below $\pi/4$ and the pendulum swings freely again. It ends up not making it over the top and falls again. After seven more boosts, the pendulum is again travelling upwards and this time manages to go over the top to the right side. This repeats many more times; most times the pendulum goes over the top, sometimes faster, others slower. At $t = 40$ it is near the top starting to fall to the left once more.

We can also run the program using floating-point arithmetic instead of exact real arithmetic. Fig. 30 is the output of the program with exactly the same parameters except for the numerical type. The two graphs are identical until $t = 35$, when the floating point output is ever so slightly above the *aern2-real*'s version. After that, they completely separate, with the floating point pendulum falling much faster to the left. At $t = 40$, the floating point pendulum is at the lowest point of an arc, already receiving upward

boosts. These differences are, of course, caused by errors in floating-point arithmetic introduced at each operation. Though present from the start, the differences only accumulate enough to become visible at around $t = 35$; after that, the chaotic nature of the system pushes the two slightly different trajectories in distinct directions.

We can also compare these results to the output of Lince for the same program, shown in Fig. 31. It diverges from the other two much sooner, at $t = 27$, where the pendulum falls back left instead of going over the top as in Jaguar. Even though Lince also uses double-precision floating point, it uses the Runge-Kutta method to evaluate differential statements, as opposed to Jaguar's approach with forward automatic differentiation and polynomial ODEs' error bounds; so it is not surprising Lince's results diverge sooner.

Sadly, *aern2-real* is the only library that can run this program until the output diverges from the floating-point runs. As we saw before, the modulus of convergence libraries are naturally slower at running hybrid programs (with *ERA* being even worse for not having memoization); and *CDAR*, the other nested intervals library, runs out of memory (8GB minus operating system) for higher t's and accuracies. For $n = 32$, it manages to run until $t = 10$ before exhausting the memory. I suspect this is caused by *CDAR*'s implementation of sine, which is woefully slower for negative angles (Fig. 20); so when the pendulum swings to the right, *CDAR* needs many more computational resources. *aern2-real* itself also runs out of memory at around $t = 46$, but until then manages to calculate approximations up to an accuracy of 50.

As for the runtimes, shown in Fig. 32, all three versions behave approximately linearly with respect to t, though, of course, doubles have a much smaller constant factor. Between the two exact real arithmetic libraries, *CDAR* appears to be about five times faster than *aern2-real* until $t = 10$, where it has a spike, and fails to run the last two test cases ($t = 18$ and $t = 32$). A similar spike can be seen for *aern2-real* at $t = 32$.

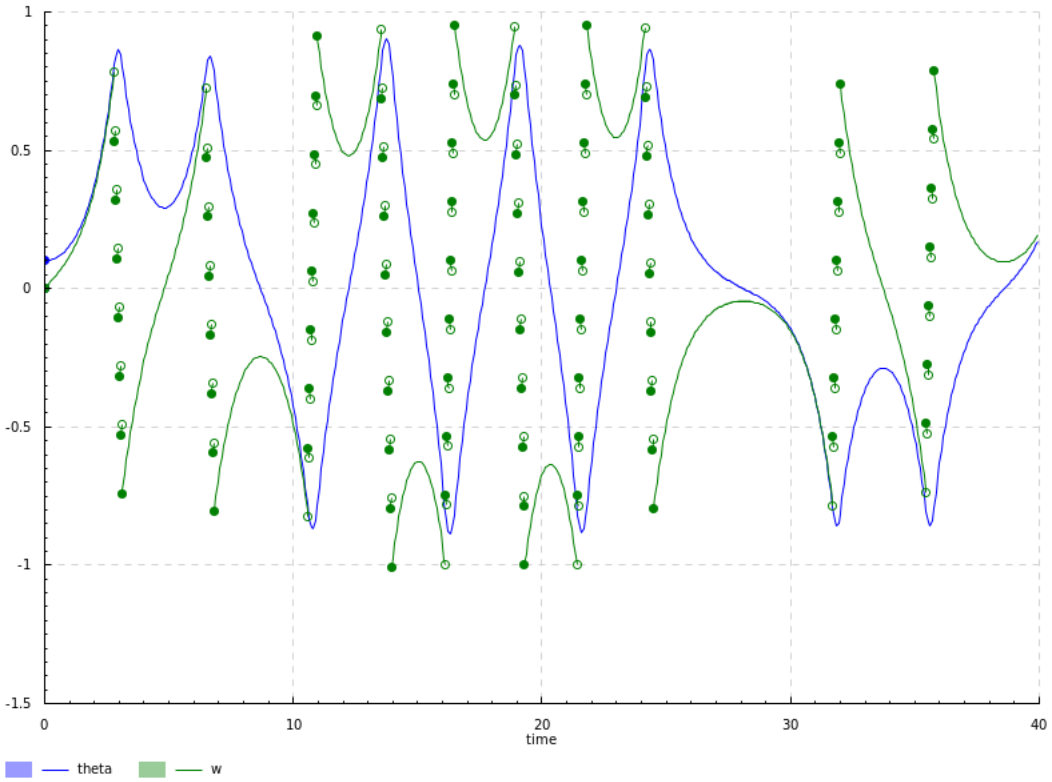


Figure 29: Plot of inverted pendulum program run with *aern2-real* ($n = 10$)

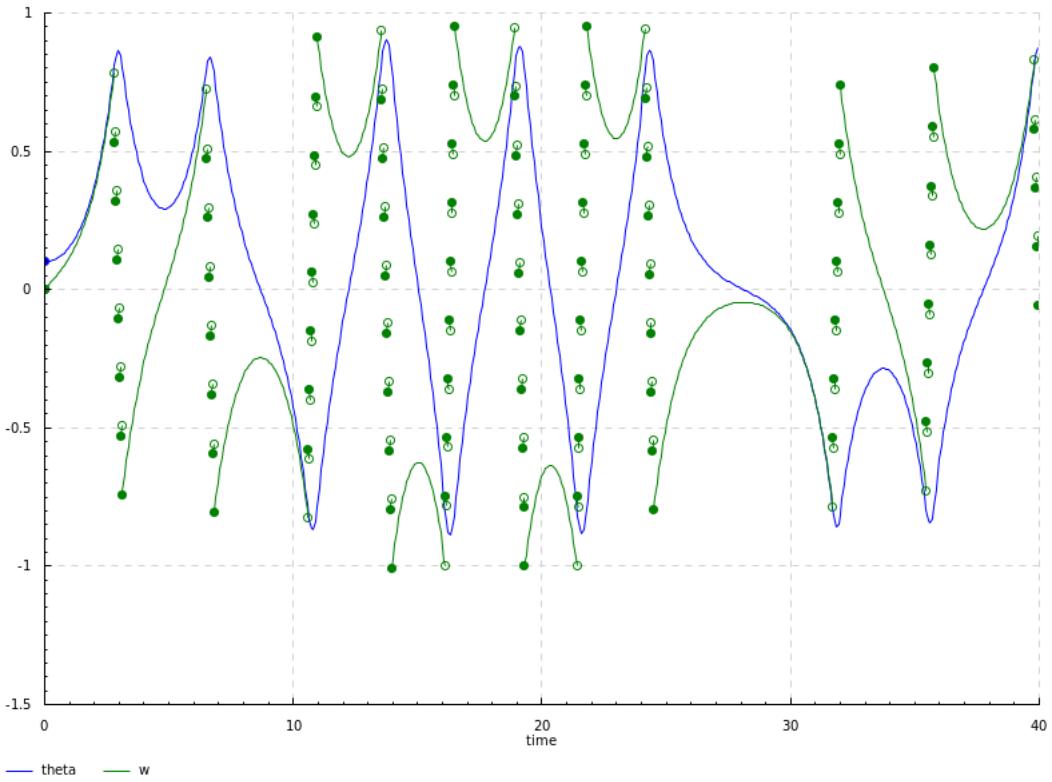


Figure 30: Plot of inverted pendulum program run with doubles

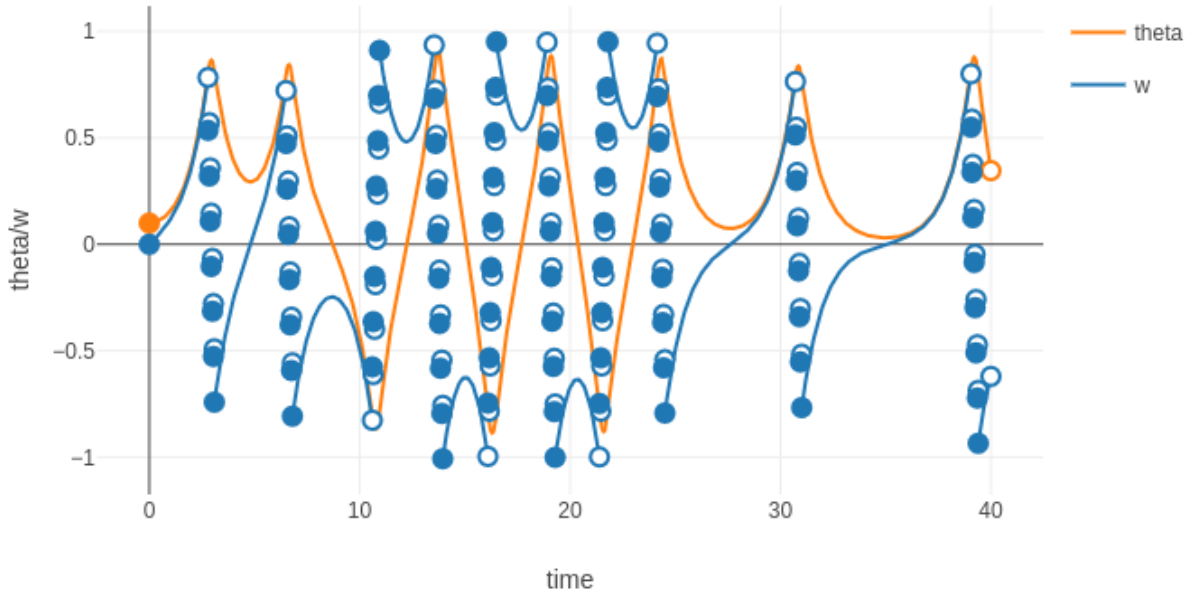


Figure 31: Plot of inverted pendulum program run in Lince

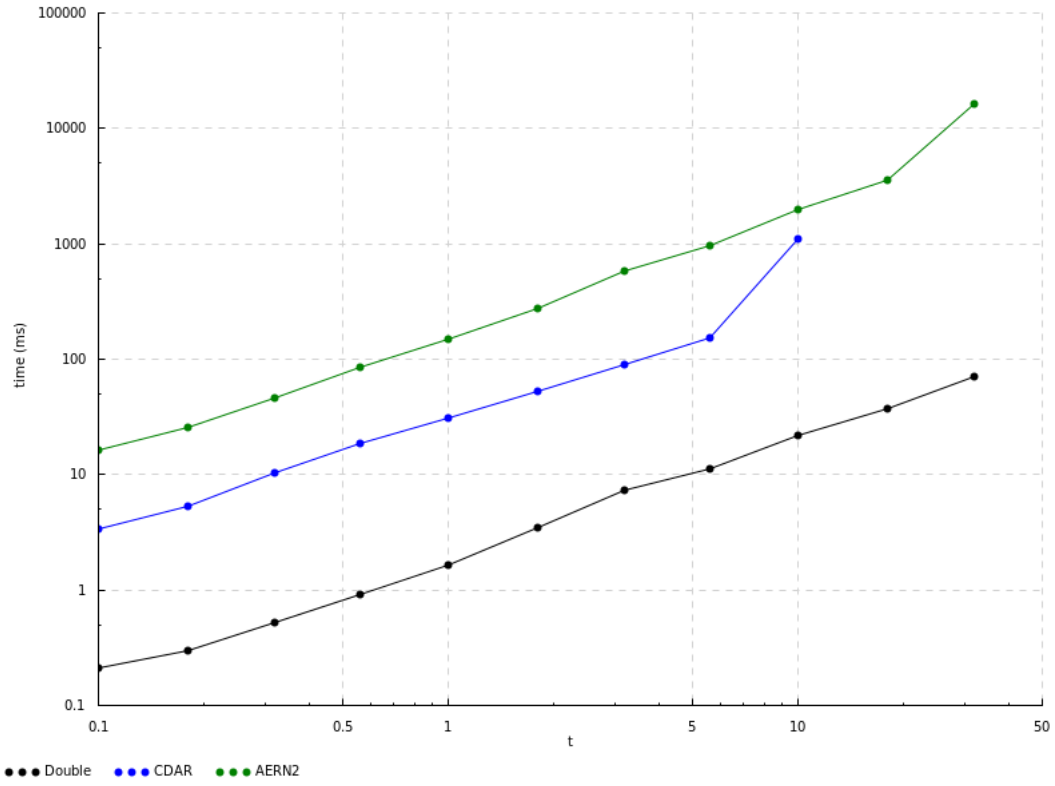


Figure 32: Benchmarks for inverted pendulum program ($n = 32$), scaled logarithmically

Chapter 5

Conclusion and Future Work

Throughout this dissertation, we analyzed two main strategies used to represent real values computationally: nested intervals and modulus of convergence. As we have seen, the two strategies have different advantages and disadvantages. Modulus of convergence representation is a more direct application of their shared mathematical foundation – namely, Cauchy sequences – while nested intervals representation, also based on domain theory, has better performance. We then studied hybridness and hybrid systems in order to build Jaguar, a hybrid simulator that uses exact real arithmetic for its calculations. After exploring its inner workings and functionalities, we tested the various exact real arithmetic libraries, both by themselves and with the hybrid simulator. Of the 5 libraries, only the two using nested intervals representation were viable when running a case study of an inverted pendulum, with *aern2-real* being able to run for long enough to notice discrepancies from a simulation using double precision floating-point numbers.

Many paths of investigation are still unexplored. Other representation strategies, such as continued fractions [38] and Dedekind cuts [39], have also been suggested as basis for exact real arithmetic. Though these other strategies do not have readily available Haskell libraries at time of writing (which is why they were not included in this dissertation), it would still be interesting to expand the analysis done here to these other methods, both to their theory and practical implementations.

As for the hybrid simulator, Jaguar, there is still plenty of room for improvement. An especially important component to improve is the differential equation solver, considering how crucial it is for the simulator's performance. The most direct way to optimize the solver is to symbolically solve the equations instead of using the Taylor method, completely circumventing the need for automatic differentiation and limits and massively reducing the number of arithmetic operations of each differential statement. Of course, not all differential equations are symbolically solvable, so the current approach could still be used in those cases. Some hybrid programs might also benefit from compiler-like optimizations on all arithmetic expressions (e.g. factoring out and reusing common subexpressions), since exact real arithmetic is

highly sensitive to the quantity, order and associativity of operations. The mechanism to evaluate boolean expressions could also be improved by symbolically analyzing boolean expressions. Adding this analysis to the interpreter could be an easy way to improve Jaguar's performance and even avoid runtime errors.

Besides optimizations, the simulator could also be improved by adding other features and language constructs, such as Lince's events (which are for statements that run for as long as a condition is true) or some equivalent specific to computable real numbers. Currently, there is also no mechanism to detect and manage arithmetic errors such as division by zero. This problem is especially difficult to manage in exact real arithmetic due to the semidecidable comparison, which makes it impossible to simply check if the divisor is zero. Even worse, evaluating the result of a division by zero might not even terminate, as the division uses smaller and smaller approximations of the divisor, driving the value of the result infinitely high.

Another important area requiring future study is memory usage, which is mostly disregarded in this dissertation but would be of critical importance in an effort to scale up programs and computations. The current technologies of exact real arithmetic would greatly benefit from a memory usage analysis similar to the execution time analysis presented in Section 4.1. The simulator itself should also be optimized for memory efficiency in the future so it can handle more complicated hybrid programs without running out of memory, a sad fate my 8GB laptop often encountered during development and testing.

In conclusion, despite being simple and flexible in application, exact real arithmetic is still too performance heavy, being best employed in specific situations for small but critical calculations. This makes exact real arithmetic, and Jaguar in particular, useful only for formal verifications and theoretical problems, while being too slow for "real world" applications. Nevertheless, this dissertation shows how this technology can be used and hopefully serves as the groundwork for more complex applications in the future.

Bibliography

[1] Renato Neves. *Hybrid programs*. PhD thesis, Minho University, 2018.

[2] Sergey Goncharov, Renato Neves, and José Proença. Implementing hybrid semantics: From functional to imperative. In *Theoretical Aspects of Computing–ICTAC 2020: 17th International Colloquium, Macau, China, November 30–December 4, 2020, Proceedings 17*, pages 262–282. Springer, 2020.

[3] Ieee standard for floating-point arithmetic. *IEEE Std 754-2019 (Revision of IEEE 754-2008)*, pages 1–84, 2019. doi: 10.1109/IEEESTD.2019.8766229.

[4] Tiago Bacelar. Jaguar. https://github.com/tiago-bacelar/jaguar, 2026. Accessed: 2026-03-25.

[5] John Whitington. *Haskell from the very beginning*. Coherent Press, Birmingham, England, 2019. ISBN 9780957671133.

[6] Simon Thompson. *Haskell: The craft of Functional Programming*. 1996. ISBN 9780201882957.

[7] Herman Geuvers, Milad Niqui, Bas Spitters, and Freek Wiedijk. Constructive analysis, types and exact real numbers. *Mathematical Structures in Computer Science*, 17(1):3–36, 2007.

[8] J. C. Butcher. *Numerical methods for ordinary differential equations*. Wiley, 2016. ISBN 9781119121503.

[9] Louis B. Rall. *Automatic Differentiation: Techniques and Applications*. Springer, 1981. ISBN 9783540108610. doi: 10.1007/3-540-10861-0. URL https://doi.org/10.1007/3-540-10861-0.

[10] P.G. Warne, D.A. Polignone Warne, J.S. Sochacki, G.E. Parker, and D.C. Carothers. Explicit a-priori error bounds and adaptive error control for approximation of nonlinear initial value differential systems. *Computers & Mathematics with Applications*, 52(12):1695–1710, 2006. ISSN 0898-1221.

doi: https://doi.org/10.1016/j.camwa.2005.12.004. URL https://www.sciencedirect.com/science/article/pii/S089812210600352X.

[11] Chart. https://hackage.haskell.org/package/Chart, 2025. Accessed: 2025-12-15.

[12] T.M. Apostol. *Calculus, Volume 1*. Wiley, 1991. ISBN 9780471000051. URL https://books.google.pt/books?id=o2D4DwAAQBAJ.

[13] Eugenio Moggi. Notions of computation and monads. *Information and computation*, 93(1):55–92, 1991.

[14] A. M. Turing. On computable numbers, with an application to the entscheidungsproblem. *Proceedings of the London Mathematical Society*, s2-42(1):230–265, 1937. doi: https://doi.org/10.1112/plms/s2-42.1.230. URL https://londmathsoc.onlinelibrary.wiley.com/doi/abs/10.1112/plms/s2-42.1.230.

[15] Laurent Fousse, Guillaume Hanrot, Vincent Lefèvre, Patrick Pélissier, and Paul Zimmermann. Mpfr: A multiple-precision binary floating-point library with correct rounding. *ACM Trans. Math. Softw.*, 33(2):13–es, June 2007. ISSN 0098-3500. doi: 10.1145/1236463.1236468. URL https://doi.org/10.1145/1236463.1236468.

[16] Norbert Th. Müller. The irram: Exact arithmetic in c++. In Jens Blanck, Vasco Brattka, and Peter Hertling, editors, *Computability and Complexity in Analysis*, pages 222–252, Berlin, Heidelberg, 2001. Springer Berlin Heidelberg. ISBN 978-3-540-45335-2.

[17] Glynn Winskel. *The formal semantics of programming languages*. MIT press, 1993. ISBN 9780262529433.

[18] S. C. Kleene. On notation for ordinal numbers. *Journal of Symbolic Logic*, 3(4):150–155, 1938. doi: 10.2307/2267778.

[19] Andrej Bauer and Iztok Kavkler. A constructive theory of continuous domains suitable for implementation. *Annals of Pure and Applied Logic*, 159(3):251–267, 2009. ISSN 0168-0072. doi: https://doi.org/10.1016/j.apal.2008.09.025. URL https://www.sciencedirect.com/science/article/pii/S0168007208001589. Joint Workshop Domains VIII — Computability over Continuous Data Types, Novosibirsk, September 11–15, 2007.

[20] Nicholas J. Higham. *Accuracy and Stability of Numerical Algorithms*. Society for Industrial and Applied Mathematics, second edition, 2002. ISBN 9780898715217. URL https://epubs.siam.org/doi/abs/10.1137/1.9780898718027.

[21] David Harvey and Joris Van Der Hoeven. Integer multiplication in time $O(n \log n)$. *Annals of Mathematics*, 193(2), 3 2021. doi: 10.4007/annals.2021.193.2.4. URL https://doi.org/10.4007/annals.2021.193.2.4.

[22] P. Lancaster. Error analysis for the Newton-Raphson method. *Numerische Mathematik*, 9(1):55–68, 11 1966. doi: 10.1007/bf02165230. URL https://doi.org/10.1007/bf02165230.

[23] Piergiorgio Odifreddi. *Classical recursion theory : the theory of functions and sets of natural numbers*. North-Holland, 1999. ISBN 9780444502056.

[24] David Lester. Era. https://hackage.haskell.org/package/numbers-3000.2.0.2/docs/Data-Number-CReal.html, 2025. Accessed: 2025-01-08.

[25] Ellie Hermaszewska. exact-real. https://github.com/expipiplus1/exact-real, 2025. Accessed: 2025-01-08.

[26] Björn von Sydow. ireal. https://github.com/sydow/ireal, 2025. Accessed: 2025-01-08.

[27] J. Blanck. Exact real arithmetic using centred intervals and bounded error terms. *The Journal of Logic and Algebraic Programming*, 66(1):50–67, 2006. ISSN 1567-8326. doi: https://doi.org/10.1016/j.jlap.2005.07.002. URL https://www.sciencedirect.com/science/article/pii/S1567832605000548.

[28] Jens Blanck. Cdar. https://github.com/jensblanck/cdar, 2025. Accessed: 2025-01-08.

[29] Michal Konecny. aern2-real. https://github.com/michalkonecny/aern2, 2025. Accessed: 2025-01-08.

[30] Michal Konecny. collect-errors. https://hackage.haskell.org/package/collect-errors, 2025. Accessed: 2025-01-08.

[31] H. Witsenhausen. A class of hybrid-state continuous-time dynamic systems. *IEEE Transactions on Automatic Control*, 11(2):161–167, 1966. doi: 10.1109/TAC.1966.1098336.

[32] Calvin C. Elgot. Monadic computation and iterative algebraic theories. In H.E. Rose and J.C. Shepherdson, editors, *Logic Colloquium '73*, volume 80 of *Studies in Logic and the Foundations of Mathematics*, pages 175–230. Elsevier, 1975. doi: https://doi.org/10.1016/S0049-237X(08)71949-9. URL https://www.sciencedirect.com/science/article/pii/S0049237X08719499.

[33] happy. https://hackage.haskell.org/package/happy, 2025. Accessed: 2025-12-17.

[34] Grzegorz Malinowski. Kleene logic and inference. *Bulletin of the Section of Logic*, 43, 01 2014.

[35] Peter Höfner and Bernhard Möller. An algebra of hybrid systems. *The Journal of Logic and Algebraic Programming*, 78(2):74 – 97, 2009. ISSN 1567-8326.

[36] Sergey Goncharov and Renato Neves. An adequate while-language for hybrid computation. In *Proceedings of the 21st International Symposium on Principles and Practice of Programming Languages 2019*, pages 1–15, 2019.

[37] criterion. https://hackage.haskell.org/package/criterion, 2025. Accessed: 2026-01-27.

[38] A. I. Khinchin. *Continued fractions*. 1961. ISBN 9780486696300.

[39] Richard Dedekind and W. W. Beman. *Essays on the Theory of Numbers*. Dover Publications, 1963. ISBN 9780486210100.

Part I
Appendices

Appendix A
Jaguar Syntax

This appendix contains Jaguars' syntax, in BNF notation.

```
Program ::= var ":=" Expr
          | For "for" Expr
          | For "forever"
          | "wait" Expr
          | "wait" "forever"
          | "if" BExpr "then" Program
          | "if" BExpr "then" Program "else" Program
          | Program ";" Program
          | Program ";"
          | "{" Program "}"
          | "{" "}"

For     ::= DifEq
          | For "," DifEq

DifEq   ::= var "'" "=" Expr

BExpr   ::= BTerm
          | "!" BExpr
          | BExpr "&&" BExpr
          | BExpr "||" BExpr
          | "(" BExpr ")"

BTerm   ::= "true"
          | "false"
          | Expr comp Expr

Expr    ::= var
          | int
          | num
          | "e"
          | "pi"
          | "abs"  "(" Expr ")"
          | "sqrt" "(" Expr ")"
          | "exp"  "(" Expr ")"
          | "ln"   "(" Expr ")"
          | "sin"  "(" Expr ")"
          | "cos"  "(" Expr ")"
          | "tan"  "(" Expr ")"
          | Expr "+" Expr
          | Expr "-" Expr
          | Expr "*" Expr
          | Expr "/" Expr
          | Expr "^" int
```

```
45            | Expr "^" Expr
46            | "min"  "(" Expr "," Expr ")"
47            | "max"  "(" Expr "," Expr ")"
48            | "log"  "(" Expr "," Expr ")"
49            | "-" Expr
50            | "(" Expr ")"
```

Listing A.1: Jaguar's syntax in BNF notation

Appendix B
Jaguar Run Instructions

This appendix contains installation instructions for Jaguar, as well as the commands used to run the Jaguar programs in the main document.

To install Jaguar, first install the Stack build tool for Haskell (haskellstack.org). Then clone the Jaguar repository from GitHub (https://github.com/tiago-bacelar/jaguar) and build the project with Stack:

```
stack install
```

You should now be able to run Jaguar from the command line. Jaguar takes an input file with a Jaguar program and runs it using the specified command line arguments. Run `jaguar --help` to see all available arguments.

Before running the following commands the corresponding Jaguar code should be put into a file *input.jag* in the current directory.

Bouncing Ball (Section 3.6)

```
jaguar -n ireal -a 4 -c 10 -t 8 -v y,v input.jag
```

Listing B.1: Command used to generate Fig. 9

```
jaguar -n ireal -a 10 -c 10 -t 8 -v y,v input.jag
```

Listing B.2: Command used to generate Fig. 10

Inverted Pendulum (Section 4.3)

```
jaguar -n aern2 -a 8 -c 8 -t 40 input.jag
```

Listing B.3: Command used to generate Fig. 29

```
jaguar -n double -a 8 -c 8 -t 40 input.jag
```

Listing B.4: Command used to generate Fig. 30

Appendix C
Benchmarks Source Code

This appendix contains the Haskell code used for the benchmarks presented in Chapter 4. Tests were compiled with the flag -fno-full-laziness to prevent accidental sharing between test runs.

fromRational (Fig. 11)

```
\n -> approx (fromRational (22%7)) n
```

fromInteger (Fig. 12)

```
\n -> approx (fromInteger 1) n
```

Dyadic fromRational (Fig. 13)

```
\n -> approx (fromRational (9%64)) n
```

cons (Fig. 14)

```
\n -> approx (cons (const 1)) n
```

cons of a series (Fig. 15)

```
\n -> approx (listCons $ scanl (+) 0 $ iterate (/2) (1%2)) n
```

limit of a series (Fig. 16)

```
\n -> approx (listLimit $ fromRational <$> scanl (+) 0 $ iterate (/2) (1%2)
    ) n
```

Sum of a list (by list size) (Fig. 17)

```
\k -> approx (foldl1' (+) [fromRational (22%7) | _ <- [1..k]]) 1000
```

Sum of a list (by accuracy) (Fig. 18)

```
\n -> approx (foldl1' (+) [fromRational (22%7) | _ <- [1..1000]]) n
```

Sum of a tree (by tree size) (Fig. 19)

```
\k -> approx (foldTree1 (+) [fromRational (22%7) | _ <- [1..k]]) 1000
```

Sine (Fig. 20)

```
\x -> approx (sin (fromRational x)) 1000
```

Memoized sum of a list (by list size) (Fig. 21)

```
\k -> approx (dp_sum (fromRational (22%7)) k) 1000
```

Memoized sum of a list (by accuracy) (Fig. 22)

```
\n -> approx (dp_sum (fromRational (22%7)) 1000) n
```